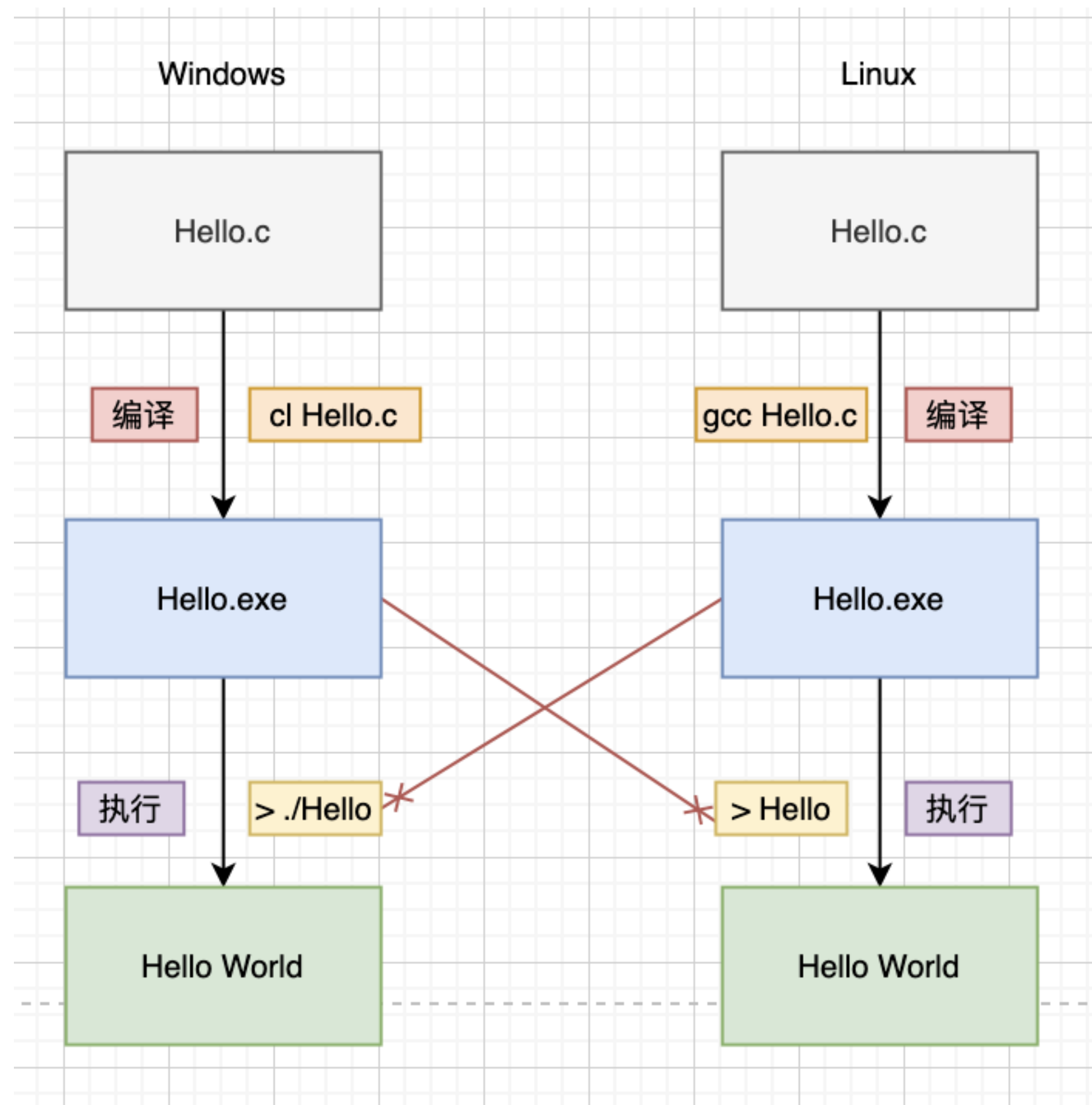


# JVM和字节码基础

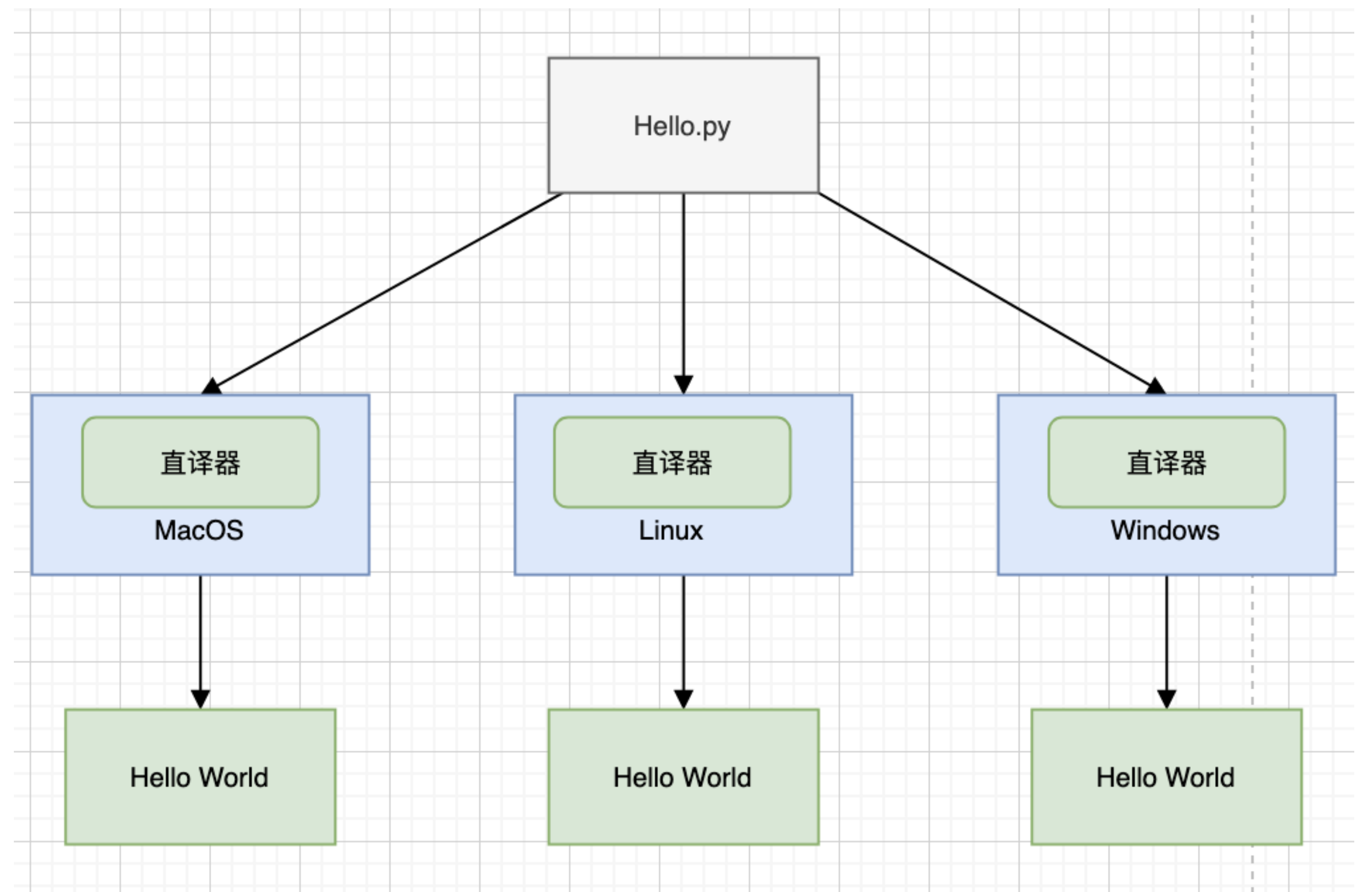
# Outline

- JVM概览
- Java类文件结构
- 深入理解运行时数据区
- 字节码指令集
- 字节码的执行
- Java指令与字节码

# 为什么需要JVM?



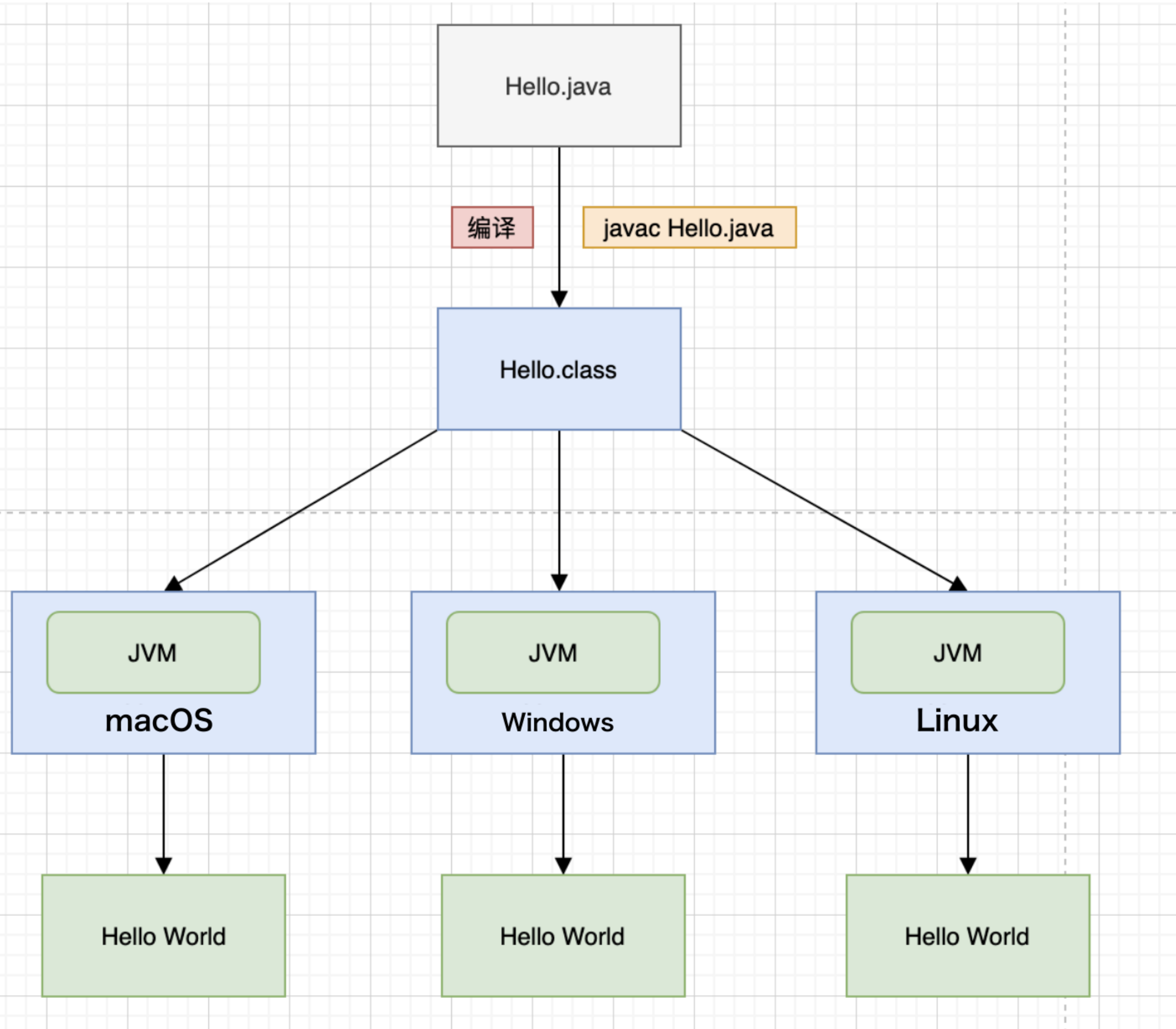
C/C++



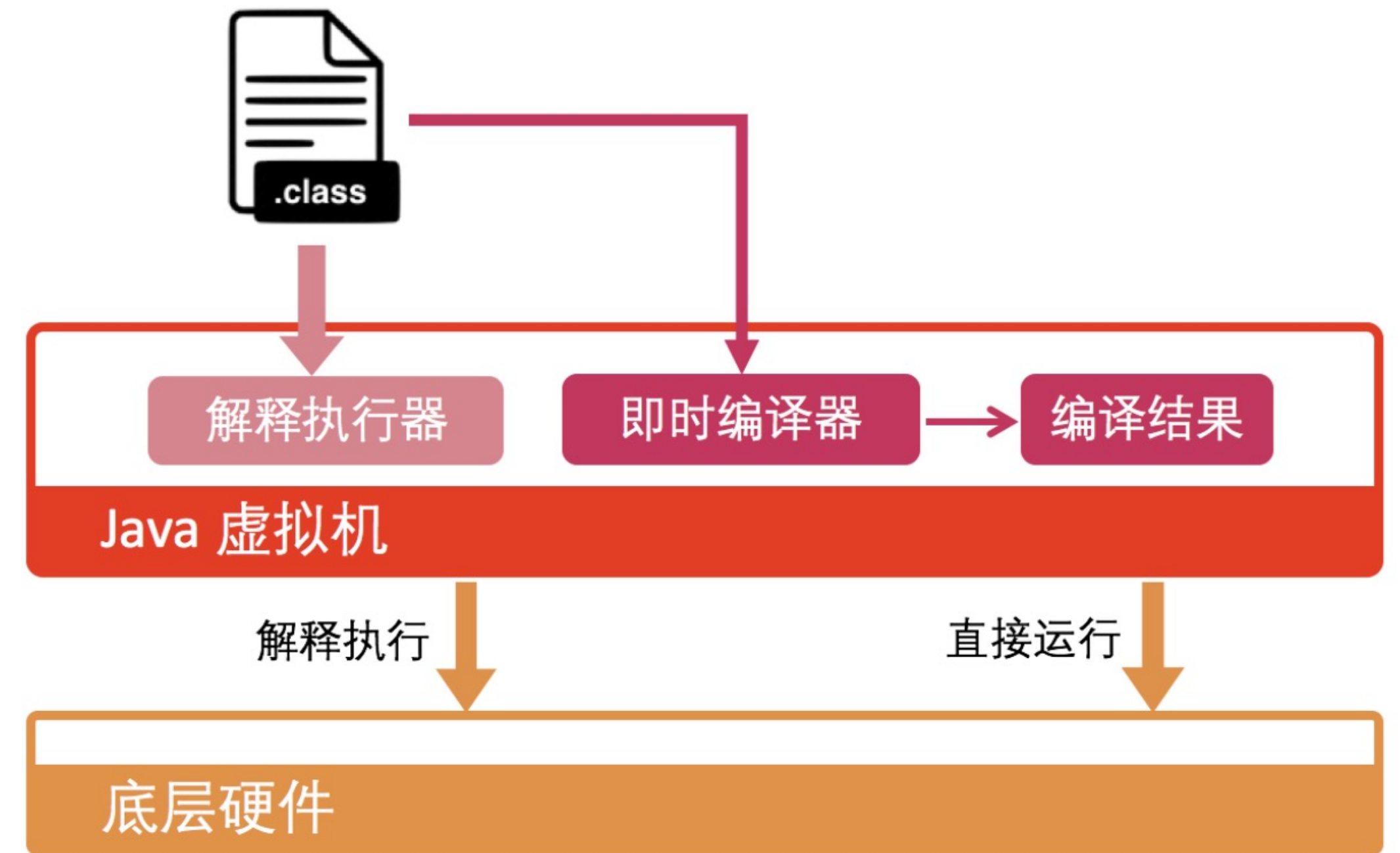
Python/Ruby



# 为什么需要JVM?



Java



Java 虚拟机 (Java Virtual Machine, JVM)

- 屏蔽平台差异
- 提供比解释器更高的性能
- 提供统一运行时环境



# JVM家族

## Java 虚拟机家族

虚拟机鼻祖：Sun Classic/Exact VM

武林盟主：HotSpot VM

小家碧玉：Mobile/Embedded VM

天下第二：BEA JRockit/IBM J9 VM

软硬合璧：BEA Liquid VM/Azul VM

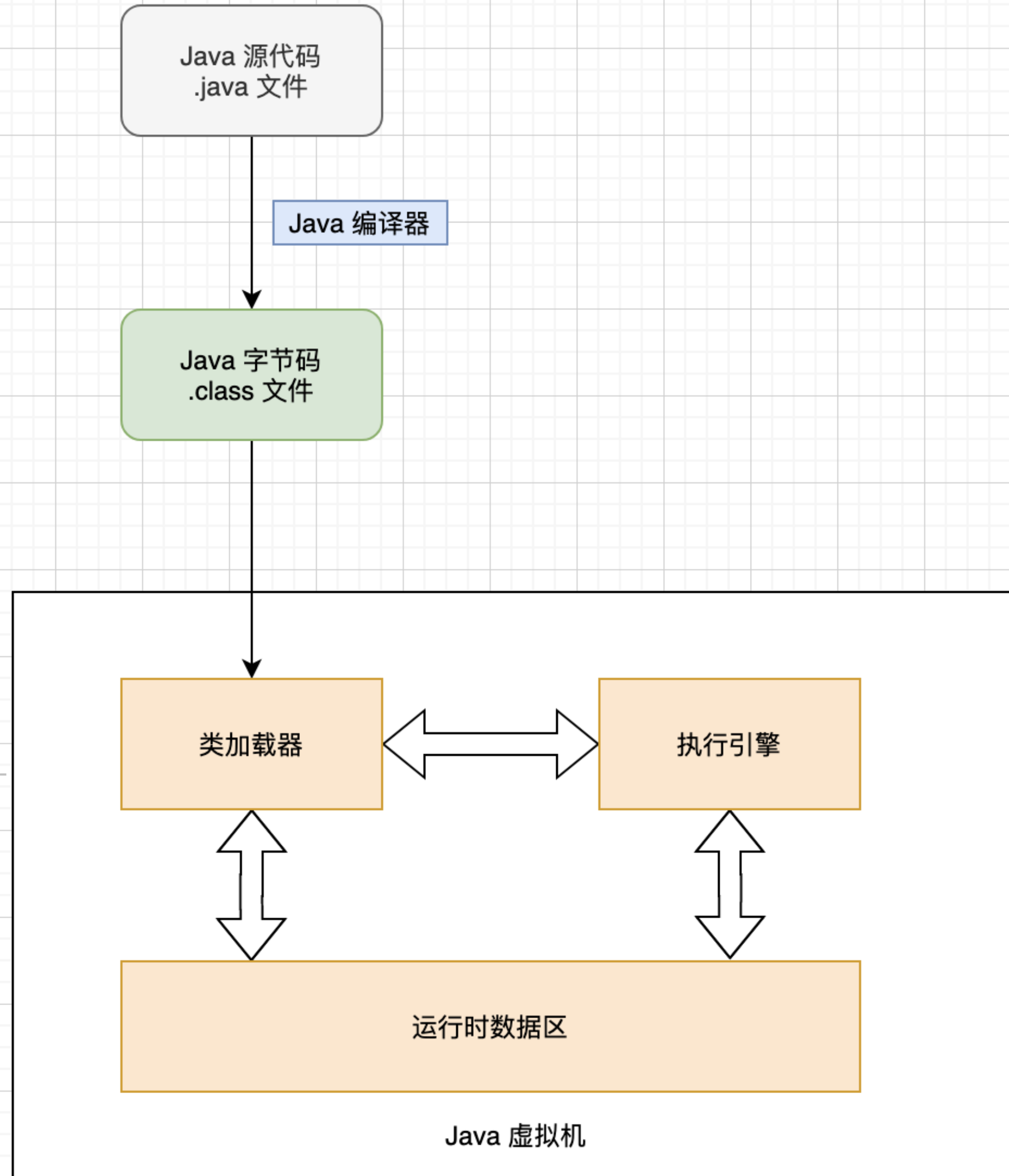
挑战者：Apache Harmony/Google Android Dalvik VM

你想不到：Microsoft JVM

## HotSpot VM

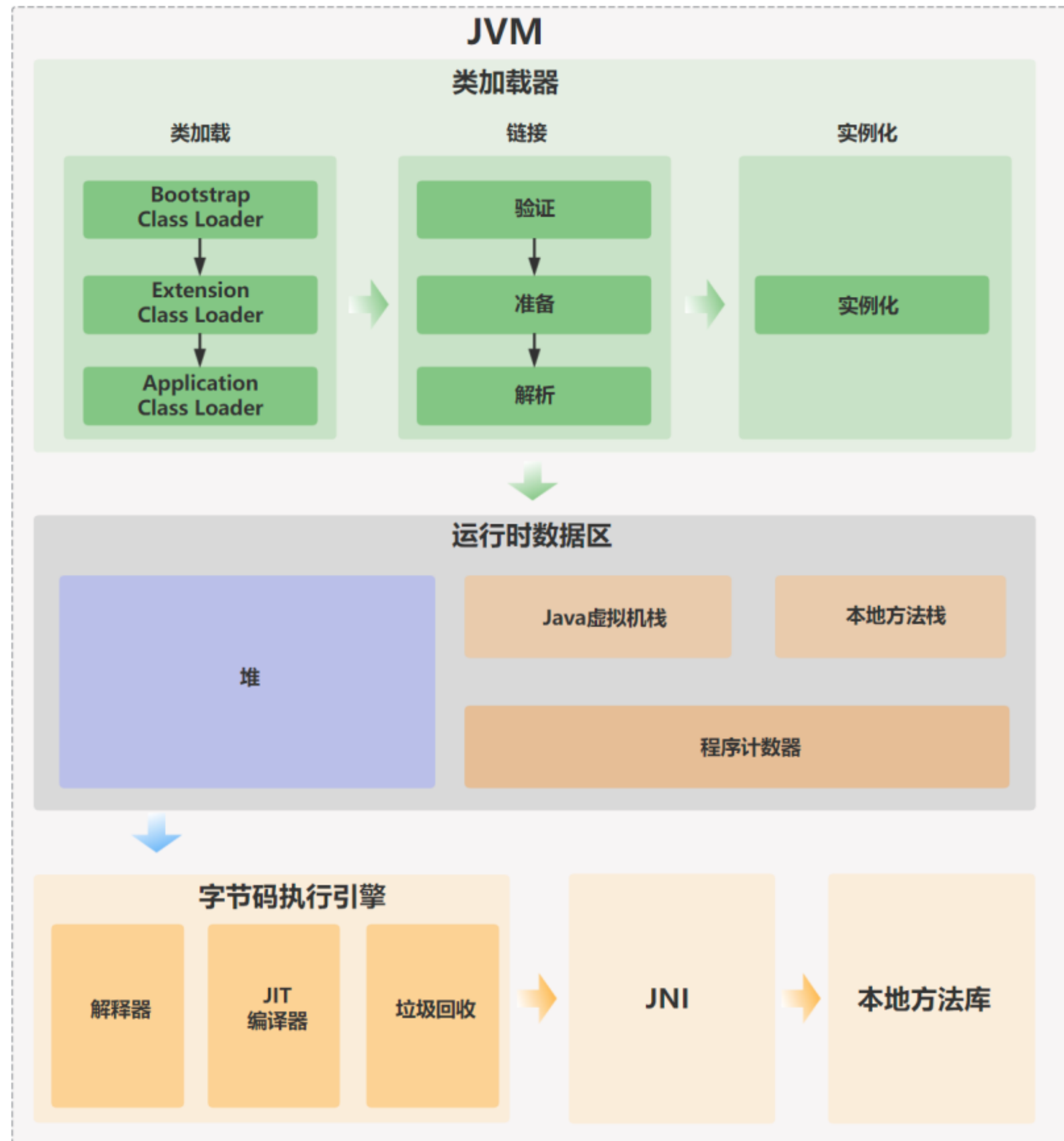
- OracleJDK（商用）和 OpenJDK（开源）的默认虚拟机，也是目前使用最广泛的 Java 虚拟机

# JVM组织架构



- JVM 大致可以划分为三个部分，分别是
- 类加载器 (Class Loader)、
  - 运行时数据区 (Runtime Data Areas)
  - 执行引擎 (Execution Engine)

# JVM组织架构



**类加载器：**JVM 需要将编译后的字节码文件加载到其内部的运行时数据区域中进行执行。Java 的类加载机制（双亲委派机制）。

**运行时数据区：**JVM 定义了 Java 程序运行期间需要使用到的内存区域，简单来说，这块内存区域存放了字节码信息以及程序执行过程的数据，垃圾收集器也会针对运行时数据区进行对象回收的工作。

**执行引擎：**将字节码指令解释/编译为对应平台上的本地机器指令以。简单来说，JVM 中的执行引擎充当了将高级语言翻译为机器语言的译者。



# 类加载

加载、验证、准备、解析、初始化、使用和卸载。

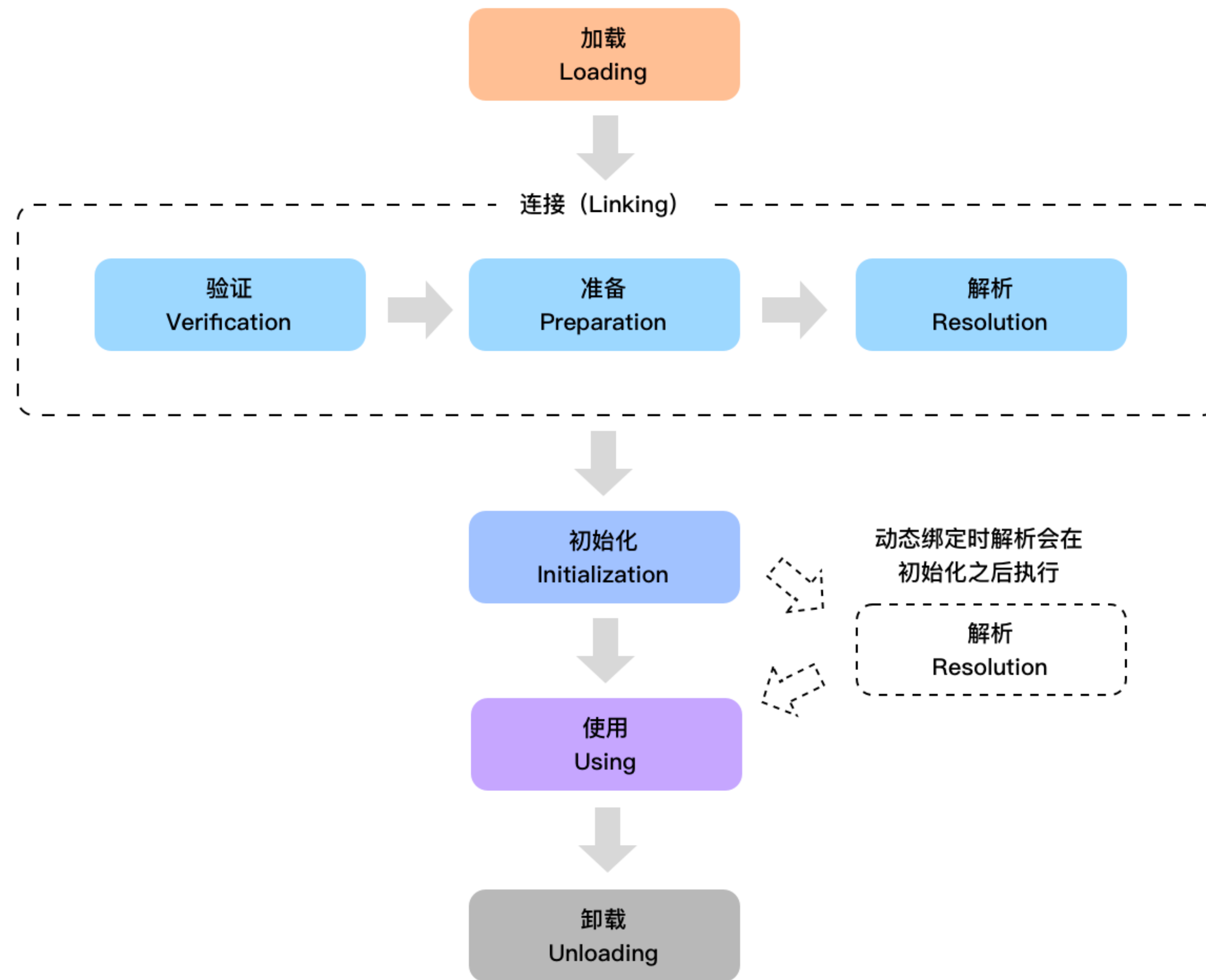
1) Loading（载入）：将字节码从不同的数据源转化为二进制字节流加载到内存中，并生成一个代表该类的 `java.lang.Class` 对象。

2) Verification（验证）：对二进制字节流进行校验

3) Preparation（准备）：对类变量（也称为静态变量，`static` 关键字修饰的）分配内存并初始化。

4) Resolution（解析）：该阶段将常量池中的符号引用转化为直接引用。

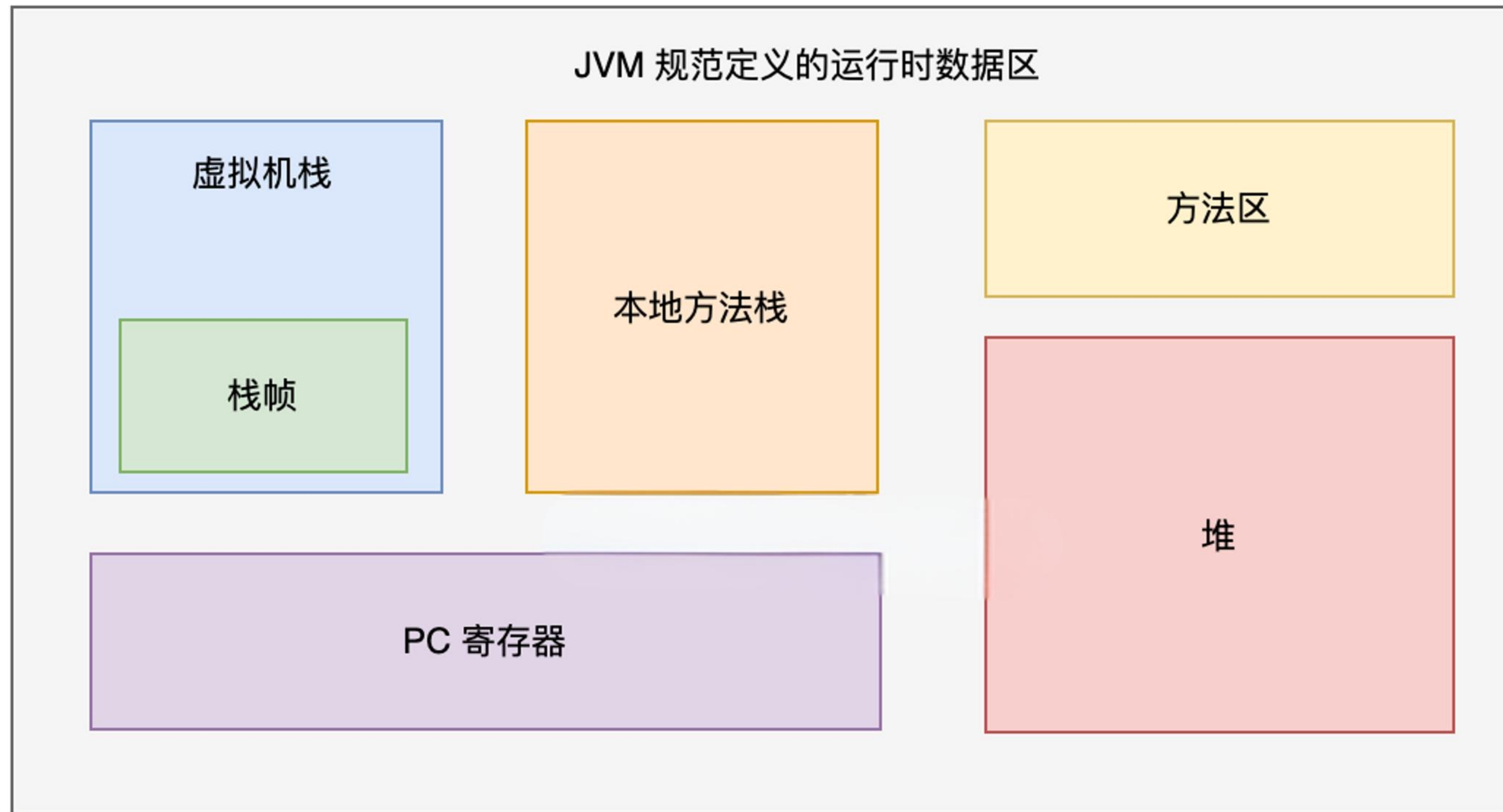
5) Initialization（初始化）：类变量将被赋值为代码期望赋的值。换句话说，执行类构造器方法的过程。





# 运行时数据区

方法区、堆、虚拟机栈、本地方法栈以及程序计数器（PC寄存器）



- PC寄存器：当前运行方法的字节码指令地址。
- 本地方法栈：运行本地方法（java代码调用其它语言）。Native方法参数、局部变量。
- **虚拟机栈（栈）**：存储方法调用和局部变量。栈帧：包括方法参数、返回地址、操作数栈等。
- **堆**：存储Java运行时所有的对象实例。对象实例、数组
- **方法区**：存储类信息、常量、静态变量等。

# 运行时数据区

| 内存区域  | 存储内容            | 线程属性 | 生命周期      | 异常类型               | 垃圾回收  |
|-------|-----------------|------|-----------|--------------------|-------|
| 堆     | 对象实例、数组         | 共享   | 随JVM启动/关闭 | OutOfMemoryError   | 是     |
| 虚拟机栈  | 局部变量、栈帧         | 私有   | 随线程创建/销毁  | StackOverflowError | 否     |
| 方法区   | 类信息、常量、静态变量     | 共享   | 随JVM启动/关闭 | OutOfMemoryError   | 是（部分） |
| 程序计数器 | 字节码指令地址         | 私有   | 随线程创建/销毁  | 无                  | 否     |
| 本地方法栈 | Native方法参数、局部变量 | 私有   | 随线程创建/销毁  | StackOverflowError | 否     |

# 运行时数据区

JDK 1.6、JDK 1.7、JDK 1.8 的内存划分都会有所不同。

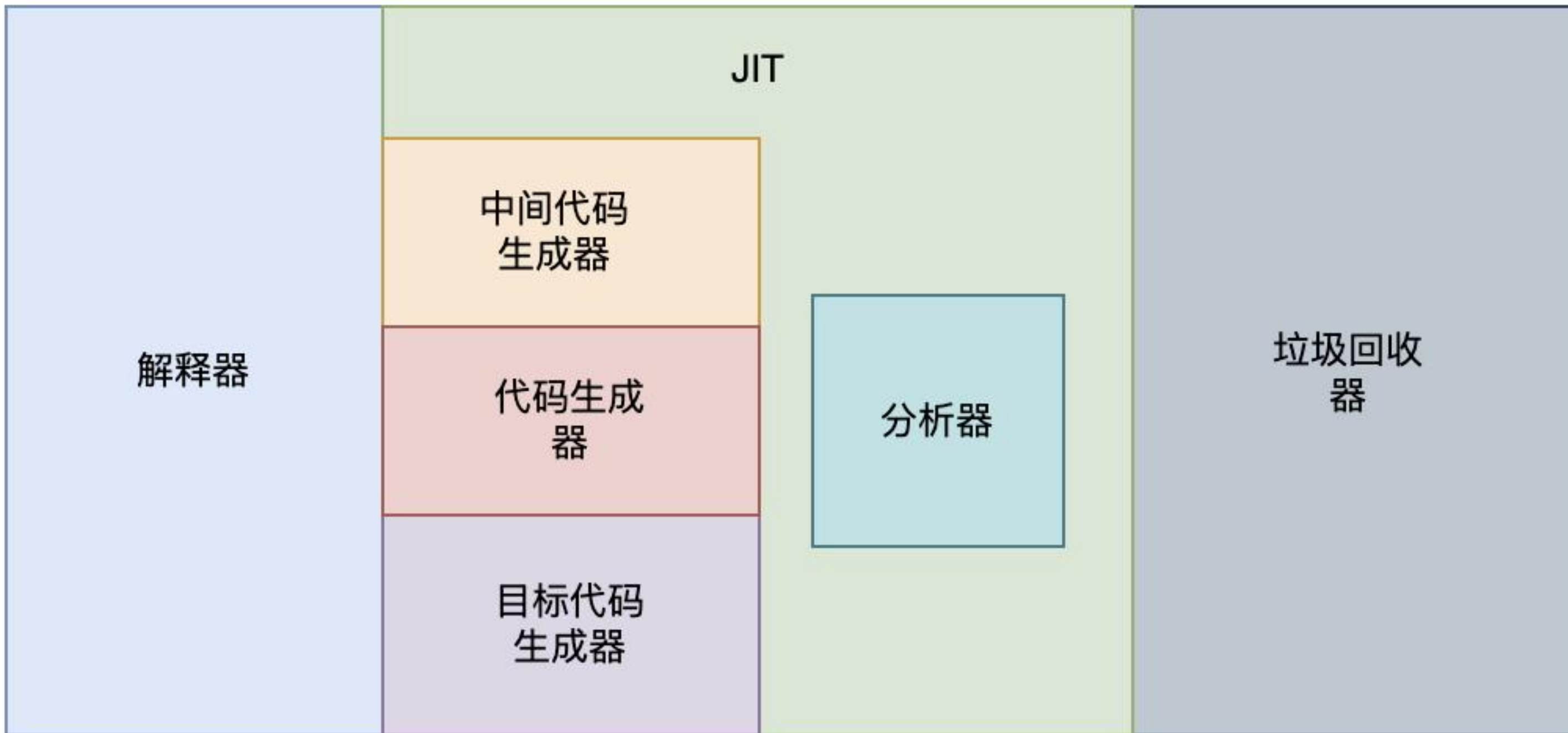


- JDK 7 之前，只有常量池的概念，都在方法区中。
- JDK 7 的时候，字符串常量池从方法区中拿出来放到了堆中，运行时常量池还在方法区中（也就是永久代中）。
- JDK 8 的时候，HotSpot 移除了永久代，取而代之的是元空间。字符串常量池还在堆中，而运行时常量池跑到了元空间。



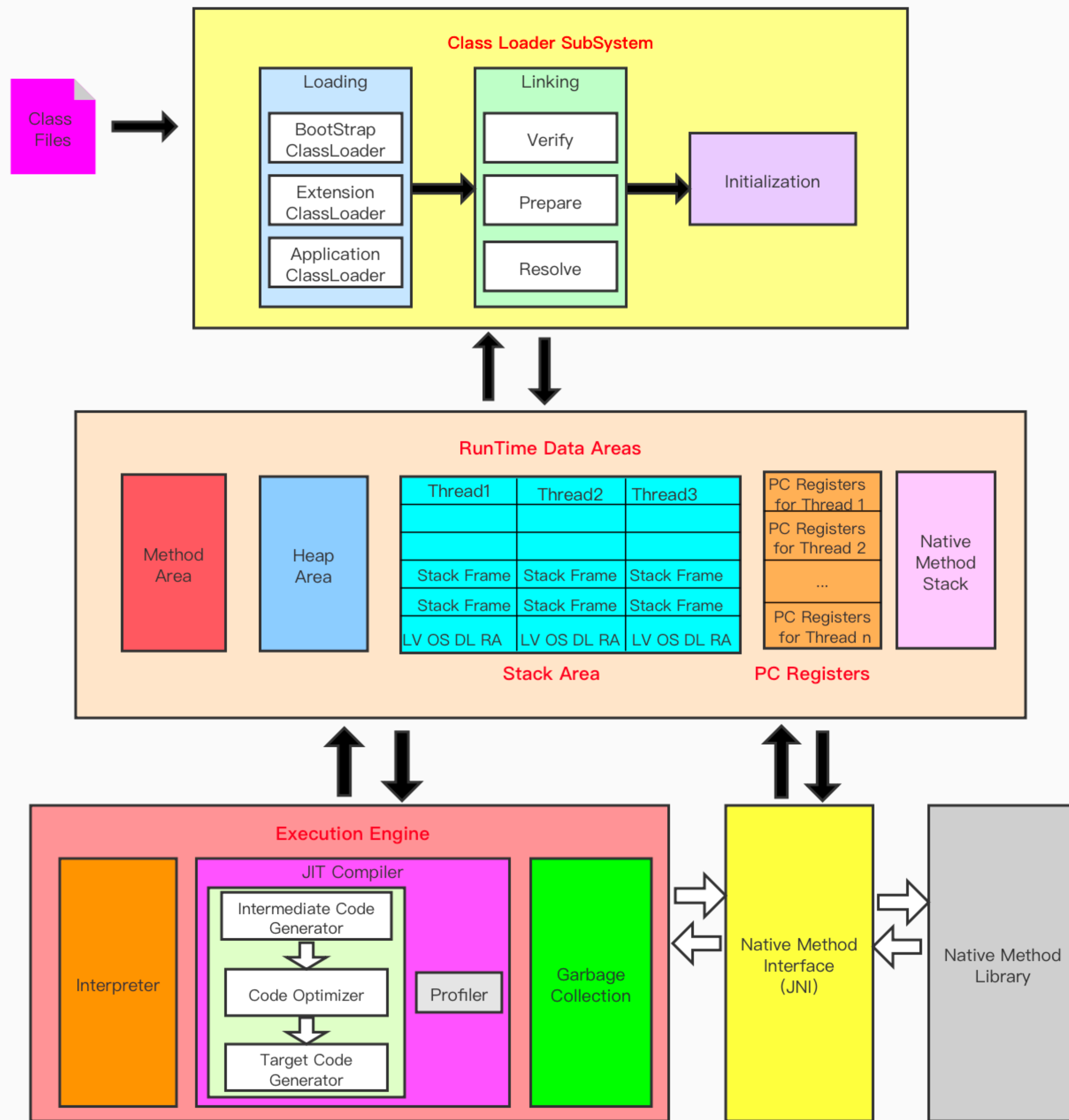
# 执行引擎

执行引擎



- 解释器：读取字节码，然后执行指令。因为它是一行一行地解释和执行指令，所以它可以很快地解释字节码，但是执行起来会比较慢（毕竟要一行执行完再执行下一行）。
- 即时编译器：执行引擎首先按照解释执行的方式来执行，随着时间推移，即时编译器会选择性的把一些热点代码编译成本地代码。执行本地代码比一条一条进行解释执行的速度快很多，因为本地代码是保存在缓存里的。
- 垃圾回收器，用来回收堆内存中的垃圾对象。





JVM架构图

# JVM架构图

# JVM如何运行Java代码？

# 编译

```
© HelloWorld.java ×  
1 package org.example;  
2  
3 public class HelloWorld {  
4     public static void main(String[] args) {  
5         System.out.println("少看手机少打游戏, 好好学习。");  
6     }  
7 }  
8
```

## 源码

Build Sync × Build Output ×



✓ **Test:** build finished At 2025/4/13 1 sec, 10 ms

Writing classes... [[Test](#)]  
Checking dependencies... [[Test](#)]  
Dependency analysis found 0 affected files  
Updating dependency information... [[Test](#)]  
Adding nullability assertions... [[Test](#)]  
Adding pattern assertions... [[Test](#)]  
Running 'after' tasks  
**javac 21.0.2 was used to compile java sources**  
Finished, saving caches...

## 生成字节码

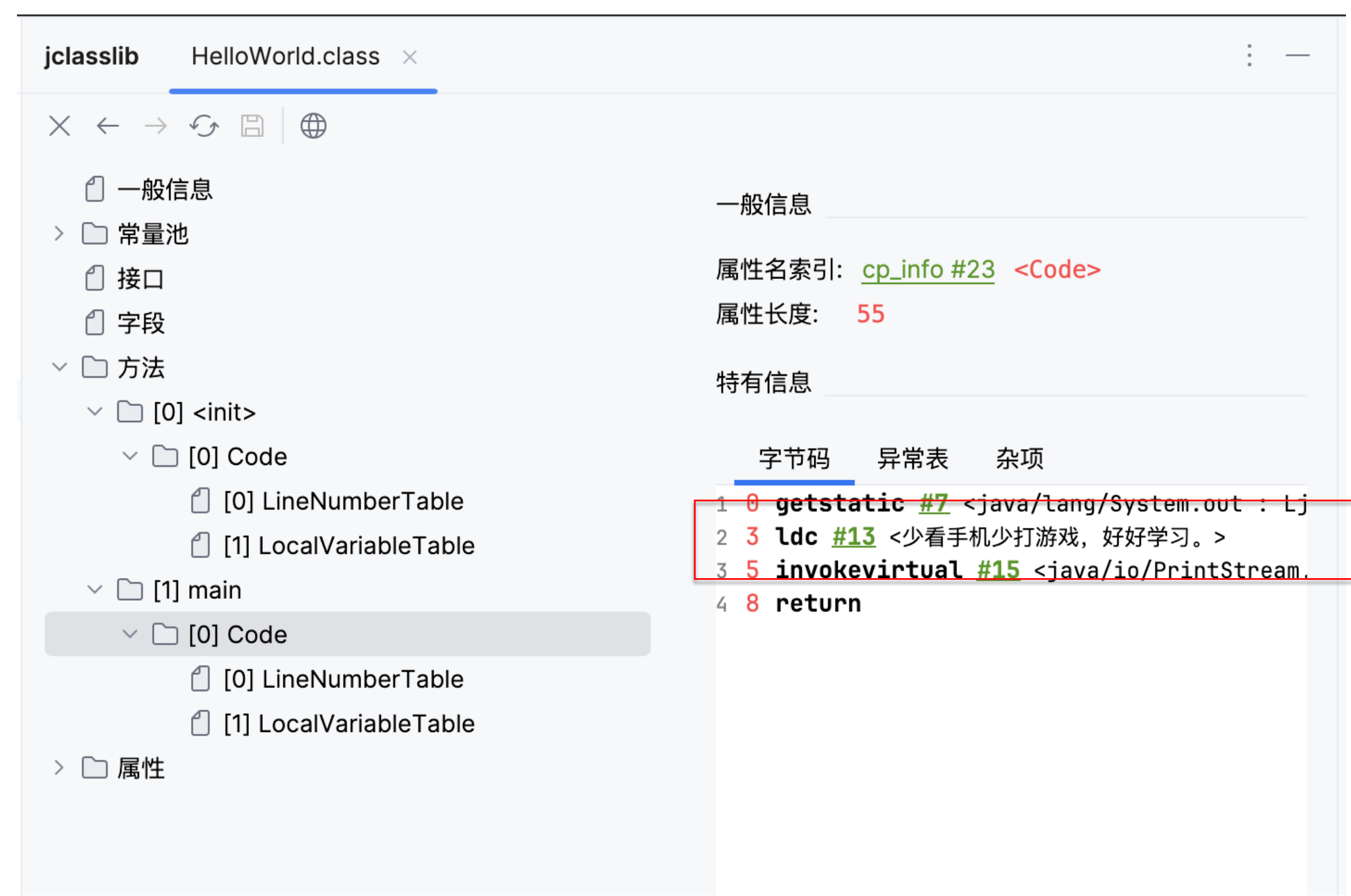


```
1  cafe babe 0000 0041 0022 0a00 0200 0307
2  0004 0c00 0500 0601 0010 6a61 7661 2f6c
3  616e 672f 4f62 6a65 6374 0100 063c 696e
4  6974 3e01 0003 2829 5609 0008 0009 0700
5  0a0c 000b 000c 0100 106a 6176 612f 6c61
6  6e67 2f53 7973 7465 6d01 0003 6f75 7401
7  0015 4c6a 6176 612f 696f 2f50 7269 6e74
8  5374 7265 616d 3b08 000e 0100 2ae5 b091
9  e79c 8be6 898b e69c bae5 b091 e689 93e6
10 b8b8 e688 8fef bc8c e5a5 bde5 a5bd e5ad
11 a6e4 b9a0 e380 820a 0010 0011 0700 120c
12 0013 0014 0100 136a 6176 612f 696f 2f50
13 7269 6e74 5374 7265 616d 0100 0770 7269
14 6e74 6c6e 0100 1528 4c6a 6176 612f 6c61
15 6e67 2f53 7472 696e 673b 2956 0700 1601
16 0016 6f72 672f 6578 616d 706c 652f 4865
17 6c6c 6f57 6f72 6c64 0100 0443 6f64 6501
18 000f 4c69 6e65 4e75 6d62 6572 5461 626c
19 6501 0012 4c6f 6361 6c56 6172 6961 626c
20 6554 6162 6c65 0100 0474 6869 7301 0018
21 4c6f 7267 2f65 7861 6d70 6c65 2f48 656c
22 6c6f 576f 726c 643b 0100 046d 6169 6e01
23 0016 285b 4c6a 6176 612f 6c61 6e67 2f53
24 7472 696e 673b 2956 0100 0461 7267 7301
25 0013 5b4c 6a61 7661 2f6c 616e 672f 5374
26 7269 6e67 3b01 000a 536f 7572 6365 4669
27 6c65 0100 0f48 656c 6c6f 576f 726c 642e
28 6a61 7661 0021 0015 0002 0000 0000 0002
```

# 字节码

class 文件的十六进制形式

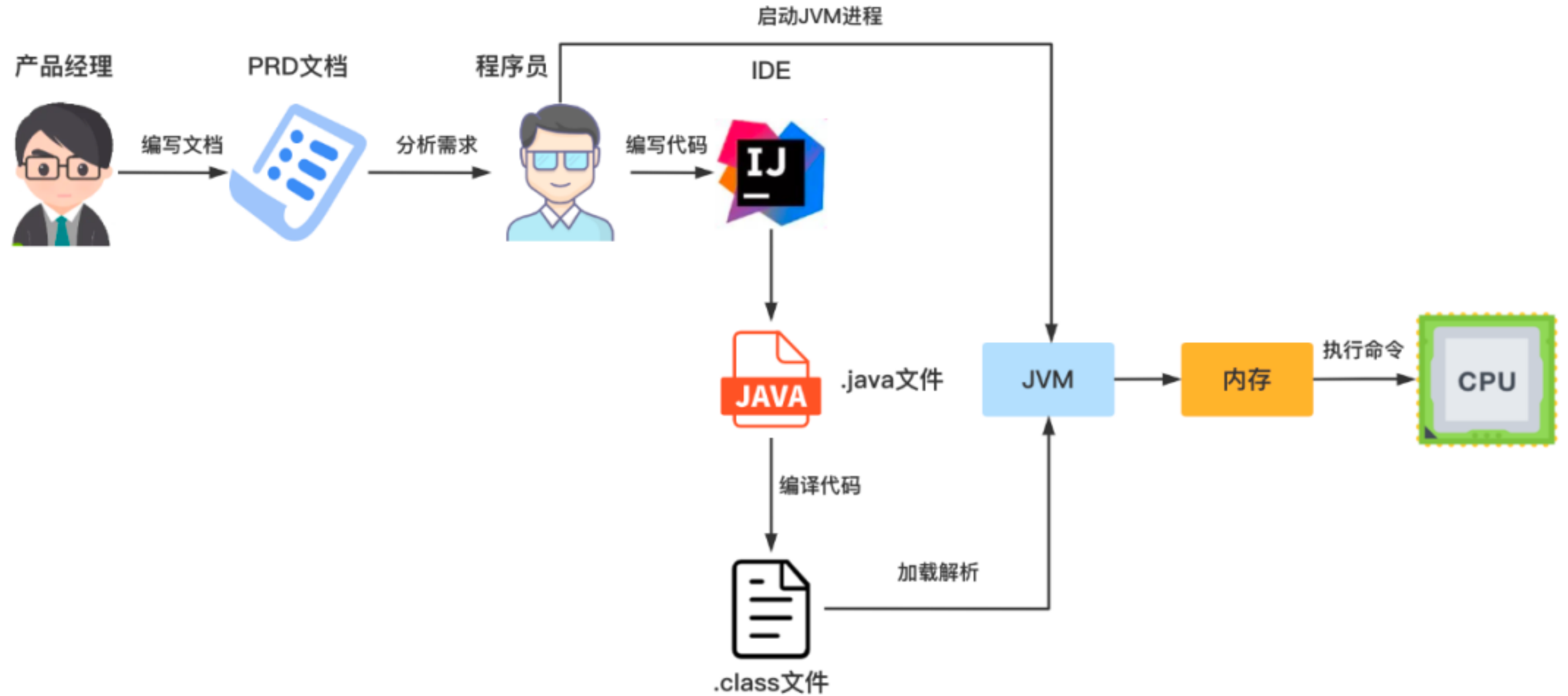
————>  
jclasslib/  
javap



```
(base) fengyi@192 example % javap -v HelloWorld.class
Classfile /Users/fengyi/Desktop/xxxxxx/Test/target/classes/org/example/HelloWorld.class
  Last modified 2025年4月13日; size 588 bytes
  SHA-256 checksum 06f2d7b4cdbcd5a022327d8639b7580e10daa0206e93f479d697b8606ad88190
  Compiled from "HelloWorld.java"
public class org.example.HelloWorld
  minor version: 0
  major version: 65
  flags: (0x0021) ACC_PUBLIC, ACC_SUPER
  this_class: #21                // org/example/HelloWorld
  super_class: #2                // java/lang/Object
  interfaces: 0, fields: 0, methods: 2, attributes: 1
Constant pool:
  #1 = Methodref                 #2.#3      // java/lang/Object."<init>":()V
```

直观地查看类对应的字节码，还可以查看类的基本信息、常量池、接口、字段、方法等信息

# 运行



当有了 .class 文件也就是字节码文件之后，启动 JVM 来运行字节码文件



# HelloWorld 的字节码指令序列

```
0: getstatic    #7      // Field java/lang/System.out:Ljava/io/PrintStream;
3: ldc          #13      // String 少看手机少打游戏，好好学习。
5: invokevirtual #15      // Method java/io/PrintStream.println:(Ljava/lang/String;)V
8: return
```

字节码指令序列通常由多条指令组成，每条指令由一个操作码和若干个操作数构成。

- 操作码：一个字节大小的指令，用于表示具体的操作。
- 操作数：跟随操作码，用于提供额外信息。

这段字节码序列的意思是调用 `System.out.println` 方法打印 "Hello World" 字符串

0: getstatic #7 <java/lang/System.out> :

- 操作码：getstatic 操作数：#7
- 描述：这条指令的作用是获取静态字段，这里获取的是 `java.lang.System` 类的 `out` 静态字段，它是一个 `PrintStream` 类型的输出流。#7 是一个指向常量池的索引，后面在讲类文件结构时会讲到。

3: ldc #13 <少看手机少打游戏，好好学习。> :

- 操作码：ldc 操作数：#13
- 描述：这条指令的作用是从常量池中加载一个常量值（字符串 "少看手机少打游戏，好好学习。"）到操作数栈顶。#13 是一个指向常量池的索引，常量池里存储了字符串的引用。

5: invokevirtual #15 <java/io/PrintStream.println> :

- 操作码：invokevirtual 操作数：#15
- 描述：这条指令的作用是调用方法。这里调用的是 `PrintStream` 类的 `println` 方法，用来打印字符串。#15 是一个指向常量池的索引，常量池里存储了 `java/io/PrintStream.println` 方法的引用信息。

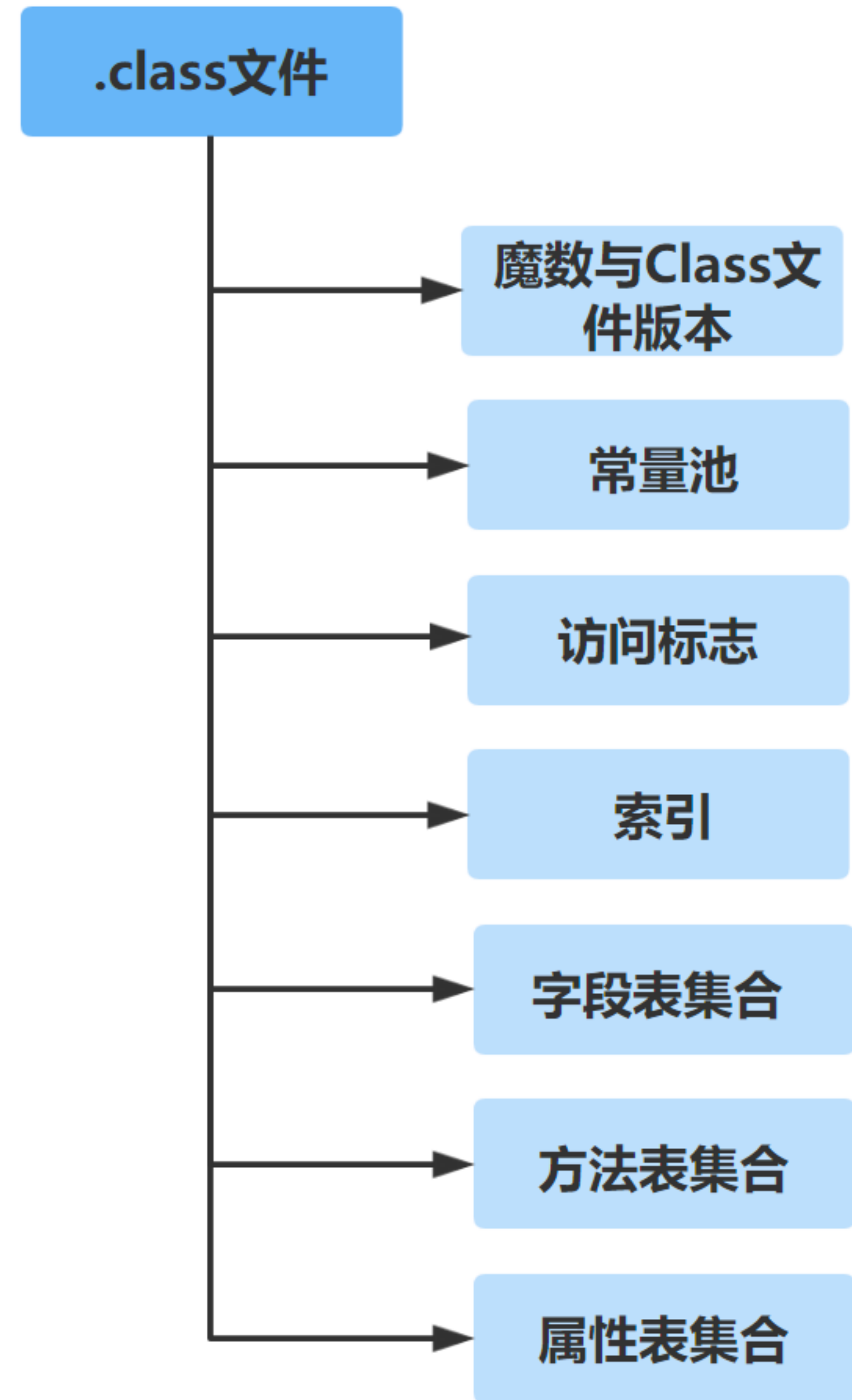
8: return:

- 操作码：return
- 描述：这条指令的作用是从当前方法返回。

# Java类文件结构

# Class文件结构

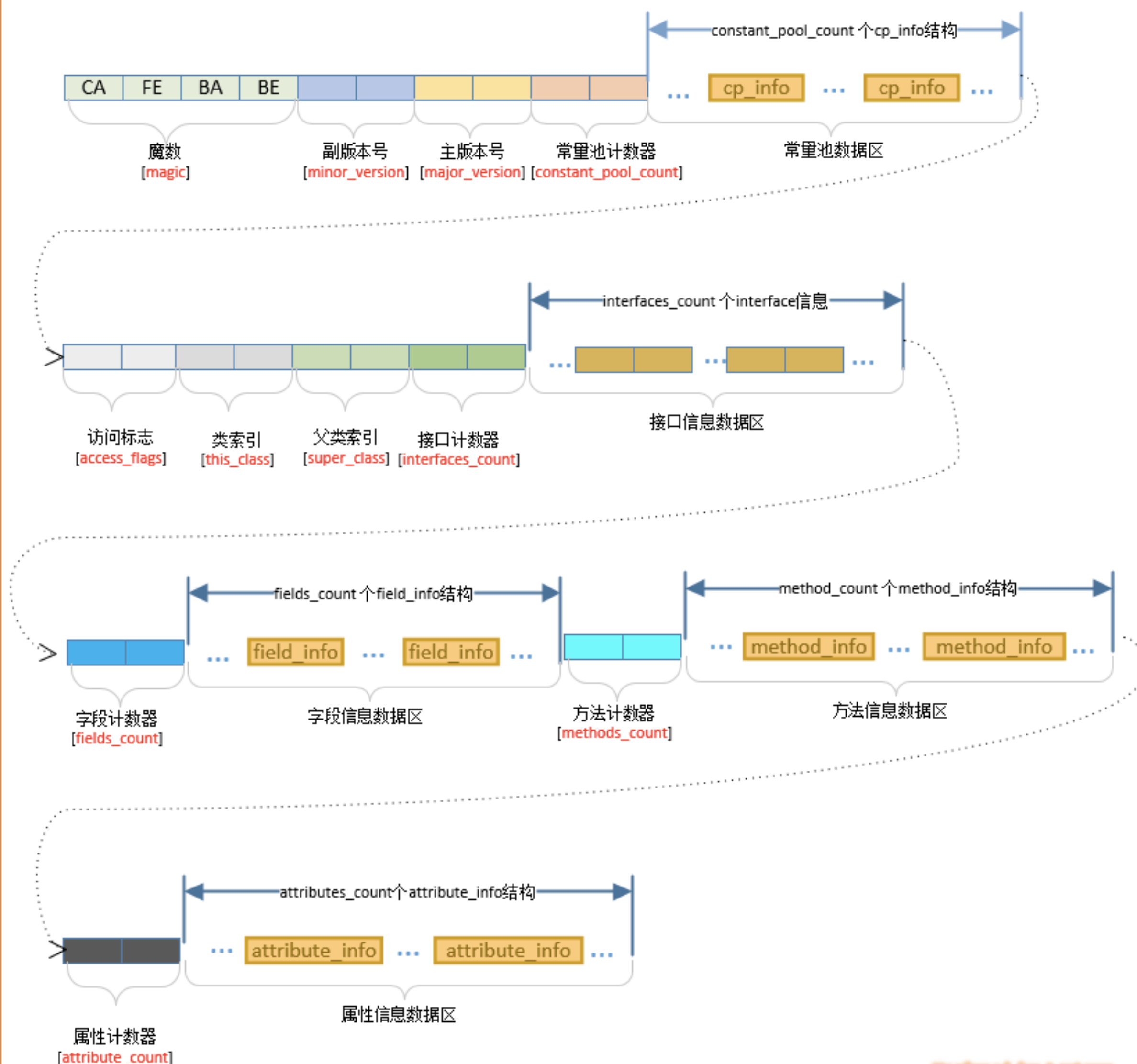
- 一组以8位字节为基础单位的二进制流
- 魔数
  - 0xCAFEBAE
- 版本号
- 常量池
  - 字面量
  - 符号引用
- 访问标志
- 类索引、父类索引与接口索引集合
- 字段表集合
- 方法表集合
- 属性表集合 (代码的实现作为一个属性值)





## Class文件字节码结构组织示意图

注：被编译器编译成的.class字节码文件的字节流以及其组织结构如下所示：



```
ClassFile {  
    u4      magic;  
    u2      minor_version;  
    u2      major_version;  
    u2      constant_pool_count;  
    cp_info  constant_pool[constant_pool_count-1];  
    u2      access_flags;  
    u2      this_class;  
    u2      super_class;  
    u2      interfaces_count;  
    u2      interfaces[interfaces_count];  
    u2      fields_count;  
    field_info fields[fields_count];  
    u2      methods_count;  
    method_info methods[methods_count];  
    u2      attributes_count;  
    attribute_info attributes[attributes_count];  
}
```



# 01、魔数

第一行中有一串特殊的字符 `cafebabe`，它就是一个魔数，是 JVM 识别 class 文件的标志，JVM 会在验证阶段检查 class 文件是否以该魔数开头，如果不是则会抛出 `ClassFormatError`。



| HelloWorld.class |      |      |      |      |      |      |      |      |  |
|------------------|------|------|------|------|------|------|------|------|--|
| 1                | cafe | babe | 0000 | 0041 | 0022 | 0a00 | 0200 | 0307 |  |
| 2                | 0004 | 0c00 | 0500 | 0601 | 0010 | 6a61 | 7661 | 2f6c |  |
| 3                | 616e | 672f | 4f62 | 6a65 | 6374 | 0100 | 063c | 696e |  |
| 4                | 6974 | 3e01 | 0003 | 2829 | 5609 | 0008 | 0009 | 0700 |  |
| 5                | 0a0c | 000b | 000c | 0100 | 106a | 6176 | 612f | 6c61 |  |
| 6                | 6e67 | 2f53 | 7973 | 7465 | 6d01 | 0003 | 6f75 | 7401 |  |
| 7                | 0015 | 4c6a | 6176 | 612f | 696f | 2f50 | 7269 | 6e74 |  |
| 8                | 5374 | 7265 | 616d | 3b08 | 000e | 0100 | 2ae5 | b091 |  |
| 9                | e79c | 8be6 | 898b | e69c | bae5 | b091 | e689 | 93e6 |  |
| 10               | b8b8 | e688 | 8fef | bc8c | e5a5 | bde5 | a5bd | e5ad |  |
| 11               | a6e4 | b9a0 | e380 | 820a | 0010 | 0011 | 0700 | 120c |  |
| 12               | 0013 | 0014 | 0100 | 136a | 6176 | 612f | 696f | 2f50 |  |
| 13               | 7269 | 6e74 | 5374 | 7265 | 616d | 0100 | 0770 | 7269 |  |
| 14               | 6e74 | 6c6e | 0100 | 1528 | 4c6a | 6176 | 612f | 6c61 |  |
| 15               | 6e67 | 2f53 | 7472 | 696e | 673b | 2956 | 0700 | 1601 |  |
| 16               | 0016 | 6f72 | 672f | 6578 | 616d | 706c | 652f | 4865 |  |
| 17               | 6c6c | 6f57 | 6f72 | 6c64 | 0100 | 0443 | 6f64 | 6501 |  |
| 18               | 000f | 4c69 | 6e65 | 4e75 | 6d62 | 6572 | 5461 | 626c |  |
| 19               | 6501 | 0012 | 4c6f | 6361 | 6c56 | 6172 | 6961 | 626c |  |
| 20               | 6554 | 6162 | 6c65 | 0100 | 0474 | 6869 | 7301 | 0018 |  |
| 21               | 4c6f | 7267 | 2f65 | 7861 | 6d70 | 6c65 | 2f48 | 656c |  |
| 22               | 6c6f | 576f | 726c | 643b | 0100 | 046d | 6169 | 6e01 |  |
| 23               | 0016 | 285b | 4c6a | 6176 | 612f | 6c61 | 6e67 | 2f53 |  |
| 24               | 7472 | 696e | 673b | 2956 | 0100 | 0461 | 7267 | 7301 |  |
| 25               | 0013 | 5b4c | 6a61 | 7661 | 2f6c | 616e | 672f | 5374 |  |
| 26               | 7269 | 6e67 | 3b01 | 000a | 536f | 7572 | 6365 | 4669 |  |
| 27               | 6c65 | 0100 | 0f48 | 656c | 6c6f | 576f | 726c | 642e |  |
| 28               | 6a61 | 7661 | 0021 | 0015 | 0002 | 0000 | 0000 | 0002 |  |

# 02、版本号

魔数后面的四个字节 0000 0041 分别表示副版本号和主版本号。也就是说，主版本号为 65（0x41 的十进制），也就是 Java 21 对应的版本号，副版本号为 0。

决定了该类文件可以被哪个版本的Java虚拟机（JVM）加载和执行。每个版本的Java平台都会引入新的特性或优化，而这些新特性或优化通常需要相应的字节码版本号来支持。

HelloWorld.class

|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 1  | cafe | babe | 0000 | 0041 | 0022 | 0a00 | 0200 | 0307 |
| 2  | 0004 | 0c00 | 0500 | 0601 | 0010 | 6a61 | 7661 | 2f6c |
| 3  | 616e | 672f | 4f62 | 6a65 | 6374 | 0100 | 063c | 696e |
| 4  | 6974 | 3e01 | 0003 | 2829 | 5609 | 0008 | 0009 | 0700 |
| 5  | 0a0c | 000b | 000c | 0100 | 106a | 6176 | 612f | 6c61 |
| 6  | 6e67 | 2f53 | 7973 | 7465 | 6d01 | 0003 | 6f75 | 7401 |
| 7  | 0015 | 4c6a | 6176 | 612f | 696f | 2f50 | 7269 | 6e74 |
| 8  | 5374 | 7265 | 616d | 3b08 | 000e | 0100 | 2ae5 | b091 |
| 9  | e79c | 8be6 | 898b | e69c | bae5 | b091 | e689 | 93e6 |
| 10 | b8b8 | e688 | 8fef | bc8c | e5a5 | bde5 | a5bd | e5ad |
| 11 | a6e4 | b9a0 | e380 | 820a | 0010 | 0011 | 0700 | 120c |
| 12 | 0013 | 0014 | 0100 | 136a | 6176 | 612f | 696f | 2f50 |
| 13 | 7269 | 6e74 | 5374 | 7265 | 616d | 0100 | 0770 | 7269 |
| 14 | 6e74 | 6c6e | 0100 | 1528 | 4c6a | 6176 | 612f | 6c61 |
| 15 | 6e67 | 2f53 | 7472 | 696e | 673b | 2956 | 0700 | 1601 |
| 16 | 0016 | 6f72 | 672f | 6578 | 616d | 706c | 652f | 4865 |
| 17 | 6c6c | 6f57 | 6f72 | 6c64 | 0100 | 0443 | 6f64 | 6501 |
| 18 | 000f | 4c69 | 6e65 | 4e75 | 6d62 | 6572 | 5461 | 626c |
| 19 | 6501 | 0012 | 4c6f | 6361 | 6c56 | 6172 | 6961 | 626c |
| 20 | 6554 | 6162 | 6c65 | 0100 | 0474 | 6869 | 7301 | 0018 |
| 21 | 4c6f | 7267 | 2f65 | 7861 | 6d70 | 6c65 | 2f48 | 656c |
| 22 | 6c6f | 576f | 726c | 643b | 0100 | 046d | 6169 | 6e01 |
| 23 | 0016 | 285b | 4c6a | 6176 | 612f | 6c61 | 6e67 | 2f53 |
| 24 | 7472 | 696e | 673b | 2956 | 0100 | 0461 | 7267 | 7301 |
| 25 | 0013 | 5b4c | 6a61 | 7661 | 2f6c | 616e | 672f | 5374 |
| 26 | 7269 | 6e67 | 3b01 | 000a | 536f | 7572 | 6365 | 4669 |
| 27 | 6c65 | 0100 | 0f48 | 656c | 6c6f | 576f | 726c | 642e |
| 28 | 6a61 | 7661 | 0021 | 0015 | 0002 | 0000 | 0000 | 0002 |



# 03、常量池

版本号之后的是常量池，它包含了类、接口、字段和方法的符号引用，以及字符串字面量和数值常量。这些信息在编译时被创建，并在运行时被Java虚拟机（JVM）使用。

相当于一个资源仓库，主要存放量大类型常量：

- 字面量（Literals）：字面量是不变的数据，主要包括数值（如整数、浮点数）和字符串字面量。例如，一个整数100或一个字符串"Hello World"，在源代码中直接赋值，编译后存储在常量池中。
- 符号引用（Symbolic References）：符号引用是对类、接口、字段、方法等的引用，它们不是由字面量值给出的，而是通过符号名称（如类名、方法名）和其他额外信息（如类型、签名）来表示。这些引用在类文件中以一种抽象的方式存在，它们在类加载时被虚拟机解析为具体的内存地址。

HelloWorld.class x

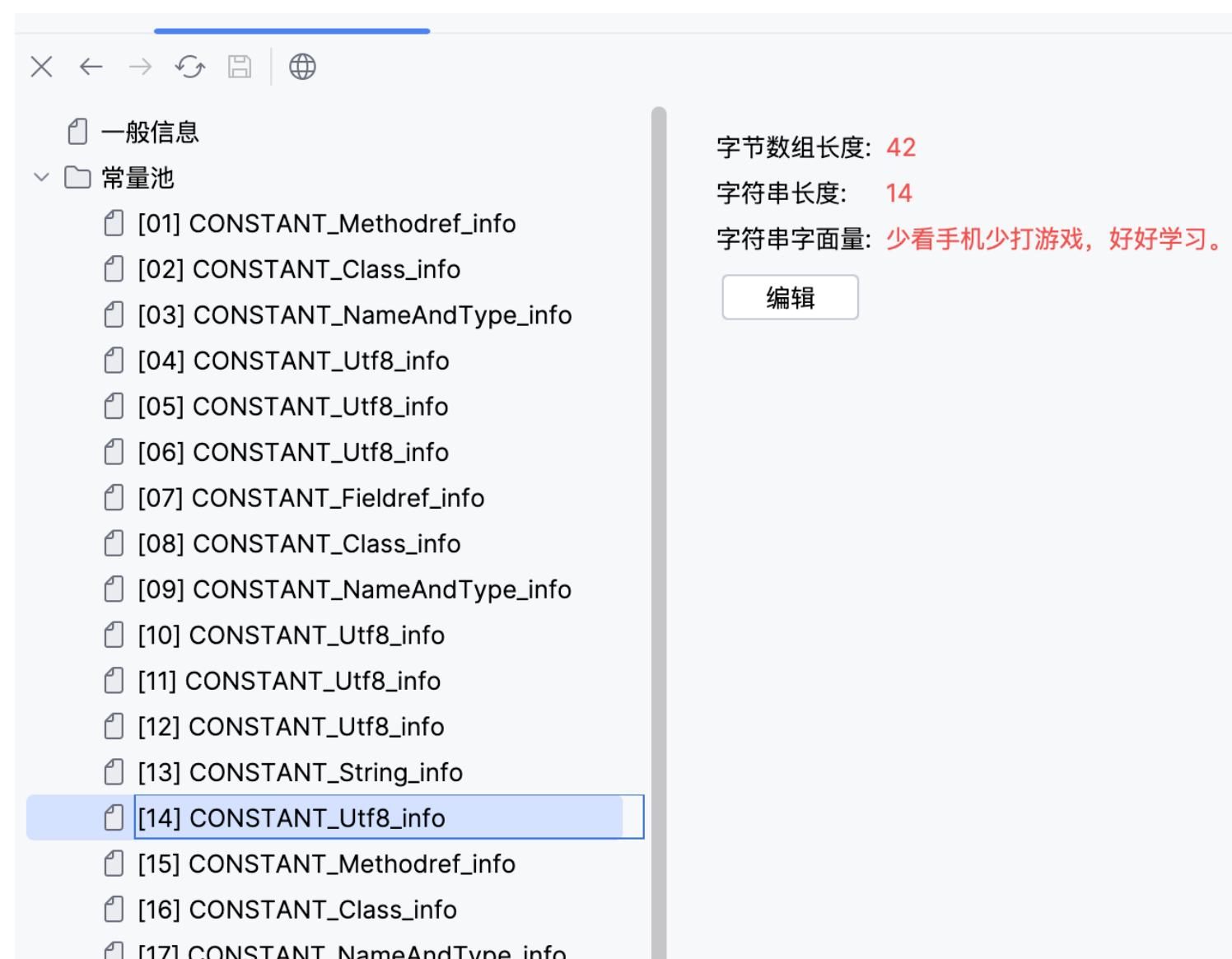
|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 1  | cafe | babe | 0000 | 0041 | 0022 | 0a00 | 0200 | 0307 |
| 2  | 0004 | 0c00 | 0500 | 0601 | 0010 | 6a61 | 7661 | 2f6c |
| 3  | 616e | 672f | 4f62 | 6a65 | 6374 | 0100 | 063c | 696e |
| 4  | 6974 | 3e01 | 0003 | 2829 | 5609 | 0008 | 0009 | 0700 |
| 5  | 0a0c | 000b | 000c | 0100 | 106a | 6176 | 612f | 6c61 |
| 6  | 6e67 | 2f53 | 7973 | 7465 | 6d01 | 0003 | 6f75 | 7401 |
| 7  | 0015 | 4c6a | 6176 | 612f | 696f | 2f50 | 7269 | 6e74 |
| 8  | 5374 | 7265 | 616d | 3b08 | 000e | 0100 | 2ae5 | b091 |
| 9  | e79c | 8be6 | 898b | e69c | bae5 | b091 | e689 | 93e6 |
| 10 | b8b8 | e688 | 8fef | bc8c | e5a5 | bde5 | a5bd | e5ad |
| 11 | a6e4 | b9a0 | e380 | 820a | 0010 | 0011 | 0700 | 120c |
| 12 | 0013 | 0014 | 0100 | 136a | 6176 | 612f | 696f | 2f50 |
| 13 | 7269 | 6e74 | 5374 | 7265 | 616d | 0100 | 0770 | 7269 |
| 14 | 6e74 | 6c6e | 0100 | 1528 | 4c6a | 6176 | 612f | 6c61 |
| 15 | 6e67 | 2f53 | 7472 | 696e | 673b | 2956 | 0700 | 1601 |
| 16 | 0016 | 6f72 | 672f | 6578 | 616d | 706c | 652f | 4865 |
| 17 | 6c6c | 6f57 | 6f72 | 6c64 | 0100 | 0443 | 6f64 | 6501 |
| 18 | 000f | 4c69 | 6e65 | 4e75 | 6d62 | 6572 | 5461 | 626c |
| 19 | 6501 | 0012 | 4c6f | 6361 | 6c56 | 6172 | 6961 | 626c |
| 20 | 6554 | 6162 | 6c65 | 0100 | 0474 | 6869 | 7301 | 0018 |
| 21 | 4c6f | 7267 | 2f65 | 7861 | 6d70 | 6c65 | 2f48 | 656c |
| 22 | 6c6f | 576f | 726c | 643b | 0100 | 046d | 6169 | 6e01 |
| 23 | 0016 | 285b | 4c6a | 6176 | 612f | 6c61 | 6e67 | 2f53 |
| 24 | 7472 | 696e | 673b | 2956 | 0100 | 0461 | 7267 | 7301 |
| 25 | 0013 | 5b4c | 6a61 | 7661 | 2f6c | 616e | 672f | 5374 |
| 26 | 7269 | 6e67 | 3b01 | 000a | 536f | 7572 | 6365 | 4669 |
| 27 | 6c65 | 0100 | 0f48 | 656c | 6c6f | 576f | 726c | 642e |
| 28 | 6a61 | 7661 | 0021 | 0015 | 0002 | 0000 | 0000 | 0002 |

## 03、常量池

前两个字节是常量的个数，十六进制的22，十进制的34，但一共只有33个常量，因为从1开始计数。

每一个常量都是个常量表。在JDK1.7之前共有11中不同结构的常量表，在JDK 1.7中有额外增加了3种常量表。每个常量表都是以u1类型的标志位开始，来区分常量表的类型。

```
CONSTANT_Utf8_info {  
    u1 tag;  
    u2 length;  
    u1 bytes[length];  
}
```



|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 1  | cafe | babe | 0000 | 0041 | 0022 | 0a00 | 0200 | 0307 |
| 2  | 0004 | 0c00 | 0500 | 0601 | 0010 | 6a61 | 7661 | 2f6c |
| 3  | 616e | 672f | 4f62 | 6a65 | 6374 | 0100 | 063c | 696e |
| 4  | 6974 | 3e01 | 0003 | 2829 | 5609 | 0008 | 0009 | 0700 |
| 5  | 0a0c | 000b | 000c | 0100 | 106a | 6176 | 612f | 6c61 |
| 6  | 6e67 | 2f53 | 7973 | 7465 | 6d01 | 0003 | 6f75 | 7401 |
| 7  | 0015 | 4c6a | 6176 | 612f | 696f | 2f50 | 7269 | 6e74 |
| 8  | 5374 | 7265 | 616d | 3b08 | 000e | 0100 | 2ae5 | b091 |
| 9  | e79c | 8be6 | 898b | e69c | bae5 | b091 | e689 | 93e6 |
| 10 | b8b8 | e688 | 8fef | bc8c | e5a5 | bde5 | a5bd | e5ad |
| 11 | a6e4 | b9a0 | e380 | 820a | 0010 | 0011 | 0700 | 120c |
| 12 | 0013 | 0014 | 0100 | 136a | 6176 | 612f | 696f | 2f50 |
| 13 | 7269 | 6e74 | 5374 | 7265 | 616d | 0100 | 0770 | 7269 |
| 14 | 6e74 | 6c6e | 0100 | 1528 | 4c6a | 6176 | 612f | 6c61 |
| 15 | 6e67 | 2f53 | 7472 | 696e | 673b | 2956 | 0700 | 1601 |
| 16 | 0016 | 6f72 | 672f | 6578 | 616d | 706c | 652f | 4865 |
| 17 | 6c6c | 6f57 | 6f72 | 6c64 | 0100 | 0443 | 6f64 | 6501 |
| 18 | 000f | 4c69 | 6e65 | 4e75 | 6d62 | 6572 | 5461 | 626c |
| 19 | 6501 | 0012 | 4c6f | 6361 | 6c56 | 6172 | 6961 | 626c |
| 20 | 6554 | 6162 | 6c65 | 0100 | 0474 | 6869 | 7301 | 0018 |
| 21 | 4c6f | 7267 | 2f65 | 7861 | 6d70 | 6c65 | 2f48 | 656c |
| 22 | 6c6f | 576f | 726c | 643b | 0100 | 046d | 6169 | 6e01 |
| 23 | 0016 | 285b | 4c6a | 6176 | 612f | 6c61 | 6e67 | 2f53 |
| 24 | 7472 | 696e | 673b | 2956 | 0100 | 0461 | 7267 | 7301 |
| 25 | 0013 | 5b4c | 6a61 | 7661 | 2f6c | 616e | 672f | 5374 |
| 26 | 7269 | 6e67 | 3b01 | 000a | 536f | 7572 | 6365 | 4669 |
| 27 | 6c65 | 0100 | 0f48 | 656c | 6c6f | 576f | 726c | 642e |
| 28 | 6a61 | 7661 | 0021 | 0015 | 0002 | 0000 | 0000 | 0002 |



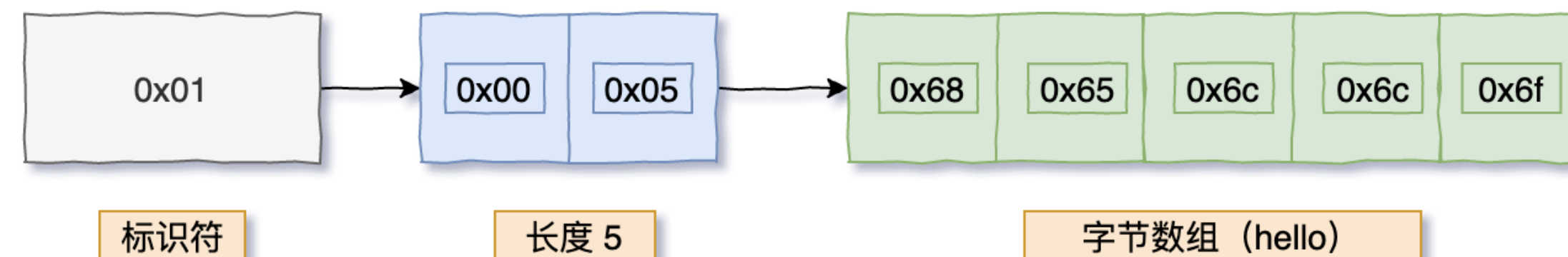
## 03、常量池

```
class Hello {  
    public final String s = "hello";  
}
```

“hello” 是一个字符串，它的十六进制为 68 65 6c 6c 6f，我们来看一下它在 class 文件中的位置。

第一个字节 0x01 是标识，标识类型为 CONSTANT\_Uft8\_info，第二个和第三个 0x00 0x05 用来表示第三部分字节数组的长度，如下图所示。

```
$ xxd -g 1 Hello.class  
00000000: ca fe ba be 00 00 00 37 00 17 0a 00 05 00 12 08 .....7.....  
00000010: 00 13 09 00 04 00 14 07 00 15 07 00 16 01 00 01 .....  
00000020: 73 01 00 12 4c 6a 61 76 61 2f 6c 61 6e 67 2f 53 s...Ljava/lang/S  
00000030: 74 72 69 6e 67 3b 01 00 0d 43 6f 6e 73 74 61 6e tring;...Constan  
00000040: 74 56 61 6c 75 65 01 00 06 3c 69 6e 69 74 3e 01 tValue...<init>.  
00000050: 00 03 28 29 56 01 00 04 43 6f 64 65 01 00 0f 4c ..()V...Code...L  
00000060: 69 6e 65 4e 75 6d 62 65 72 54 61 62 6c 65 01 00 ineNumberTable..  
00000070: 12 4c 6f 63 61 6c 56 61 72 69 61 62 6c 65 54 61 .LocalVariableTa  
00000080: 62 6c 65 01 00 04 74 68 69 73 01 00 18 4c 63 6f ble...this...Lco  
00000090: 6d 2f 69 74 77 61 6e 67 65 72 2f 6a 76 6d 2f 48 m/itwanger/jvm/H  
000000a0: 65 6c 6c 6f 3b 01 00 0a 53 6f 75 72 63 65 46 69 ello;...SourceFi  
000000b0: 6c 65 01 00 0a 48 65 6c 6c 6f 2e 6a 61 76 61 0c le...Hello.java.  
000000c0: 00 09 00 0a 01 00 05 68 65 6c 6c 6f 0c 00 06 00 .....hello....  
000000d0: 07 01 00 16 63 6f 6d 2f 69 74 77 61 6e 67 65 72 ....com/itwanger  
000000e0: 2f 6a 76 6d 2f 48 65 6c 6c 6f 01 00 10 6a 61 76 /jvm/Hello...jav  
000000f0: 61 2f 6c 61 6e 67 2f 4f 62 6a 65 63 74 00 20 00 a/lang/Object. .  
00000100: 04 00 05 00 00 00 01 00 11 00 06 00 07 00 01 00 .....  
00000110: 08 00 00 00 02 00 02 00 01 00 00 00 09 00 0a 00 .....  
00000120: 01 00 0b 00 00 00 39 00 02 00 01 00 00 00 0b 2a .....9.....*  
00000130: b7 00 01 2a 12 02 b5 00 03 b1 00 00 00 02 00 0c ...*.....  
00000140: 00 00 00 0a 00 02 00 00 00 06 00 04 00 07 00 0d .....  
00000150: 00 00 00 0c 00 01 00 00 00 0b 00 0e 00 0f 00 00 .....  
00000160: 00 01 00 10 00 00 00 02 00 11 .....
```



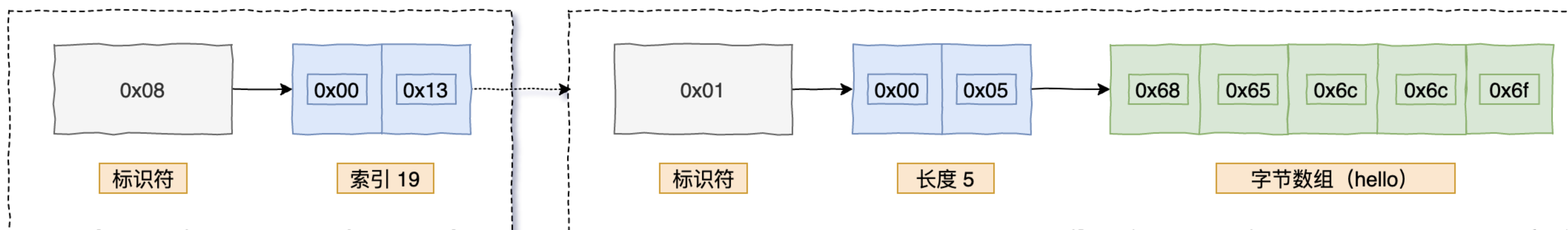


## 03、常量池

```
class Hello {  
    public final String s = "hello";  
}
```

与 CONSTANT\_Uft8\_info 类型对应的，还有一个 CONSTANT\_String\_info，用来表示字符串对象的引用（之前代码中的 s），标识是 0x08。前者存储了字符串真正的值，后者并不包含字符串的内容，仅仅包含了一个指向常量池中 CONSTANT\_Uft8\_info 的索引。

```
$ xxd -g 1 Hello.class  
00000000: ca fe ba be 00 00 00 37 00 17 0a 00 05 00 12 08 .....7.....  
00000010: 00 13 09 00 04 00 14 07 00 15 07 00 16 01 00 01 .....  
00000020: 73 01 00 12 4c 6a 61 76 61 2f 6c 61 6e 67 2f 53 s...Ljava/lang/S  
00000030: 74 72 69 6e 67 3b 01 00 0d 43 6f 6e 73 74 61 6e tring;...Constan  
00000040: 74 56 61 65 75 65 01 00 06 3c 69 6e 69 74 3e 01 tValue...<init>.  
00000050: 00 03 28 29 56 01 00 04 43 6f 64 65 01 00 0f 4c ..()V...Code...L  
00000060: 69 6e 65 4e 75 6d 62 65 72 54 61 62 6c 65 01 00 ineNumberTable..  
00000070: 12 4c 6f 63 61 6c 56 61 72 69 61 62 6c 65 54 61 .LocalVariableTa  
00000080: 62 6c 65 01 00 04 74 68 69 73 01 00 18 4c 63 6f ble...this...Lco  
00000090: 6d 2f 69 74 77 61 6e 67 65 72 2f 6a 76 6d 2f 48 m/itwanger/jvm/H  
000000a0: 65 6c 6c 6f 3b 01 00 0a 53 6f 75 72 63 65 46 69 ello;...SourceFi  
000000b0: 6c 65 01 00 0a 48 65 6c 6c 6f 2e 6a 61 76 61 0c le...Hello.java.  
000000c0: 00 09 00 0a 01 00 05 68 65 6c 6c 6f 0c 00 06 00 .....hello....  
000000d0: 07 01 00 16 63 6f 6d 2f 69 74 77 61 6e 67 65 72 ....com/itwanger  
000000e0: 2f 6a 76 6d 2f 48 65 6c 6c 6f 01 00 10 6a 61 76 /jvm/Hello...jav  
000000f0: 61 2f 6c 61 6e 67 2f 4f 62 6a 65 63 74 00 20 00 a/lang/Object. .  
00000100: 04 00 05 00 00 00 01 00 11 00 06 00 07 00 01 00 .....  
00000110: 08 00 00 00 02 00 02 00 01 00 00 00 09 00 0a 00 .....  
00000120: 01 00 0b 00 00 00 39 00 02 00 01 00 00 00 0b 2a .....9.....*  
00000130: b7 00 01 2a 12 02 b5 00 03 b1 00 00 00 02 00 0c ...*.....  
00000140: 00 00 00 0a 00 02 00 00 00 06 00 04 00 07 00 0d .....  
00000150: 00 00 00 0c 00 01 00 00 00 0b 00 0e 00 0f 00 00 .....  
00000160: 00 01 00 10 00 00 00 02 00 11 .....
```



# 03、常量池

```
public class ConstantTest {
    public final boolean bool = true;
    public final char aChar = 'a';
    public final byte b = 66;
    public final short s = 67;
    public final int i = 68;
}
```

布尔值 true 的十六进制是 0x01、字符 a 的十六进制是 0x61，字节 66 的十六进制是 0x42，短整型 67 的十六进制是 0x43，整型 68 的十六进制是 0x44

```
$ xxd -g 1 ConstantTest.class
00000000: ca fe ba be 00 00 00 37 00 2a 0a 00 08 00 22 09 .....7.*....".
00000010: 00 07 00 23 09 00 07 00 24 09 00 07 00 25 09 00 ...#....$....%..
00000020: 07 00 26 09 00 07 00 27 07 00 28 07 00 29 01 00 ..&....'..(..)..
00000030: 04 62 6f 6f 6c 01 00 01 5a 01 00 0d 43 6f 6e 73 .bool...Z...Cons
00000040: 74 61 6e 74 56 61 6c 75 65 03 00 00 00 01 01 00 tantValue.....
00000050: 05 61 43 68 61 72 01 00 01 43 03 00 00 00 61 01 .aChar...C....a.
00000060: 00 01 62 01 00 01 42 03 00 00 00 42 01 00 01 73 ..b...B....B...s
00000070: 01 00 01 53 03 00 00 00 43 01 00 01 69 01 00 01 ...S....C...i...
00000080: 49 03 00 00 00 44 01 00 06 3c 69 6e 69 74 3e 01 I....D...<init>.
00000090: 00 03 28 29 56 01 00 04 43 6f 64 65 01 00 0f 4c ..()V...Code...L
000000a0: 69 6e 65 4e 75 6d 62 65 72 54 61 62 6c 65 01 00 ineNumberTable..
000000b0: 12 4c 6f 63 61 6c 56 61 72 69 61 62 6c 65 54 61 .LocalVariableTa
000000c0: 62 6c 65 01 00 04 74 68 69 73 01 00 1f 4c 63 6f ble...this...Lco
000000d0: 6d 2f 69 74 77 61 6e 67 65 72 2f 6a 76 6d 2f 43 m/itwanger/jvm/C
000000e0: 6f 6e 73 74 61 6e 74 54 65 73 74 3b 01 00 0a 53 onstantTest;...S
```

| 常量类型                  | 标识符  | 描述符          |
|-----------------------|------|--------------|
| CONSTANT_Integer_info | 0x03 | int 类型字面量    |
| CONSTANT_Float_info   | 0x04 | float 类型字面量  |
| CONSTANT_Long_info    | 0x05 | long 类型字面量   |
| CONSTANT_Double_info  | 0x06 | double 类型字面量 |

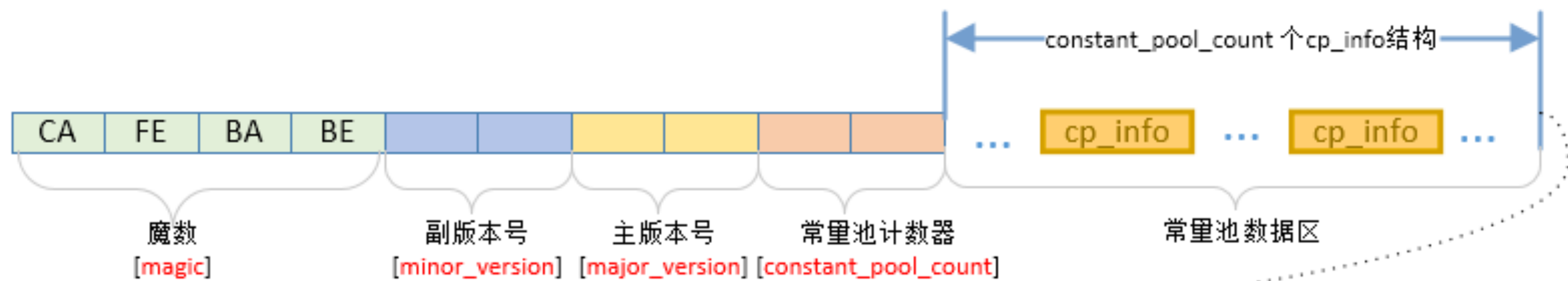


## 03、常量池

```
public class ConstantTest {  
    public final long ong = Long.MAX_VALUE;  
}
```

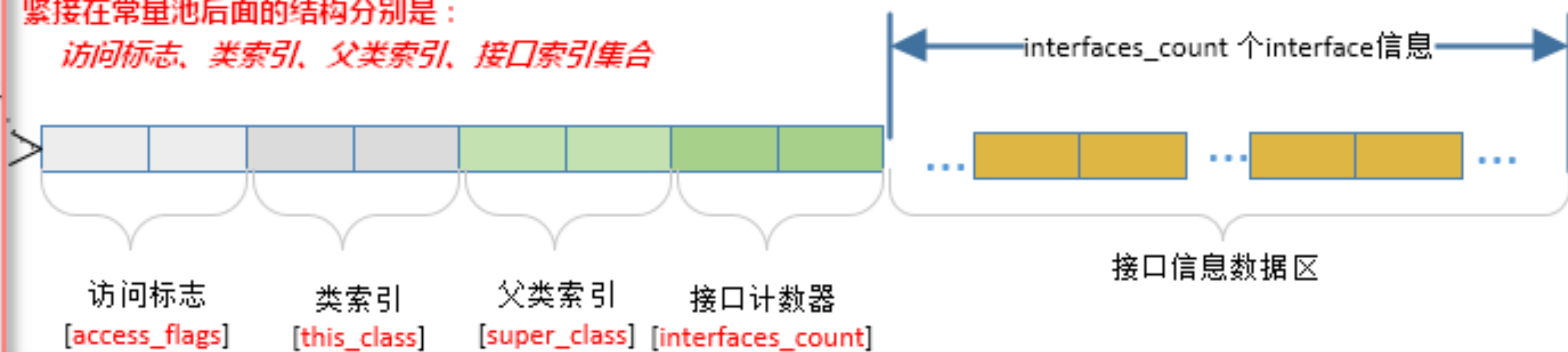
long占了8个字节

```
$ xxd -g 1 ConstantTest.class  
00000000: ca fe ba be 00 00 00 37 00 19 0a 00 07 00 14 07 .....7.....  
00000010: 00 15 05 7f ff ff ff ff ff ff 09 00 06 00 16 .....  
00000020: 07 00 17 07 00 18 01 00 03 6f 6e 67 01 00 01 4a .....ong...J  
00000030: 01 00 0d 43 6f 6e 73 74 61 6e 74 56 61 6c 75 65 ...ConstantValue  
00000040: 01 00 06 3c 69 6e 69 74 3e 01 00 03 28 29 56 01 ...<init>...()V.  
00000050: 00 04 43 6f 64 65 01 00 0f 4c 69 6e 65 4e 75 6d ..Code...LineNum  
00000060: 62 65 72 54 61 62 6c 65 01 00 12 4c 6f 63 61 6c berTable...Local
```



紧接在常量池后面的结构分别是：

访问标志、类索引、父类索引、接口索引集合

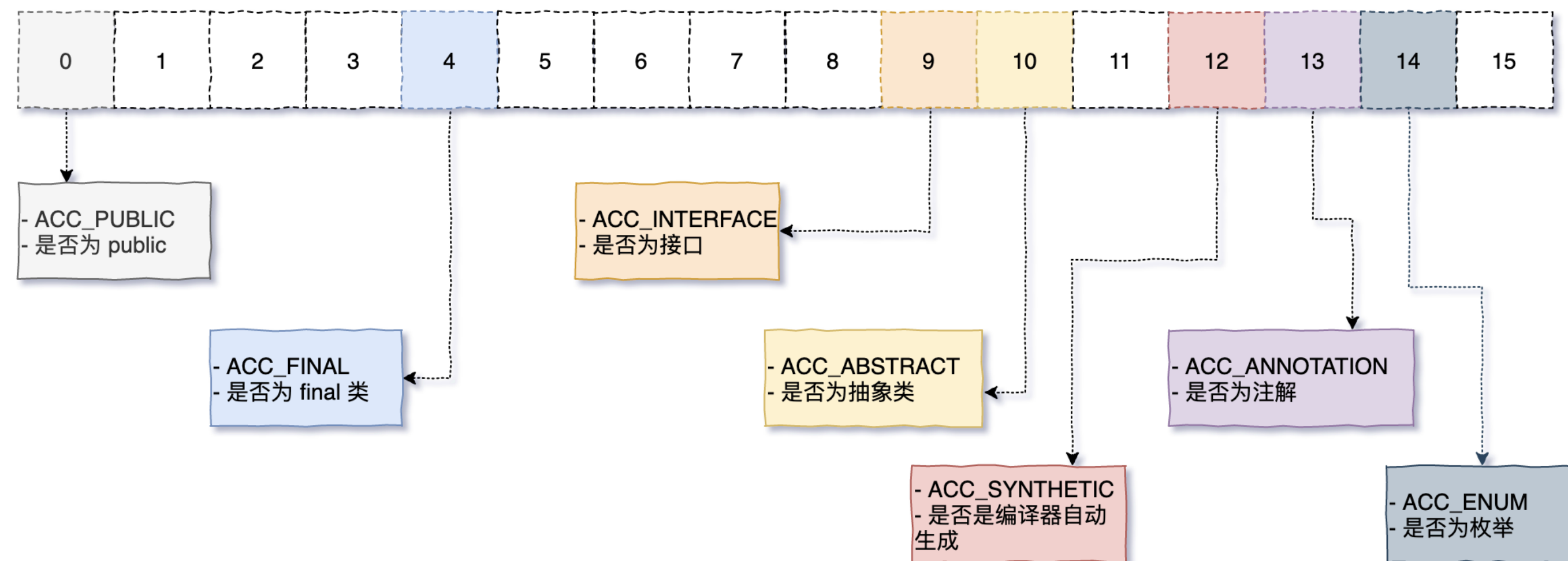




## 04、访问标记

常量池之后的区域就是访问标记（Access flags），这个标记用于识别类或接口的访问信息，比如说：

- 到底是 class 类 还是 interface 接口？
- 是 public 吗？
- 是 abstract 抽象类吗？
- 是 final 类吗？



总共有 16 个标记位可供使用，但常用的只有其中 7 个

# 05, 类索引、父类索引和接口索引

这三部分用来确定类的继承关系, this\_class 为当前类的索引, super\_class 为父类的索引, interfaces 为接口。

```
class Hello { public static void main(String[] args) { } }
```

General Information

Constant Pool

Interfaces

Fields

Methods

Attributes

Minor version: 0

Major version: 55 [11]

Constant pool count: 20

Access flags: 0x0020 []

This class: cp\_info #2 <com/itwanger/jvm/Hello>

Super class: cp\_info #3 <java/lang/Object>

Interfaces count: 0

Fields count: 0

Methods count: 2

Attributes count: 1

000000d0: 01 00 16 63 6f 6d 2f 69 74 77 61 6e 67 65 72 2f ...com/itwanger/

000000e0: 6a 76 6d 2f 48 65 6c 6c 6f 01 00 10 6a 61 76 61 jvm/Hello...java

000000f0: 2f 6c 61 6e 67 2f 4f 62 6a 65 63 74 00 20 00 02 /lang/Object. ..

00000100: 00 03 00 00 00 00 00 02 00 00 00 04 00 05 00 01 .....

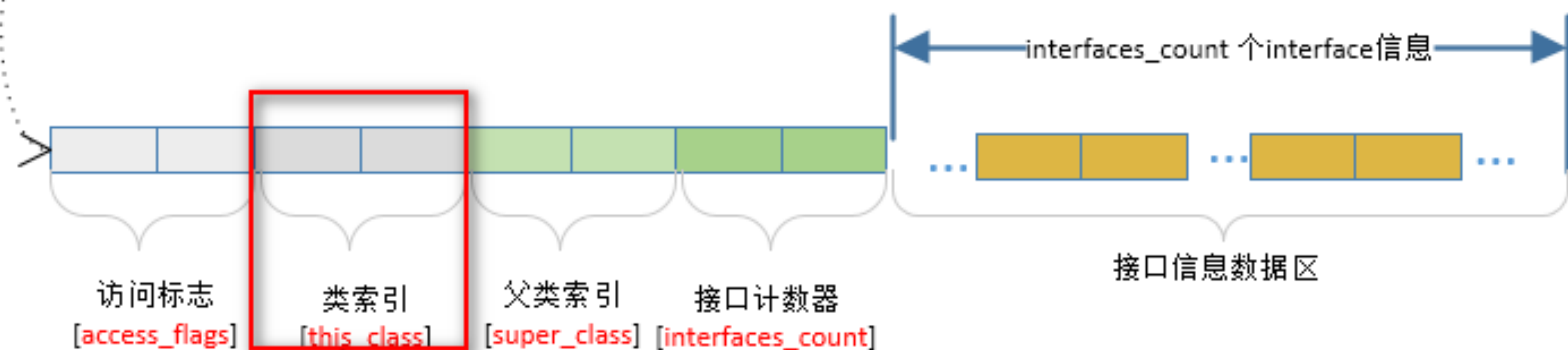
00000110: 00 06 00 00 00 2f 00 01 00 01 00 00 00 05 2a b7 ..../.....\*

00000120: 00 01 b1 00 00 00 02 00 07 00 00 00 06 00 01 00 .....

00000130: 00 00 06 00 08 00 00 00 0c 00 01 00 00 00 05 00 .....

对应 class 文件中的位置

- this\_class 指向常量池中索引为 2 的 CONSTANT\_Class\_info。
- super\_class 指向常量池中索引为 3 的 CONSTANT\_Class\_info。
- 由于没有接口, 所以 interfaces 的信息为空。



类索引有两个字节组成，两个字节中存储的值是是某个常量池中的常量池项CONSTANT\_Class\_info的索引，而这个CONSTANT\_Class\_info项会进一步指向常量池项CONSTANT\_Utf8\_info，该项表示此类的完全限定名字字符串。



```
package com.louis.jvm;
class Simple { public static void
main(String[] args) { } }
```

0x0021 在 Access Flags 中是 0x0020 和 0x0001 的并集,表示 ACC\_PUBLIC与 ACC\_SUPER合并的结果。说明该类的声明修饰是 public并且继承Object

| Simple.class |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |                    |
|--------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|--------------------|
|              | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | a  | b  | c  | d  | e  | f  |                    |
| 00000000h:   | CA | FE | BA | BE | 00 | 00 | 00 | 32 | 00 | 10 | 07 | 00 | 02 | 01 | 00 | 14 | ; 漱壕...2.....      |
| 00000010h:   | 63 | 6F | 6D | 2F | 6C | 6F | 75 | 69 | 73 | 2F | 6A | 76 | 6D | 2F | 53 | 69 | ; com/louis/jvm/Si |
| 00000020h:   | 6D | 70 | 6C | 65 | 07 | 00 | 04 | 01 | 00 | 10 | 6A | 61 | 76 | 61 | 2F | 6C | ; mple.....java/l  |
| 00000030h:   | 61 | 6E | 67 | 2F | 4F | 62 | 6A | 65 | 63 | 74 | 01 | 00 | 06 | 3C | 69 | 6E | ; ang/Object...<in |
| 00000040h:   | 69 | 74 | 3E | 01 | 00 | 03 | 28 | 29 | 56 | 01 | 00 | 04 | 43 | 6F | 64 | 65 | ; it>...()V...Code |
| 00000050h:   | 0A | 00 | 03 | 00 | 09 | 0C | 00 | 05 | 00 | 06 | 01 | 00 | 0F | 4C | 69 | 6E | ; .....Lin         |
| 00000060h:   | 65 | 4E | 75 | 6D | 62 | 65 | 72 | 54 | 61 | 62 | 6C | 65 | 01 | 00 | 12 | 4C | ; eNumberTable...L |
| 00000070h:   | 6F | 63 | 61 | 6C | 56 | 61 | 72 | 69 | 61 | 62 | 6C | 65 | 54 | 61 | 62 | 6C | ; ocalVariableTabl |
| 00000080h:   | 65 | 01 | 00 | 04 | 74 | 68 | 69 | 73 | 01 | 00 | 16 | 4C | 63 | 6F | 6D | 2F | ; e...this...Lcom/ |
| 00000090h:   | 6C | 6F | 75 | 69 | 73 | 2F | 6A | 76 | 6D | 2F | 53 | 69 | 6D | 70 | 6C | 65 | ; louis/jvm/Simple |
| 000000a0h:   | 3B | 01 | 00 | 0A | 53 | 6F | 75 | 72 | 63 | 65 | 46 | 69 | 6C | 65 | 01 | 00 | ; ;...SourceFile.. |
| 000000b0h:   | 0B | 53 | 69 | 6D | 70 | 6C | 65 | 2E | 6A | 61 | 76 | 61 | 00 | 21 | 00 | 01 | ; .Simple.java.!.. |
| 000000c0h:   | 00 | 03 | 00 | 00 | 00 | 00 | 00 | 01 | 00 | 01 | 00 | 05 | 00 | 06 | 00 | 01 | ; .....            |
| 000000d0h:   | 00 | 07 | 00 | 00 | 00 | 2F | 00 | 01 | 00 | 01 | 00 | 00 | 00 | 05 | 2A | B7 | ; ...../*?         |
| 000000e0h:   | 00 | 08 | B1 | 00 | 00 | 00 | 02 | 00 | 0A | 00 | 00 | 00 | 06 | 00 | 01 | 00 | ; ..?.....         |
| 000000f0h:   | 00 | 00 | 03 | 00 | 0B | 00 | 00 | 00 | 0C | 00 | 01 | 00 | 00 | 00 | 05 | 00 | ; .....            |
| 00000100h:   | 0C | 00 | 0D | 00 | 00 | 00 | 01 | 00 | 0E | 00 | 00 | 00 | 02 | 00 | 0F |    | ; .....            |

访问标志(access\_flags)

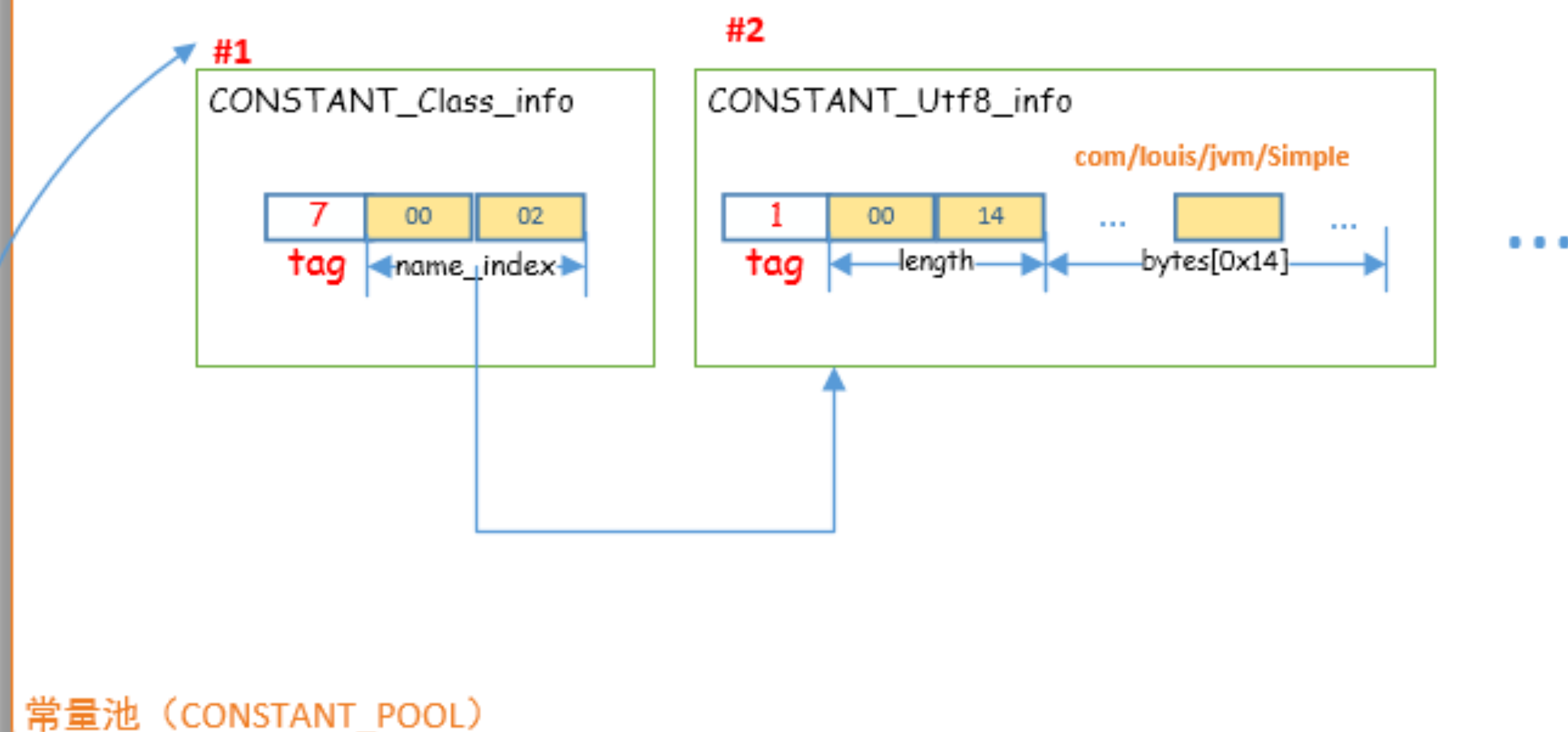
类索引(this\_class) , 其内的值是0x0001

Flags: ACC\_PUBLIC, ACC\_SUPER

Constant pool:

```
#1 = Class           #2 // com/louis/jvm/Simple
#2 = Utf8            com/louis/jvm/Simple
#3 = Class           #4 // java/lang/Object
#4 = Utf8            java/lang/Object
#5 = Utf8            <init>
#6 = Utf8            ()V
```

Designed by LouLuan  
<http://blog.csdn.net/luanlouis>



类索引[this\_class] 00 01

Designed by LouLuan  
<http://blog.csdn.net/luanlouis>



Simple.class x

|            | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | a  | b  | c  | d  | e  | f  |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 00000000h: | CA | FE | BA | BE | 00 | 00 | 00 | 32 | 00 | 10 | 07 | 00 | 02 | 01 | 00 | 14 |
| 00000010h: | 63 | 6F | 6D | 2F | 6C | 6F | 75 | 69 | 73 | 2F | 6A | 76 | 6D | 2F | 53 | 69 |
| 00000020h: | 6D | 70 | 6C | 65 | 07 | 00 | 04 | 01 | 00 | 10 | 6A | 61 | 76 | 61 | 2F | 6C |
| 00000030h: | 61 | 6E | 67 | 2F | 4F | 62 | 6A | 65 | 63 | 74 | 01 | 00 | 06 | 3C | 69 | 6E |
| 00000040h: | 69 | 74 | 3E | 01 | 00 | 03 | 28 | 29 | 56 | 01 | 00 | 04 | 43 | 6F | 64 | 65 |
| 00000050h: | 0A | 00 | 03 | 00 | 09 | 0C | 00 | 05 | 00 | 06 | 01 | 00 | 0F | 4C | 69 | 6E |
| 00000060h: | 65 | 4E | 75 | 6D | 62 | 65 | 72 | 54 | 61 | 62 | 6C | 65 | 01 | 00 | 12 | 4C |
| 00000070h: | 6F | 63 | 61 | 6C | 56 | 61 | 72 | 69 | 61 | 62 | 6C | 65 | 54 | 61 | 62 | 6C |
| 00000080h: | 65 | 01 | 00 | 04 | 74 | 68 | 69 | 73 | 01 | 00 | 16 | 4C | 63 | 6F | 6D | 2F |
| 00000090h: | 6C | 6F | 75 | 69 | 73 | 2F | 6A | 76 | 6D | 2F | 53 | 69 | 6D | 70 | 6C | 65 |
| 000000a0h: | 3B | 01 | 00 | 0A | 53 | 6F | 75 | 72 | 63 | 65 | 46 | 69 | 6C | 65 | 01 | 00 |
| 000000b0h: | 0B | 53 | 69 | 6D | 70 | 6C | 65 | 2E | 6A | 61 | 76 | 61 | 00 | 21 | 00 | 01 |
| 000000c0h: | 00 | 03 | 00 | 00 | 00 | 00 | 00 | 01 | 00 | 01 | 00 | 05 | 00 | 06 | 00 | 01 |
| 000000d0h: | 00 | 07 | 00 | 00 | 00 | 2F | 00 | 01 | 00 | 01 | 00 | 00 | 00 | 05 | 2A | B7 |
| 000000e0h: | 00 | 08 | B1 | 00 | 00 | 00 | 02 | 00 | 0A | 00 | 00 | 00 | 06 | 00 | 01 | 00 |
| 000000f0h: | 00 | 00 | 03 | 00 | 0B | 00 | 00 | 00 | 0C | 00 | 01 | 00 | 00 | 00 | 05 | 00 |
| 00000100h: | 0C | 00 | 0D | 00 | 00 | 00 | 01 | 00 | 0E | 00 | 00 | 00 | 02 | 00 | 0F |    |

Constant pool:

#1 = Class

#2 = Utf8

#3 = Class

#4 = Utf8

#2

#4

// com/louis/jvm/Simple

com/louis/jvm/Simple

// java/lang/Object

java/lang/Object

父类索引，值为0x0003

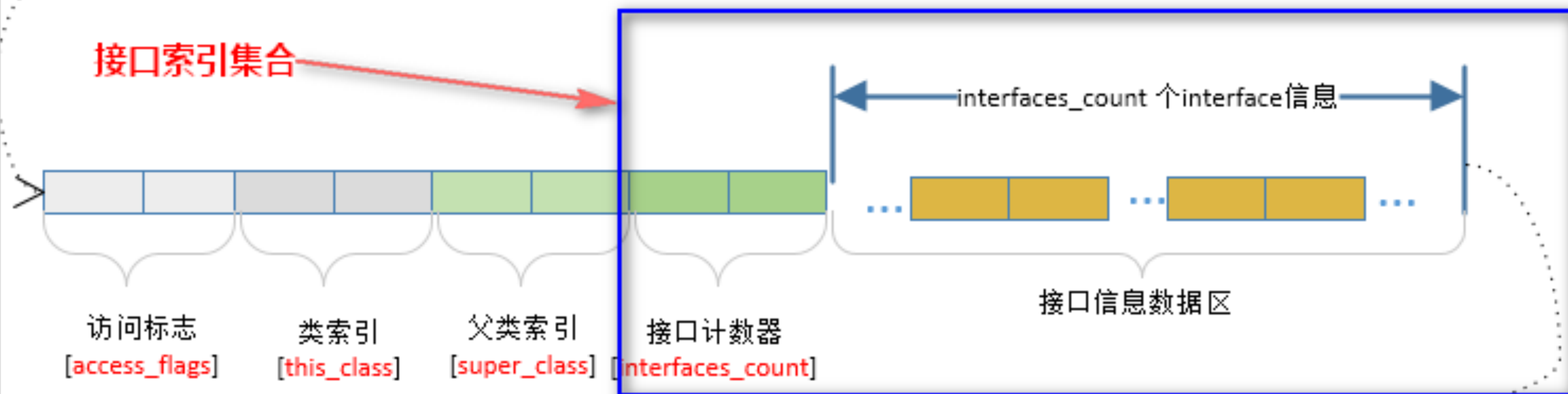
根据父类索引指向常量池中的信息可以看出：  
Simple类继承自Object类

Designed by LouLuan

<http://blog.csdn.net/luanlouis>



接口索引集合



Designed by LouLuan

<http://blog.csdn.net/luanlouis>

[java] view plain copy print ? C 🔗

```
01. /**
02.  * Worker 接口类
03.  * @author luan louis
04.  */
05. public interface Worker{
06.
07.     public void work();
08.
09. }
```

[java] view plain copy print ? C 🔗

```
01. package com.louis.jvm;
02.
03. public class Programmer implements Worker {
04.
05.     @Override
06.     public void work() {
07.         System.out.println("I'm Programmer,Just coding....");
08.     }
09. }
```

Programmer.class x

|            | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | a  | b  | c  | d  | e  | f  |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 00000000h: | CA | FE | BA | BE | 00 | 00 | 00 | 32 | 00 | 21 | 07 | 00 | 02 | 01 | 00 | 18 |
| 00000010h: | 63 | 6F | 6D | 2F | 6C | 6F | 75 | 69 | 73 | 2F | 6A | 76 | 6D | 2F | 50 | 72 |
| 00000020h: | 6F | 67 | 72 | 61 | 6D | 6D | 65 | 72 | 07 | 00 | 04 | 01 | 00 | 10 | 6A | 61 |
| 00000030h: | 76 | 61 | 2F | 6C | 61 | 6E | 67 | 2F | 4F | 62 | 6A | 65 | 63 | 74 | 07 | 00 |
| 00000040h: | 06 | 01 | 00 | 14 | 63 | 6F | 6D | 2F | 6C | 6F | 75 | 69 | 73 | 2F | 6A | 76 |
| 00000050h: | 6D | 2F | 57 | 6F | 72 | 6B | 65 | 72 | 01 | 00 | 06 | 3C | 69 | 6E | 69 | 74 |
| 00000060h: | 3E | 01 | 00 | 03 | 28 | 29 | 56 | 01 | 00 | 04 | 43 | 6F | 64 | 65 | 0A | 00 |
| 00000070h: | 03 | 00 | 0B | 0C | 00 | 07 | 00 | 08 | 01 | 00 | 0F | 4C | 69 | 6E | 65 | 4E |
| 00000080h: | 75 | 6D | 62 | 65 | 72 | 54 | 61 | 62 | 6C | 65 | 54 | 61 | 62 | 6C | 65 | 01 |
| 00000090h: | 61 | 6C | 56 | 61 | 72 | 69 | 61 | 62 | 6C | 65 | 54 | 61 | 62 | 6C | 65 | 01 |
| 000000a0h: | 00 | 04 | 74 | 68 | 69 | 73 | 01 | 00 | 1A | 4C | 63 | 6F | 6D | 2F | 6C | 6F |
| 000000b0h: | 75 | 69 | 73 | 2F | 6A | 76 | 6D | 2F | 50 | 72 | 6F | 67 | 72 | 61 | 6D | 6D |
| 000000c0h: | 65 | 72 | 3B | 01 | 00 | 04 | 77 | 6F | 72 | 6B | 09 | 00 | 12 | 00 | 14 | 07 |
| 000000d0h: | 00 | 13 | 01 | 00 | 10 | 6A | 61 | 76 | 61 | 2F | 6C | 61 | 6E | 67 | 2F | 53 |
| 000000e0h: | 79 | 73 | 74 | 65 | 6D | 0C | 00 | 15 | 00 | 16 | 01 | 00 | 03 | 6F | 75 | 74 |
| 000000f0h: | 01 | 00 | 15 | 4C | 6A | 61 | 76 | 61 | 2F | 69 | 6F | 2F | 50 | 72 | 69 | 6E |
| 00000100h: | 74 | 53 | 74 | 72 | 65 | 61 | 6D | 3B | 08 | 00 | 18 | 01 | 00 | 1E | 49 | 27 |
| 00000110h: | 6D | 20 | 50 | 72 | 6F | 67 | 72 | 61 | 6D | 6D | 65 | 72 | 2C | 4A | 75 | 73 |
| 00000120h: | 74 | 20 | 63 | 6F | 64 | 69 | 6E | 67 | 2E | 2E | 2E | 2E | 0A | 00 | 1A | 00 |
| 00000130h: | 1C | 07 | 00 | 1B | 01 | 00 | 13 | 6A | 61 | 76 | 61 | 2F | 69 | 6F | 2F | 50 |
| 00000140h: | 72 | 69 | 6E | 74 | 53 | 74 | 72 | 65 | 61 | 6D | 0C | 00 | 1D | 00 | 1E | 01 |
| 00000150h: | 00 | 07 | 70 | 72 | 69 | 6E | 74 | 6C | 6E | 01 | 00 | 15 | 28 | 4C | 6A | 61 |
| 00000160h: | 76 | 61 | 2F | 6C | 61 | 6E | 67 | 2F | 53 | 74 | 72 | 69 | 6E | 67 | 3B | 29 |
| 00000170h: | 56 | 01 | 00 | 0A | 53 | 6F | 75 | 72 | 63 | 65 | 46 | 69 | 6C | 65 | 01 | 00 |
| 00000180h: | 0F | 50 | 72 | 6F | 67 | 72 | 61 | 6D | 6D | 65 | 72 | 2E | 6A | 61 | 76 | 61 |
| 00000190h: | 00 | 21 | 00 | 01 | 00 | 03 | 00 | 01 | 00 | 05 | 00 | 00 | 00 | 02 | 00 | 01 |
| 000001a0h: | 00 | 07 | 00 | 08 | 00 | 01 | 00 | 09 | 00 | 00 | 00 | 2F | 00 | 01 | 00 | 01 |
| 000001b0h: | 00 | 00 | 00 | 05 | 2A | B7 | 00 | 0A | B1 | 00 | 00 | 00 | 02 | 00 | 0C | 00 |
| 000001c0h: | 00 | 00 | 06 | 00 | 01 | 00 | 00 | 00 | 03 | 00 | 0D | 00 | 00 | 00 | 0C | 00 |
| 000001d0h: | 01 | 00 | 00 | 00 | 05 | 00 | 0E | 00 | 0F | 00 | 00 | 00 | 01 | 00 | 10 | 00 |
| 000001e0h: | 08 | 00 | 01 | 00 | 09 | 00 | 00 | 00 | 37 | 00 | 02 | 00 | 01 | 00 | 00 | 00 |
| 000001f0h: | 09 | B2 | 00 | 11 | 12 | 17 | B6 | 00 | 19 | B1 | 00 | 00 | 00 | 02 | 00 | 0C |
| 00000200h: | 00 | 00 | 00 | 0A | 00 | 02 | 00 | 00 | 00 | 07 | 00 | 08 | 00 | 08 | 00 | 0D |
| 00000210h: | 00 | 00 | 00 | 0C | 00 | 01 | 00 | 00 | 00 | 09 | 00 | 0E | 00 | 0F | 00 | 00 |
| 00000220h: | 00 | 01 | 00 | 1F | 00 | 00 | 00 | 02 | 00 | 20 |    |    |    |    |    |    |

常量池

访问标志 类索引 父类索引 接口索引集合

接口索引集合区域，接口计数器为01，表示就实现了一个接口，该接口引用指向常量池的第5个常量池项

Constant pool:

#1 = Class

#2 = Utf8

#3 = Class

#4 = Utf8

#5 = Class

#6 = Utf8

#2

#4

#6

// com/louis/jvm/Programmer

com/louis/jvm/Programmer

// java/lang/Object

java/lang/Object

// com/louis/jvm/Worker

com/louis/jvm/Worker

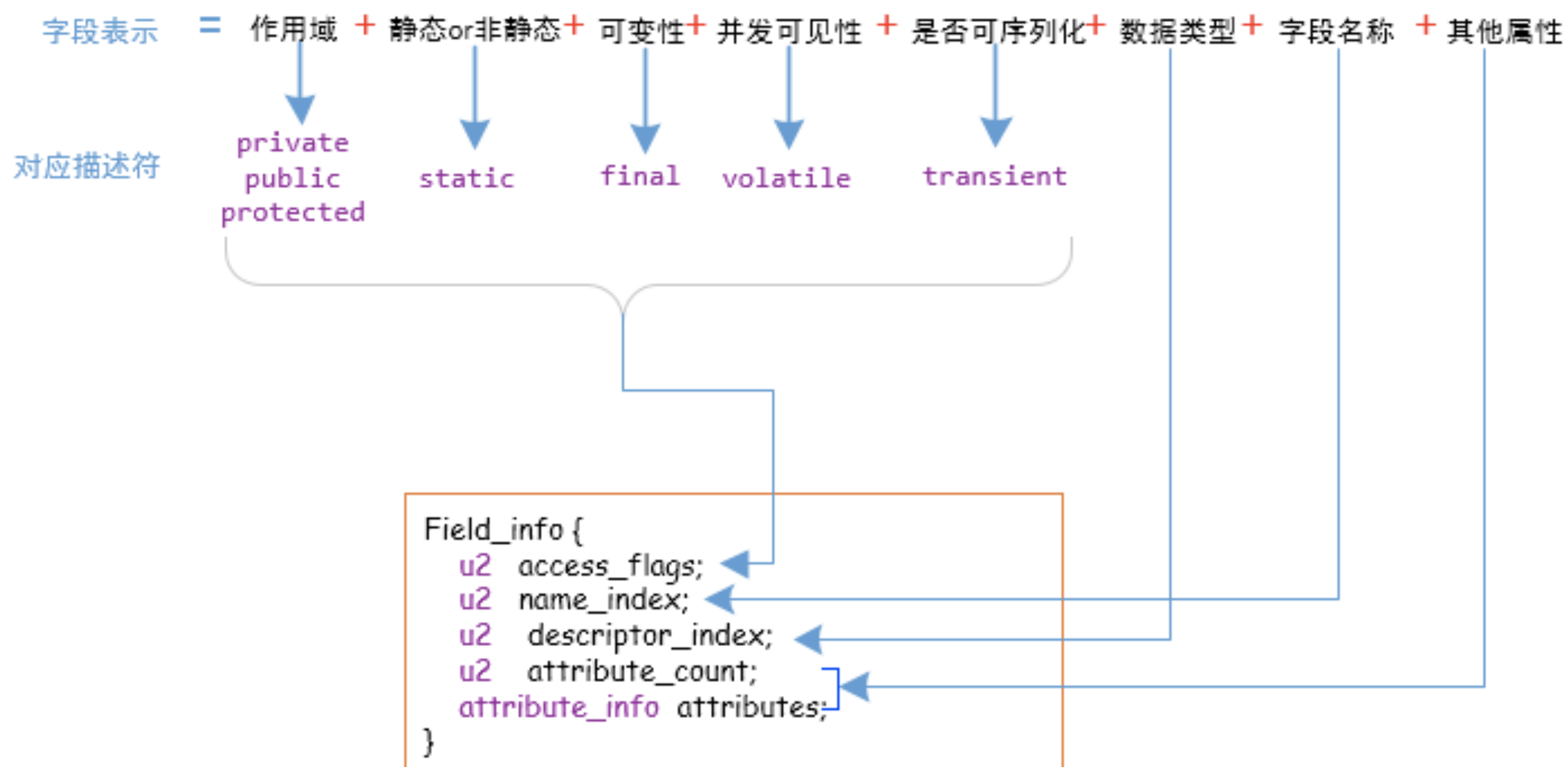
Designed by LouLuan

http://blog.csdn.net/luanlouis



## 06、字段表

一个类中定义的字段会被存储在字段表。



JVM定义了field\_info结构体来描述关于字段的描述。

作用域、静态或非静态、可变性、并发可见性、是否可序列化这些描述对字段来说就是有和无的关系，适合使用访问标志来表示。

# Sample Code

```
package com.louis.jvm;  
  
public class Simple {  
    private transient static final String str = "This is a test";  
}
```

## Field\_info



 表示一个字节

## ConstantValue

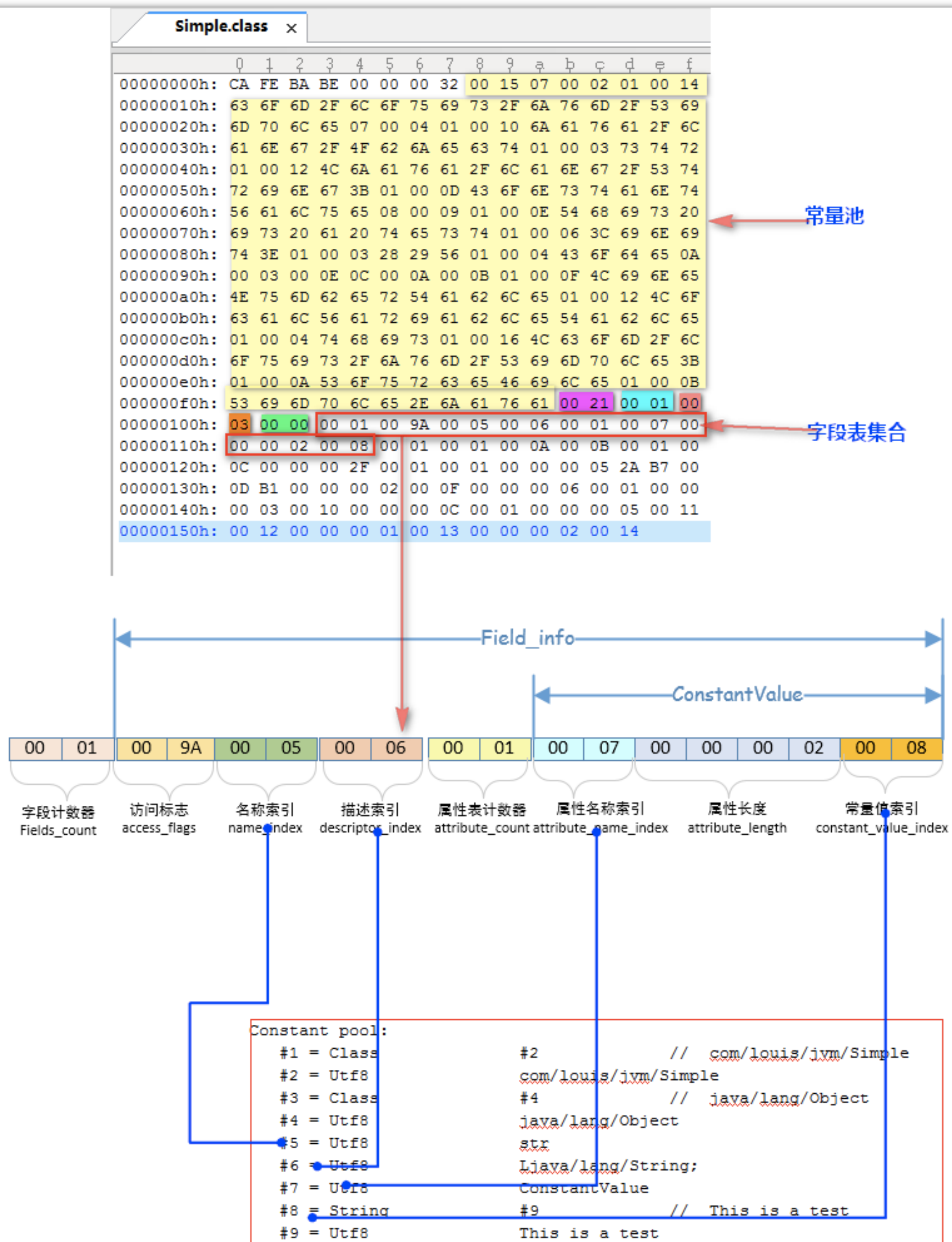


属性名称索引的值是指向常量池中某个常量池项的索引，该常量池项表示的字符串为“ConstantValue”

属性长度中的值固定为2

常量池索引的值是指向常量池中某个常量池项的索引，表示要复制给field字段的值



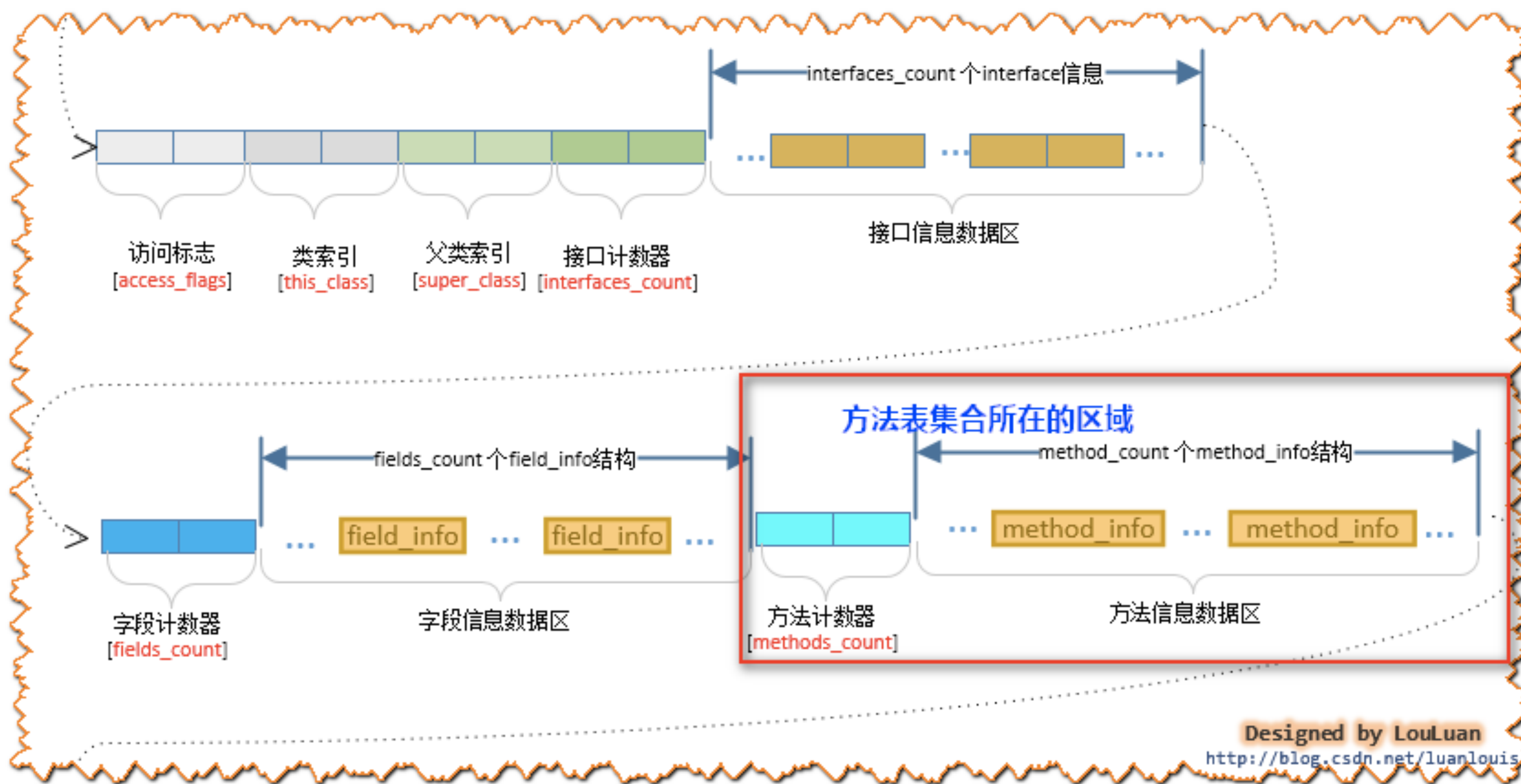


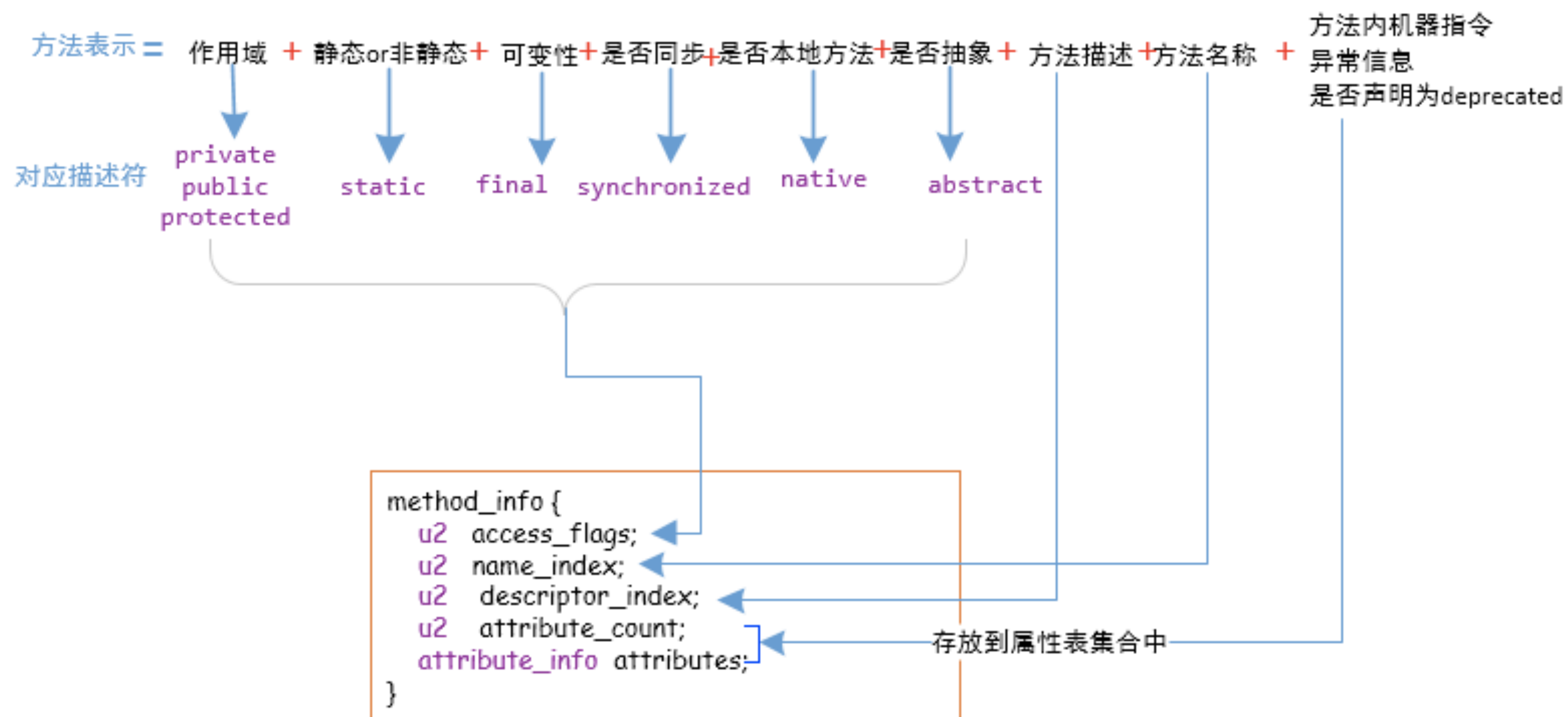
## 描述索引 (description\_index)

- 1) 对于基本数据类型来说, 使用一个字符来表示, 比如说 I 对应的是 int, B 对应的是 byte。
- 2) 对于引用数据类型来说, 使用 L\*\*\*; 的方式来表示, L 开头, ; 结束, 比如字符串类型为 Ljava/lang/String;。
- 3) 对于数组来说, 会用一个前置的 [ 来表示, 比如说字符串数组为 [Ljava/lang/String;。

## 06、方法表

方法表和字段表类似，区别是用来存储方法的信息，包括方法名，方法的参数，方法的签名。

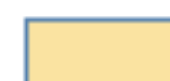






## Method\_info



 表示一个字节

## Attribute\_info



## Attribute\_info



“Code” ,表明此属性表(attribute\_info)记录的是机器码

“Exceptions” ,此属性表记录的是异常信息

“Deprecated” ,此属性表被@Deprecated修饰

“Synthetic” ,此属性表表示被编译器自动生成

属性名称索引 (attribute\_name\_index) 占有两个字节，其中的值是指向了常量池中的某一项，该项表示的字符串表明该attribute\_info表示的是什么类型的属性表。如：如上所示，如果指向的常量池项表示“Code”，则表明该属性表(attribute\_info)表示的是代码实现，即可执行的机器码；

对于不同类型的属性表，它们的属性长度和组织形式会有所不同。

属性长度(attribute\_length)占有两个字节，它的值表示紧跟其后的多少个字节是拿来表示这个属性信息的。即如果属性长度中的值为N，则表示它后面的N个字节是用来表示属性信息的。

属性值(attribute\_value)是由若干个字节构成的，字节的个数由属性长度中的值决定。

Code属性表信息 = 机器指令 + 异常处理跳转信息 + Java源码行号和机器码对应关系 + 局部变量表描述信息

```
Code_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u2 max_stack;  
    u2 max_locals;  
    u4 code_length;  
    u1 code[code_length];  
    u2 exception_table_length;  
    {  
        u2 start_pc;  
        u2 end_pc;  
        u2 handler_pc;  
        u2 catch_type;  
    } exception_table[exception_table_length];  
    u2 attributes_count;  
    u1 attribute_info attributes[attributes_count];  
}
```

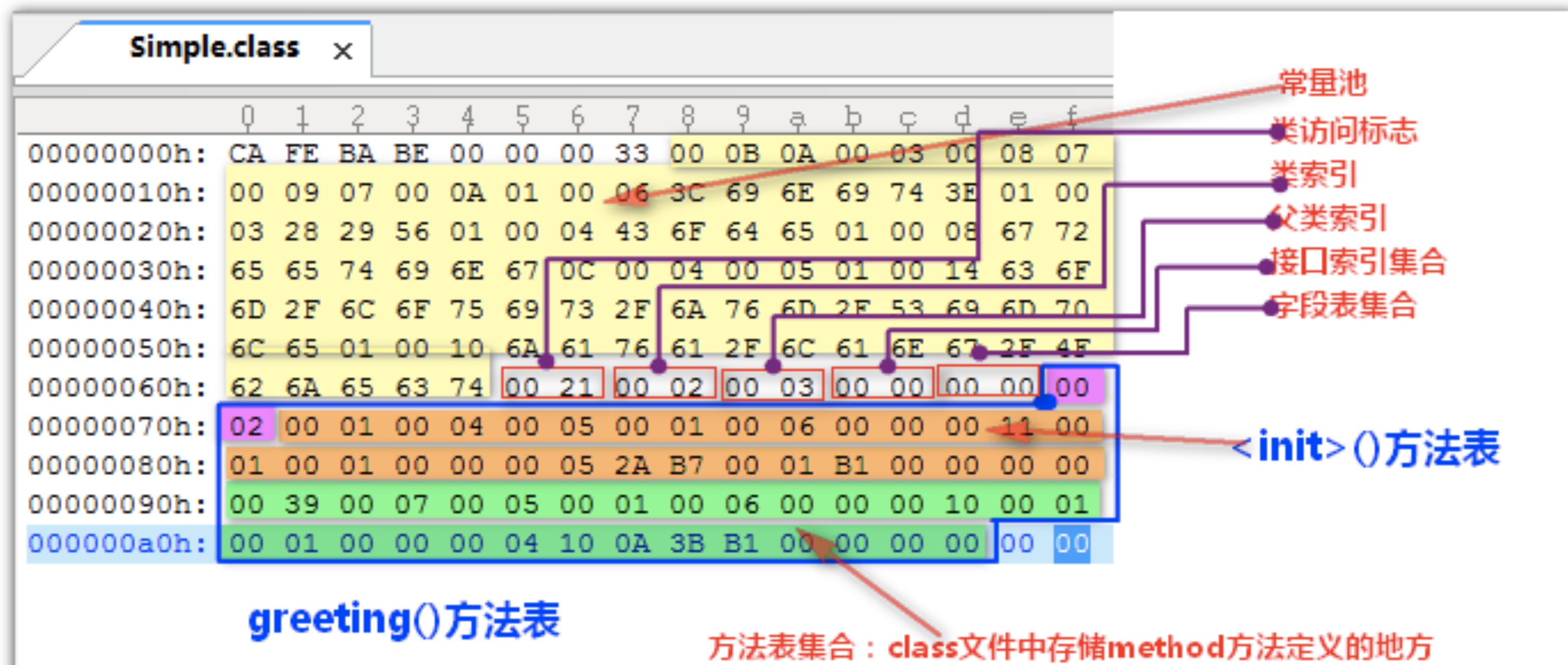
以LineNumberTable类型的  
属性表保存在属性表集合  
中

以LocalVariableTable类型  
的属性表保存在属性表集合  
中



# Sample Code - 方法

```
package com.louis.jvm;  
  
public class Simple {  
  
    public static synchronized final void greeting(){  
  
        int a = 10;  
  
    }  
  
}
```



Simple.class文件信息组织情况

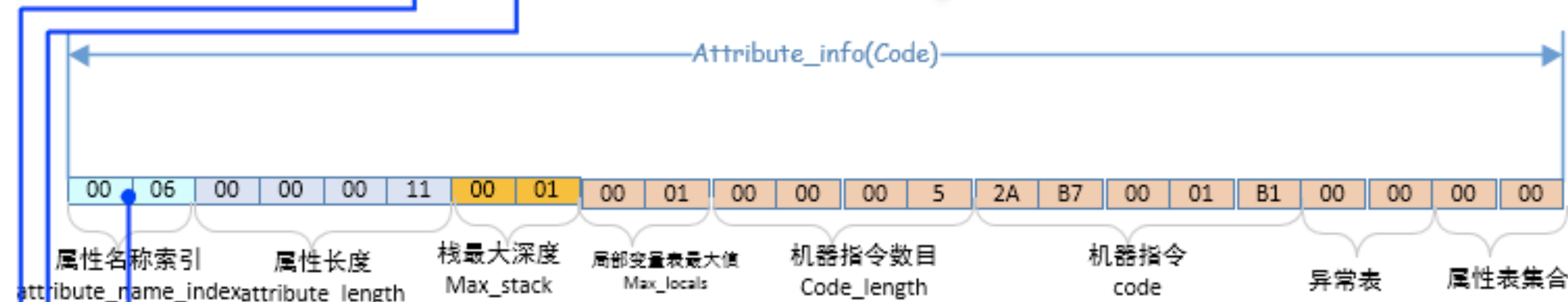
```

00000040h: 6D 2F 6C 6F 75 69 73 2F 6A 76 6D 2F 53 69 6D 70
00000050h: 6C 65 01 00 10 6A 61 76 61 2F 6C 61 6E 67 2F 4F
00000060h: 62 6A 65 63 74 00 21 00 02 00 03 00 00 00 00 00
00000070h: 02 00 01 00 04 00 05 00 01 00 06 00 00 00 11 00
00000080h: 01 00 01 00 00 00 05 2A B7 00 01 B1 00 00 00 00
00000090h: 00 39 00 07 00 05 00 01 00 06 00 00 00 10 00 01
000000a0h: 00 01 00 00 00 04 10 0A 3B B1 00 00 00 00 00 00

```

<init>()方法表信息

### Method\_info



### Constant pool:

```

#1 = Methodref      #3.#8      // java/lang/Object."<init>":()V
#2 = Class           #9          // com/louis/jvm/Simple
#3 = Class           #10         // java/lang/Object
#4 = Utf8            <init>
#5 = Utf8            ()V
#6 = Utf8            Code
#7 = Utf8            greeting
#8 = NameAndType     #4:#5      // "<init>":()V
#9 = Utf8            com/louis/jvm/Simple
#10 = Utf8           java/lang/Object

```



```
public com.louis.jvm.Simple();
```

```
  flags: ACC_PUBLIC
```

```
Code:
```

```
  stack=1, locals=1, args_size=1
```

```
    0: aload_0      2A
```

```
    1: invokespecial #1 B7 00 01
```

```
    4: return      B1
```

```
// Method java/lang/Object."<init>":()V
```

Designed by LouLuan

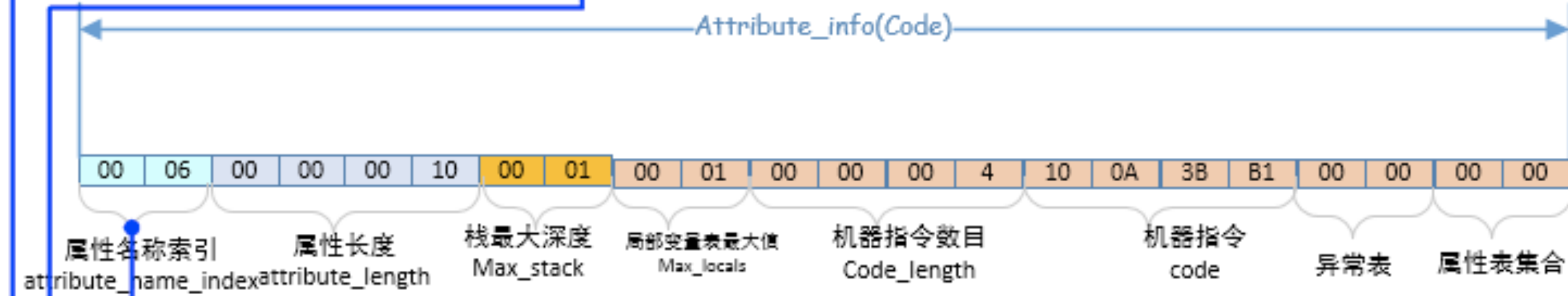
<http://blog.csdn.net/luanlouis>

```

00000070h: 02 00 01 00 00 04 00 05 00 01 00 06 00 00 00 11 00
00000080h: 01 00 01 00 00 00 05 2A B7 00 01 B1 00 00 00 00
00000090h: 00 39 00 07 00 05 00 01 00 06 00 00 00 10 00 01
000000a0h: 00 01 00 00 00 04 10 0A 3B B1 00 00 00 00 00 00

```

### Method\_info



### Constant pool:

```

#1 = Methodref      #3.#8      // java/lang/Object."<init>":()V
#2 = Class          #9          // com/louis/jvm/Simple
#3 = Class          #10         // java/lang/Object
#4 = Utf8           <init>
#5 = Utf8           ()V
#6 = Utf8           Code
#7 = Utf8           greeting
#8 = NameAndType    #4:#5      // "<init>":()V
#9 = Utf8           com/louis/jvm/Simple
#10 = Utf8          java/lang/Object

```

```
public static final synchronized void greeting();  
    flags: ACC_PUBLIC, ACC_STATIC, ACC_FINAL, ACC_SYNCHRONIZED
```

Code:

```
    stack=1, locals=1, args_size=0
```

```
    0: bipush          10  10 0A : 将单字节10 推送到栈上
```

```
    2: istore_0          3B : 取栈顶的int类型的值赋给局部变量表的第一个变量
```

```
    3: return  B1 : 无结果返回
```

```
}
```

Designed by LouLuan

<http://blog.csdn.net/luanlouis>



# 实例文件

```
package org.fenixsoft.clazz;
```

```
public class TestClass{
```

```
    private int m;
```

```
    public int inc(){
```

```
        return m+1;
```

```
    }
```

```
}
```

```
promote:~ qinliu$ javac TestClass.java
promote:~ qinliu$ javap -verbose TestClass
警告: 二进制文件TestClass包含org.fenixsoft.clazz.TestClass
Classfile /Users/qinliu/TestClass.class
  Last modified 2015-4-15; size 295 bytes
  MD5 checksum 81f2ab948a7a3068839b61a8f91f634b
  Compiled from "TestClass.java"
public class org.fenixsoft.clazz.TestClass
  minor version: 0
  major version: 52
  flags: ACC_PUBLIC, ACC_SUPER
Constant pool:
  #1 = Methodref      #4.#15      // java/lang/Object."<init>":()V
  #2 = Fieldref       #3.#16      // org/fenixsoft/clazz/TestClass.m:I
  #3 = Class          #17         // org/fenixsoft/clazz/TestClass
  #4 = Class          #18         // java/lang/Object
  #5 = Utf8           m
  #6 = Utf8           I
  #7 = Utf8           <init>
  #8 = Utf8           ()V
  #9 = Utf8           Code
 #10 = Utf8          LineNumberTable
 #11 = Utf8           inc
 #12 = Utf8           ()I
 #13 = Utf8           SourceFile
 #14 = Utf8           TestClass.java
 #15 = NameAndType    #7:#8      // "<init>":()V
 #16 = NameAndType    #5:#6      // m:I
 #17 = Utf8           org/fenixsoft/clazz/TestClass
 #18 = Utf8           java/lang/Object
```

```
{
  public org.fenixsoft.clazz.TestClass();
    descriptor: ()V
    flags: ACC_PUBLIC
    Code:
      stack=1, locals=1, args_size=1
      0: aload_0
      1: invokespecial #1             // Method j
      4: return
    LineNumberTable:
      line 3: 0
  public int inc();
    descriptor: ()I
    flags: ACC_PUBLIC
    Code:
      stack=2, locals=1, args_size=1
      0: aload_0
      1: getfield      #2             // Field m:
      4: iconst_1
      5: iadd
      6: ireturn
    LineNumberTable:
      line 8: 0
}
```

SourceFile: "TestClass.java"

# 小结

- class 文件是一串连续的二进制，由 0 和 1 组成，但我们仍然可以借助一些工具来看清楚它的真面目。
- class 文件的内容通常可以分为下面这几部分，魔数、版本号、常量池、访问标记、类索引、父类索引、接口索引、字段表、方法表、属性表。
- 常量池包含了类、接口、字段和方法的符号引用，以及字符串字面量和数值常量。
- 访问标记用于识别类或接口的访问信息，比如说是不是 public | private | protected，是不是 static，是不是 final 等。
- 类索引、父类索引和接口索引用来确定类的继承关系。
- 字段表用来存储字段的信息，包括字段名，字段的参数，字段的签名。
- 方法表用来存储方法的信息，包括方法名，方法的参数，方法的签名。
- 属性表用来存储属性的信息，包括字段的初始值，方法的字节码指令等。



# 深度理解运行时数据区域



JVM 规范定义的运行时数据区

虚拟机栈

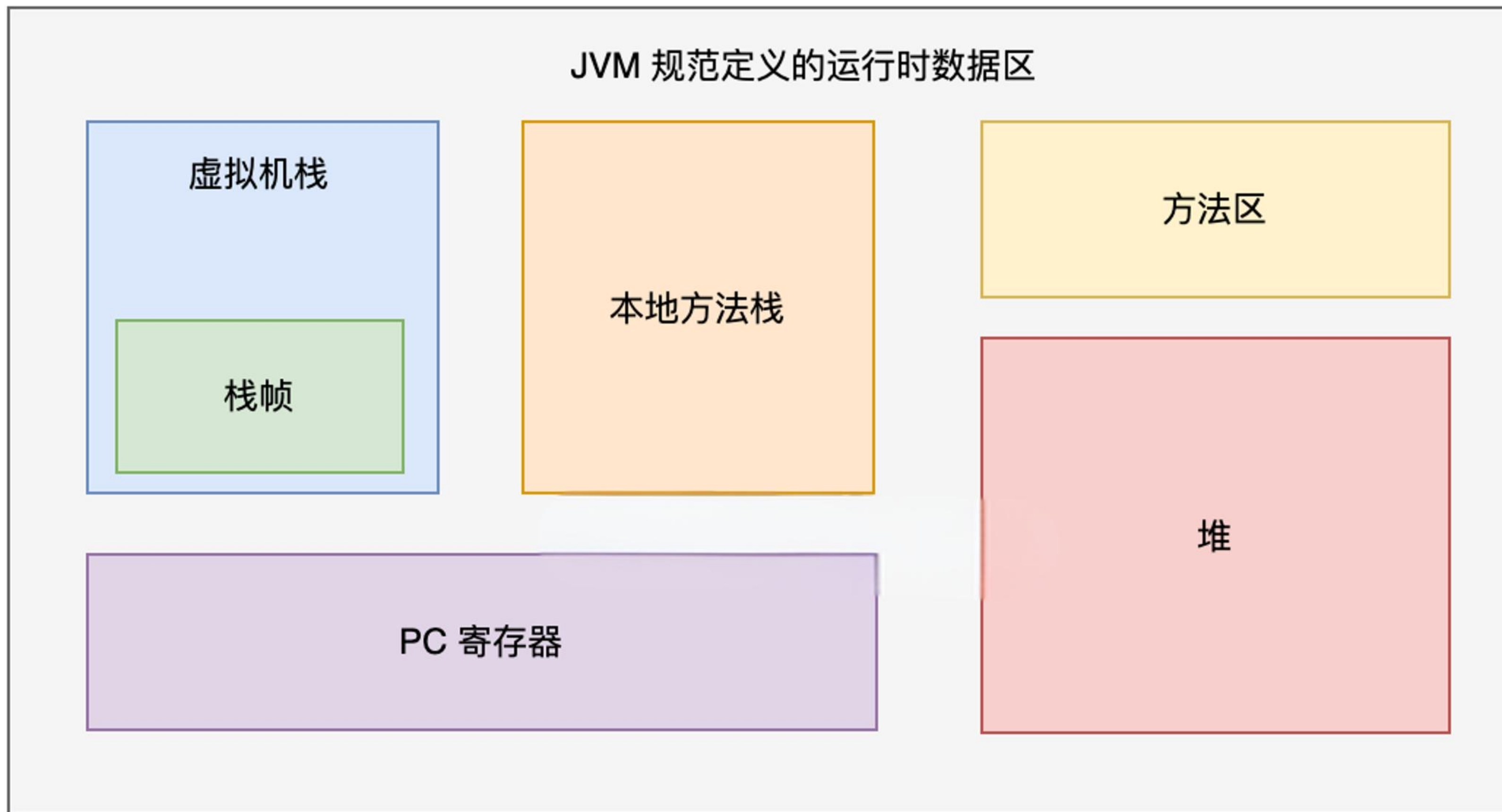
栈帧

本地方法栈

方法区

堆

PC 寄存器



# 程序计数器

- 程序计数器（Program Counter Register）所占的内存空间不大，很小很小一块，可以看作是当前线程所执行的字节码指令的行号指示器。字节码解释器会在工作的时候改变这个计数器的值来选取下一条需要执行的字节码指令，像分支、循环、跳转、异常处理、线程恢复等功能都需要依赖这个计数器来完成。
- 在 JVM 中，多线程是通过线程轮流切换来获得 CPU 执行时间的，因此，在任一具体时刻，一个 CPU 的内核只会执行一条线程中的指令，因此，为了线程切换后能恢复到正确的执行位置，每个线程都需要有一个独立的程序计数器，并且不能互相干扰，否则就会影响到程序的正常执行次序。即程序计数器是线程私有的。
- 如果线程执行的是非本地方法，则程序计数器中保存的是当前需要执行的指令地址；如果线程执行的是本地方法，则程序计数器中的值是 undefined。因为本地方法大多是通过 C/C++ 实现的，并未编译成需要执行的字节码指令。

# 程序计数器

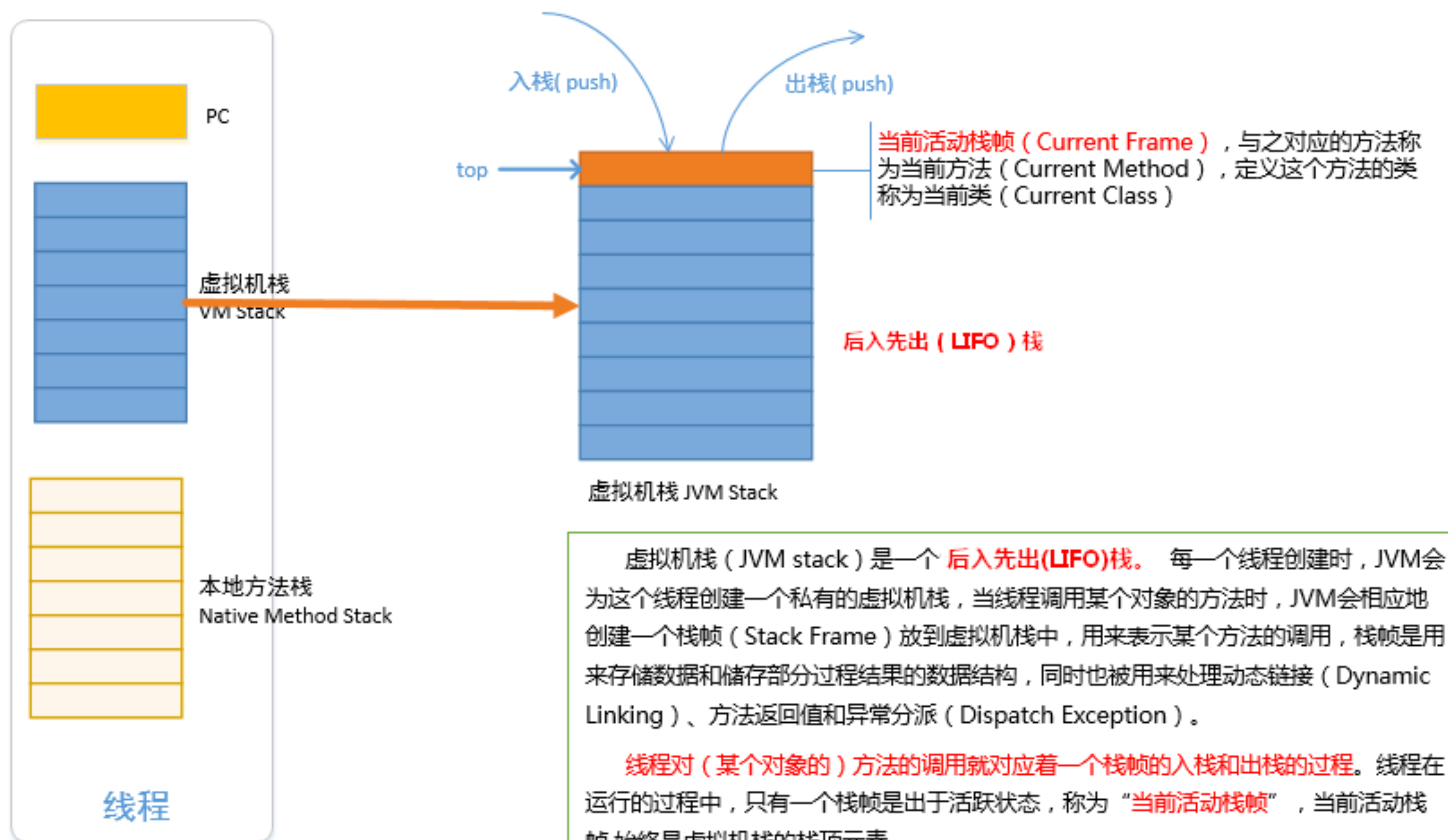
```
public static int add (int a, int b) {  
    return a + b;  
}
```

```
0: iload_0    // 从局部变量表中加载变量 a 到操作数栈  
1: iload_1    // 从局部变量表中加载变量 b 到操作数栈  
2: iadd       // 两数相加  
3: ireturn    // 返回
```

- 1.初始状态：当方法开始执行时，PC 计数器设置为 0，指向第一条指令 0: iload\_0。
- 2.执行第一条指令：
  - 1.执行 iload\_0 指令，将局部变量表中索引为 0 的整数（即方法的第一个参数 a）加载到操作数栈顶。
  - 2.执行完成后，PC 计数器更新为 1，指向下一条指令 1: iload\_1。
- 3.执行第二条指令：
  - 1.执行 iload\_1 指令，将局部变量表中索引为 1 的整数（即方法的第二个参数 b）加载到操作数栈顶。
  - 2.执行完成后，PC 计数器更新为 2，指向下一条指令 2: iadd。
- 4.执行第三条指令：
  - 1.执行 iadd 指令，弹出操作数栈顶的两个整数（即 a 和 b），将它们相加，然后将结果压入操作数栈顶。
  - 2.执行完成后，PC 计数器更新为 3，指向下一条指令 3: ireturn。
- 5.执行最后一条指令：
  - 1.执行 ireturn 指令，弹出操作数栈顶的整数（即 a + b 的结果），并将这个值作为方法的返回值。
  - 2.方法执行完成，控制权返回到方法调用者。



## Topic 2. 虚拟机栈是什么？虚拟机栈里有什么？



虚拟机栈 (JVM stack) 是一个 **后入先出(LIFO)栈**。每一个线程创建时，JVM会为此线程创建一个私有的虚拟机栈，当线程调用某个对象的方法时，JVM会相应地创建一个栈帧 (Stack Frame) 放到虚拟机栈中，用来表示某个方法的调用，栈帧是用来存储数据和储存部分过程结果的数据结构，同时也被用来处理动态链接 (Dynamic Linking)、方法返回值和异常分派 (Dispatch Exception)。

**线程对 (某个对象的) 方法的调用就对应着一个栈帧的入栈和出栈的过程。**线程在运行的过程中，只有一个栈帧是处于活跃状态，称为“**当前活动栈帧**”，当前活动栈帧始终是虚拟机栈的栈顶元素。

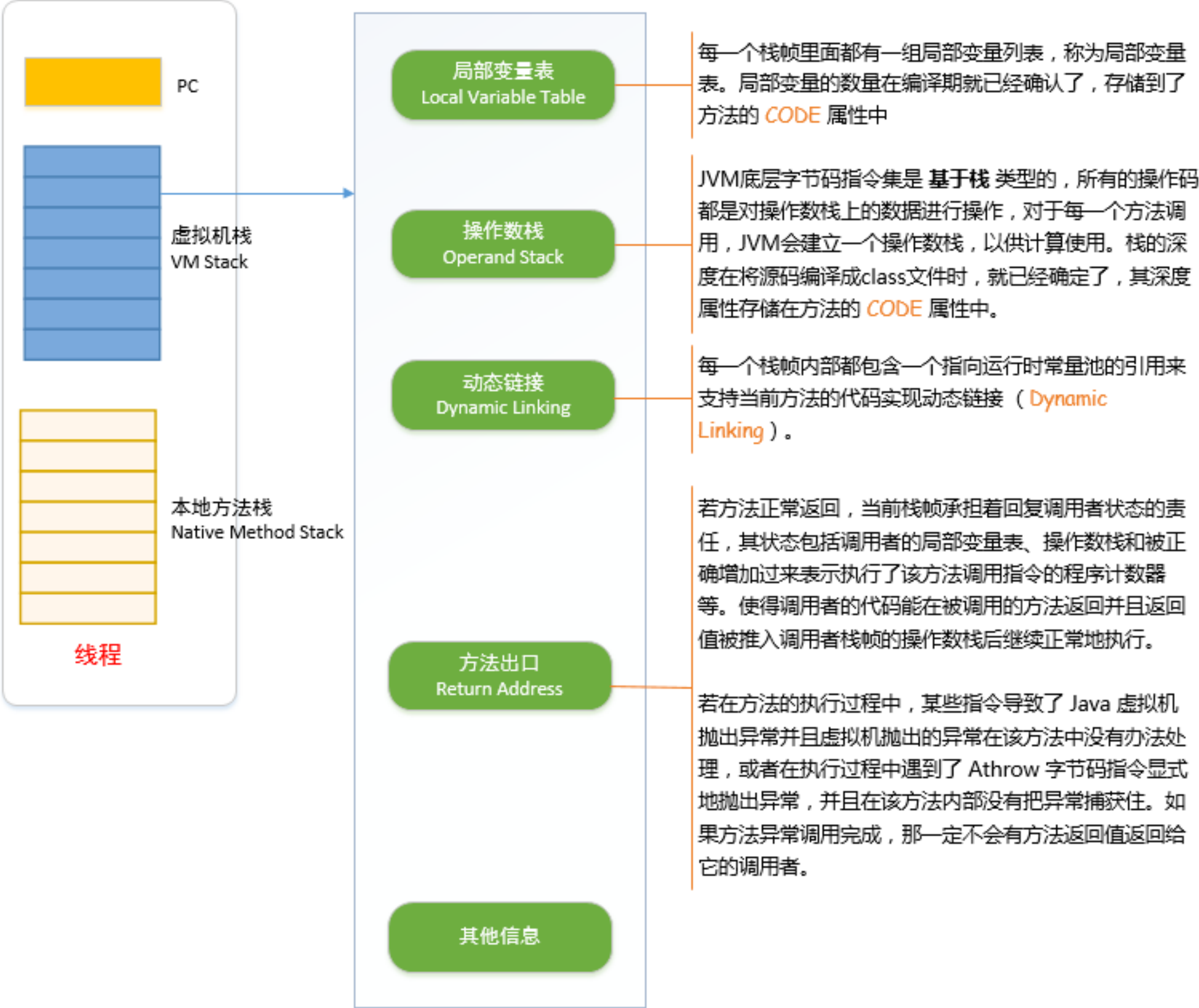
TIPS：

在线程创建的时候，都会为其创建一个私有的虚拟机栈，即虚拟机栈的生命周期跟线程是一样的



### Topic 3. 栈帧是什么？栈帧里有什么？

#### 栈帧 Stack Frame



虚拟机栈操作的基本元素就是栈帧，栈帧主要包含了局部变量表、操作数栈、动态链接以及方法返回地址。栈帧是一个先进后出的数据结构，每个方法从调用到执行完成都会对应一个栈帧在虚拟机栈中入栈和出栈。

# 虚拟机栈

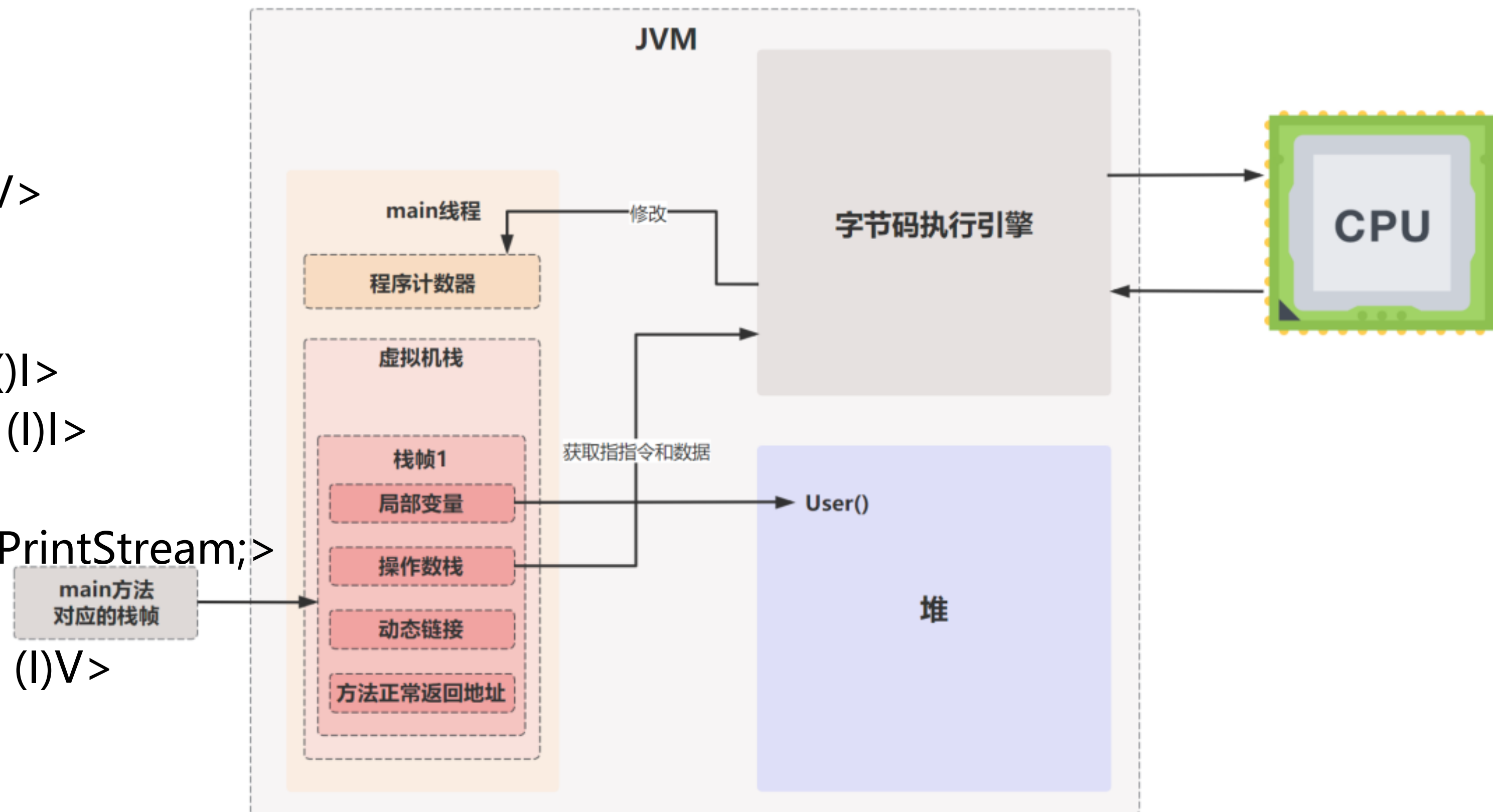
```
public class Test {  
    public static void main(String[] args) {  
        User user = new User();  
        int result = calculate(user.getAge());  
        System.out.println(result);  
    }  
    private static int calculate(int age) {  
        int data = age + 3;  
        return data;  
    }  
}
```

一个 Test 类，main 方法里 new 了一个 User 对象，会将 User 的 age 作为参数传递给静态方法 calculate 进行一个简单的加法操作并返回，最后打印到控制台。

# 虚拟机栈

- 1、JVM 完成 .class 文件加载之后，会创建一个名为"main"的线程，该线程会自动调用名为"main"的静态方法，这是 Java 程序的入口点；
- 2、main 线程在执行 main 方法时，JVM 会在虚拟机栈中压入 main 方法对应的栈帧；
- 3、栈帧的操作数栈中存储了操作的数据，JVM 执行字节码指令的时候会从操作数栈中获取数据，执行计算操作后会将结果再次压入操作数栈中；

```
0 new #7 <org/example/User>
3 dup
4 invokespecial #9 <org/example/User.<init> : ()V>
7 astore_1
8 aload_1
9 invokevirtual #10 <org/example/User.getAge : ()I>
12 invokestatic #14 <org/example/Test.calculate : (I)I>
15 istore_2
16 getstatic #20 <java/lang/System.out :Ljava/io/PrintStream;>
19 iload_2
20 invokevirtual #26 <java/io/PrintStream.println : (I)V>
23 return
```

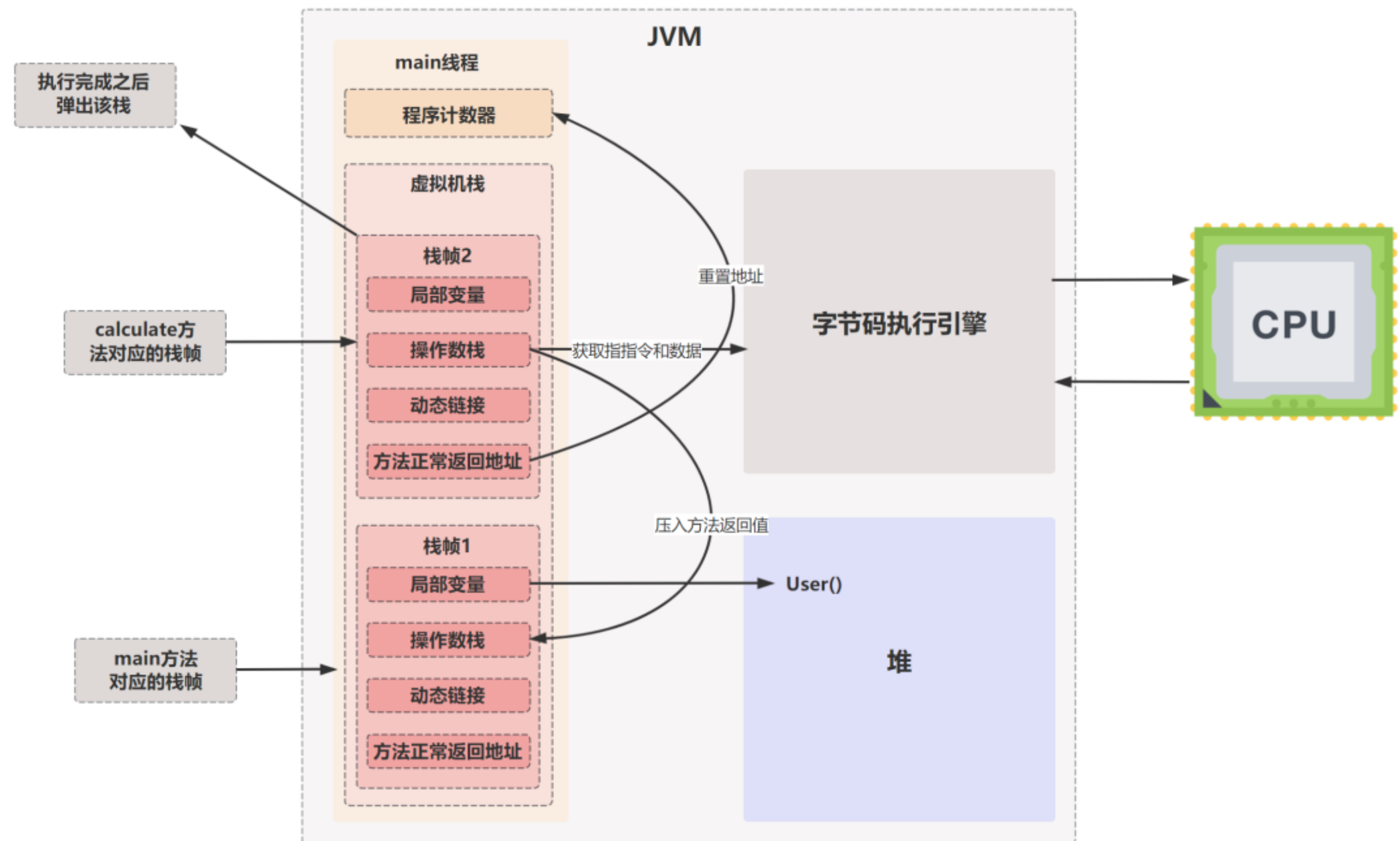




# 虚拟机栈

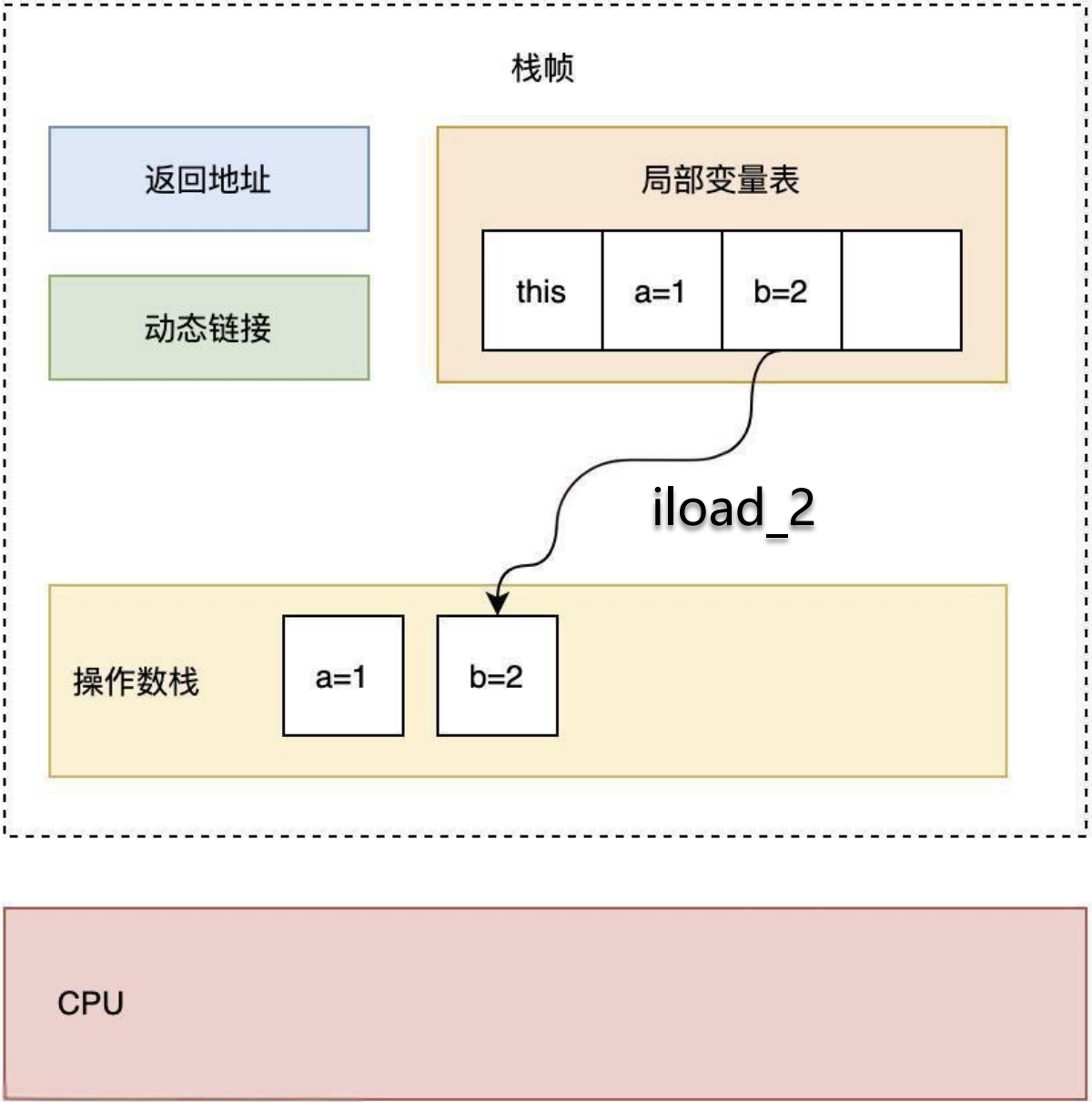
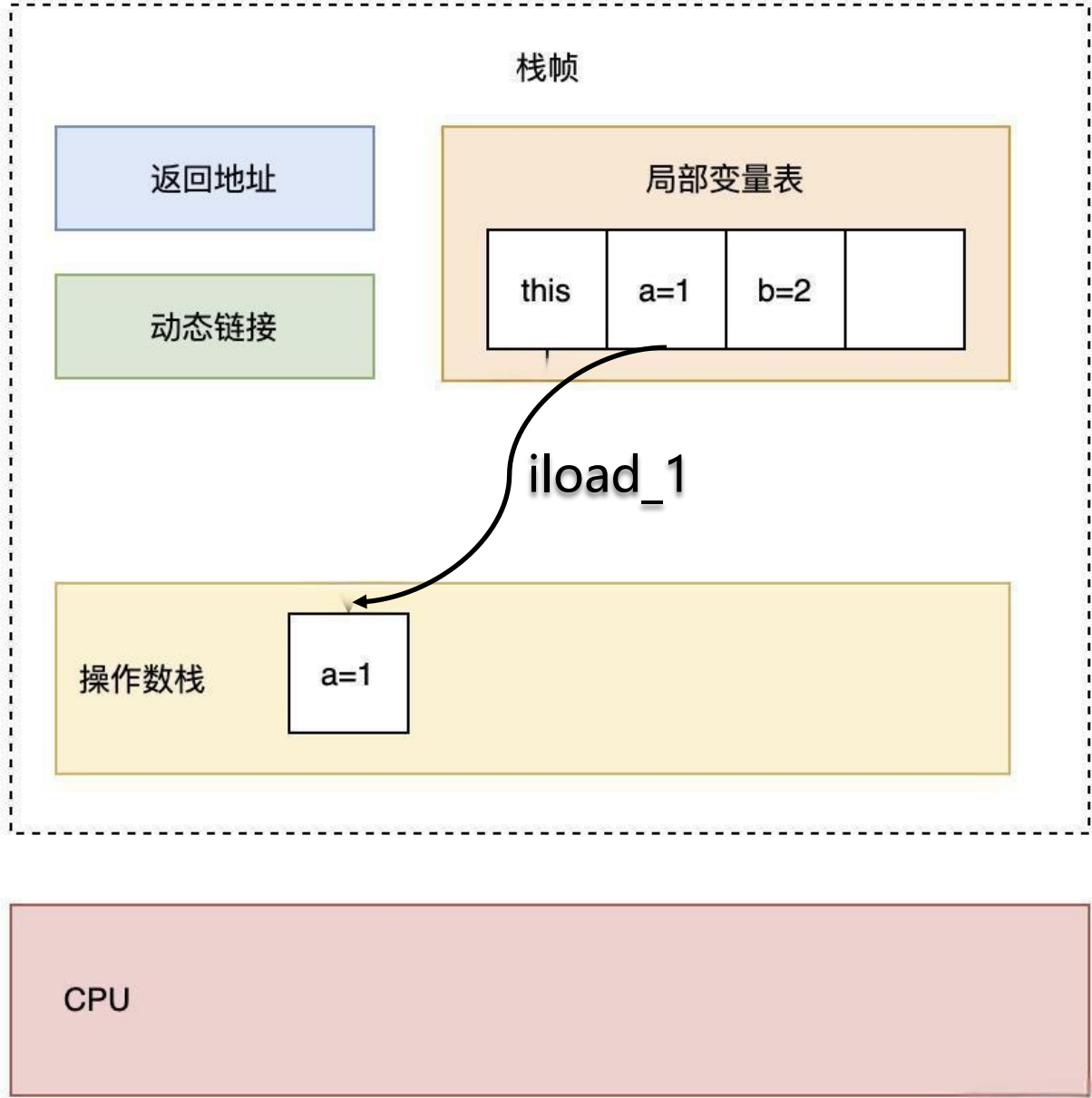
- 4、当进行 calculate 方法调用的时候，虚拟机栈继续压入 calculate 方法对应的栈帧
- 5、PC 计数器中存储了下一条需要执行的字节码指令地址。
- 6、当 calculate 方法执行完成后，对应的栈帧将从虚拟机栈中弹出，方法执行的结果会被压入 main 栈帧中的操作数栈中，而方法返回地址被重置到 main 线程的 PC 寄存器中，以便于后续字节码执行引擎从 PC 寄存器中获取下一条命令的地址。

```
0 iload_0  
1 iconst_3  
2 iadd  
3 istore_1  
4 iload_1  
5 ireturn
```



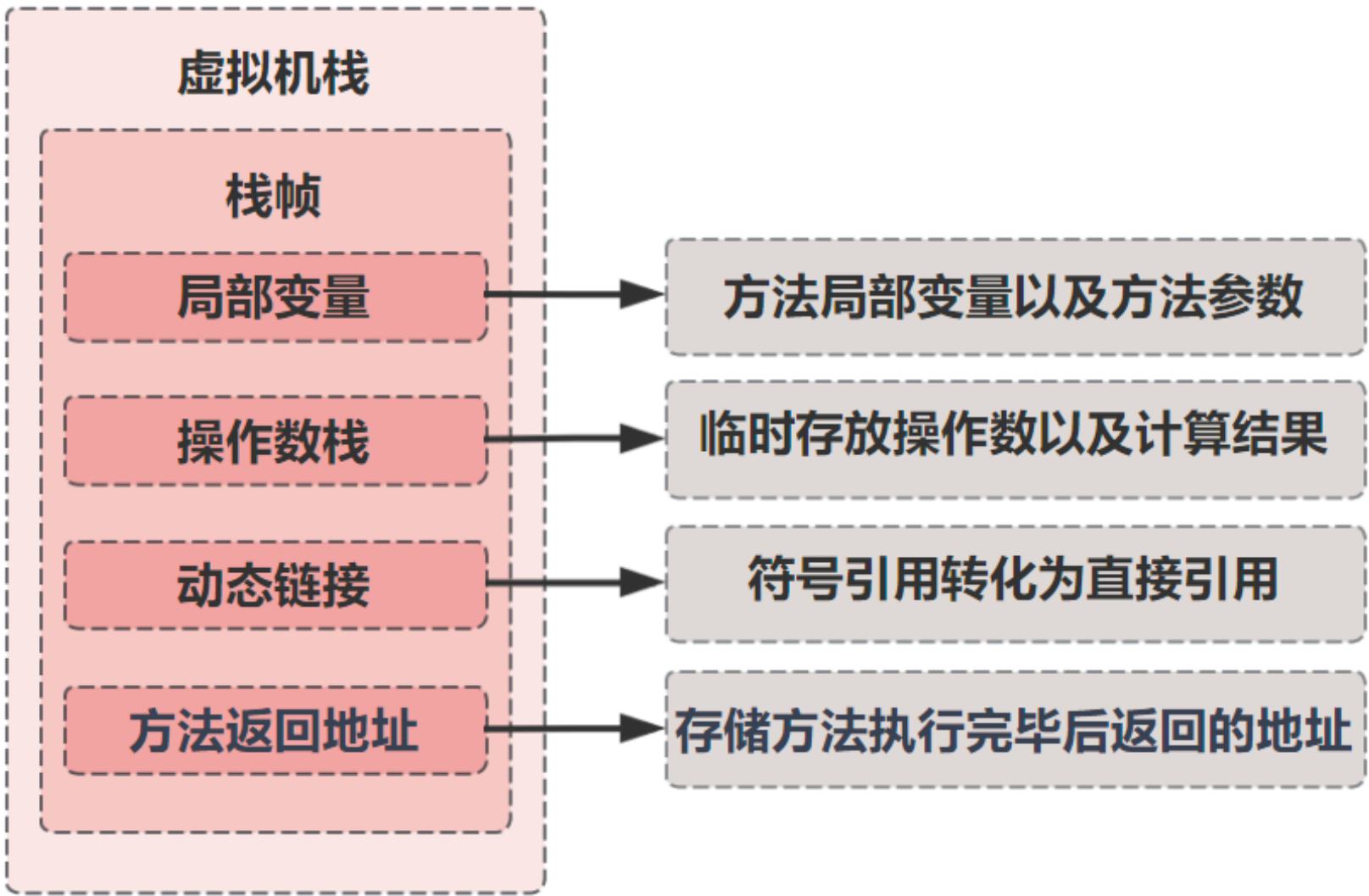
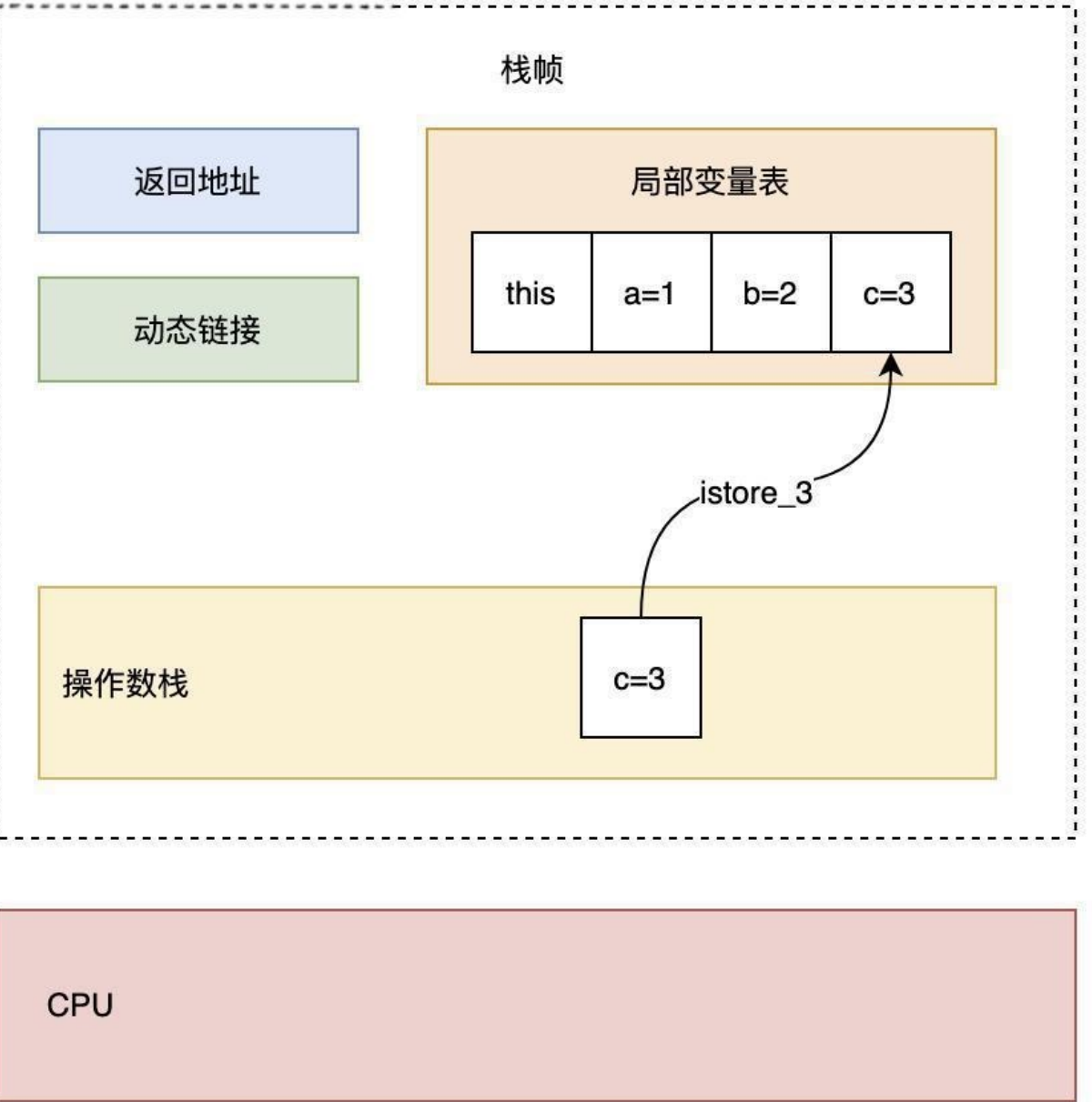
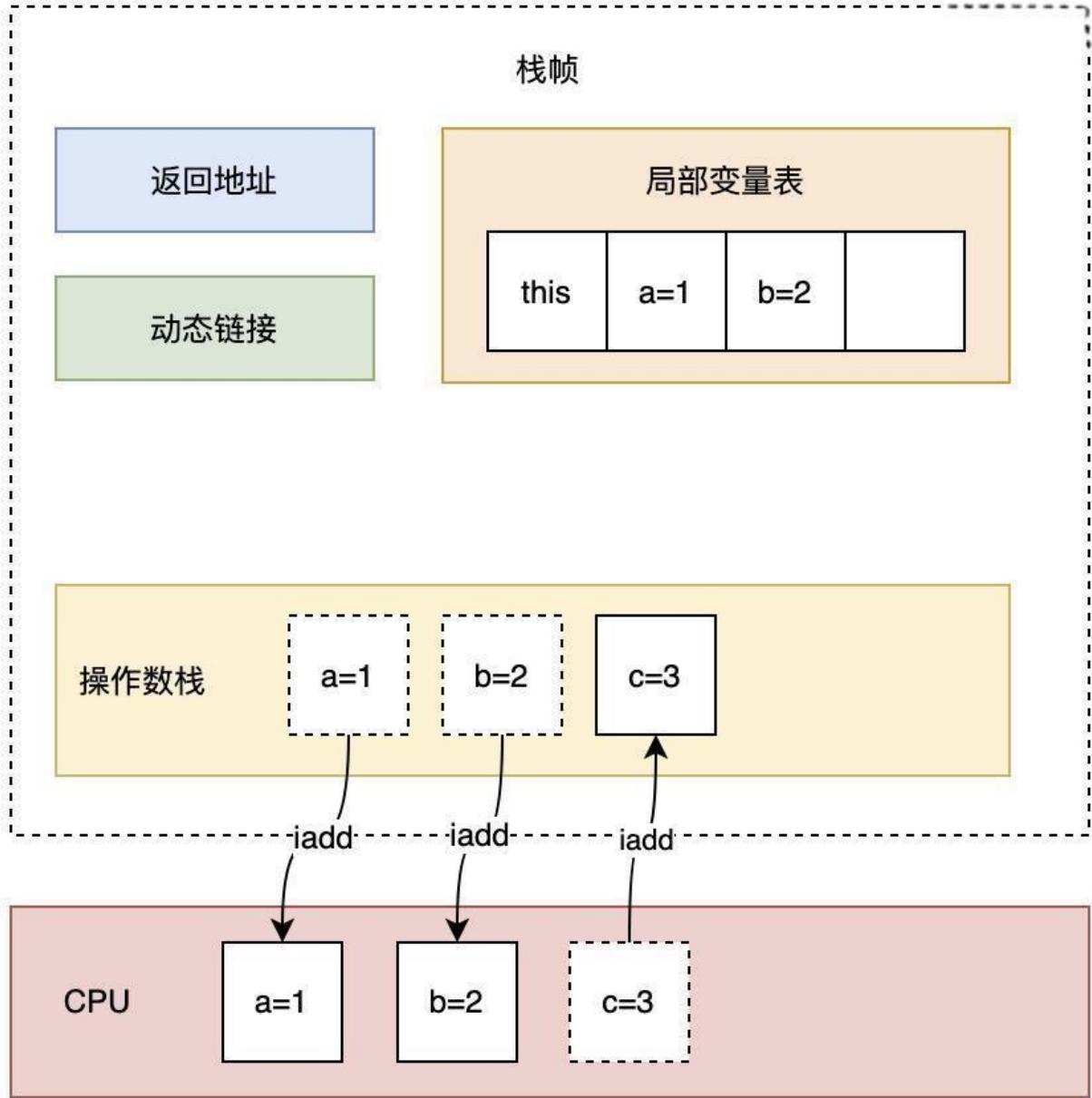


# 栈帧



```
Public int add (int a, int b) {  
    int result = a + b;  
    return result;  
}
```

```
0 iload_1  
1 iload_2  
2 iadd  
3 istore_3  
4 iload_3  
5 ireturn
```



# 动态链接

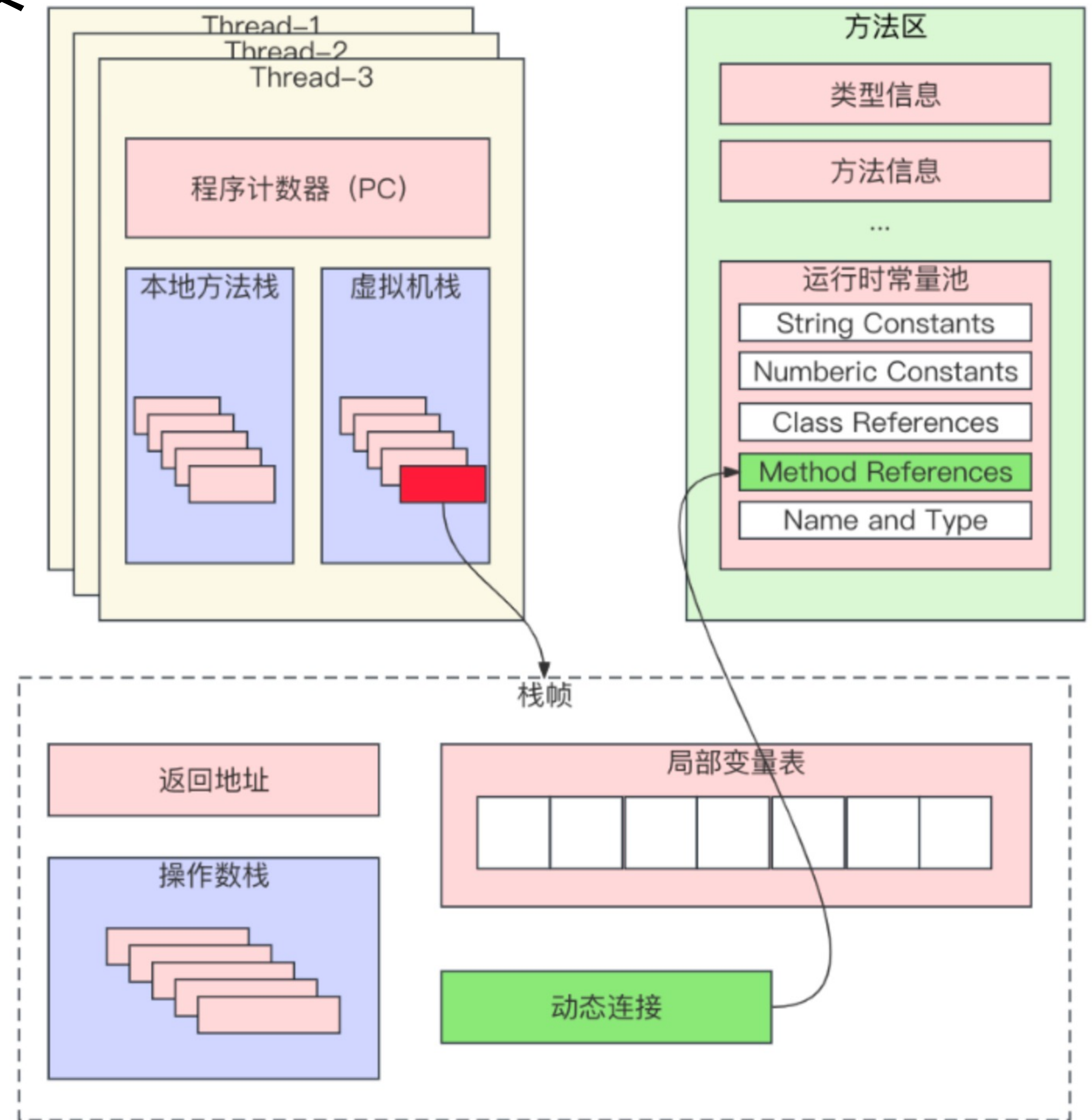
每个栈帧都包含了一个指向运行时常量池中该栈帧所属方法的引用，持有这个引用是为了支持方法调用过程中的动态链接（Dynamic Linking）。

动态链接是实现Java多态特性的基础机制：  
在运行时确定实际要调用的方法版本

1. 方法引用存储：每个栈帧保存一个指向运行时常量池中方法符号引用的指针
2. 符号引用解析：第一次使用时，将符号引用解析为直接引用（方法实际入口地址）
3. 方法调用：通过这个连接找到并执行目标方法

符号引用是一组描述性的字符串，用来唯一标识一个类、方法、字段等目标，与虚拟机的内存布局无关。  
java/lang/Object

直接引用是能够直接定位到目标的指针、偏移量或句柄，与虚拟机的内存布局相关。



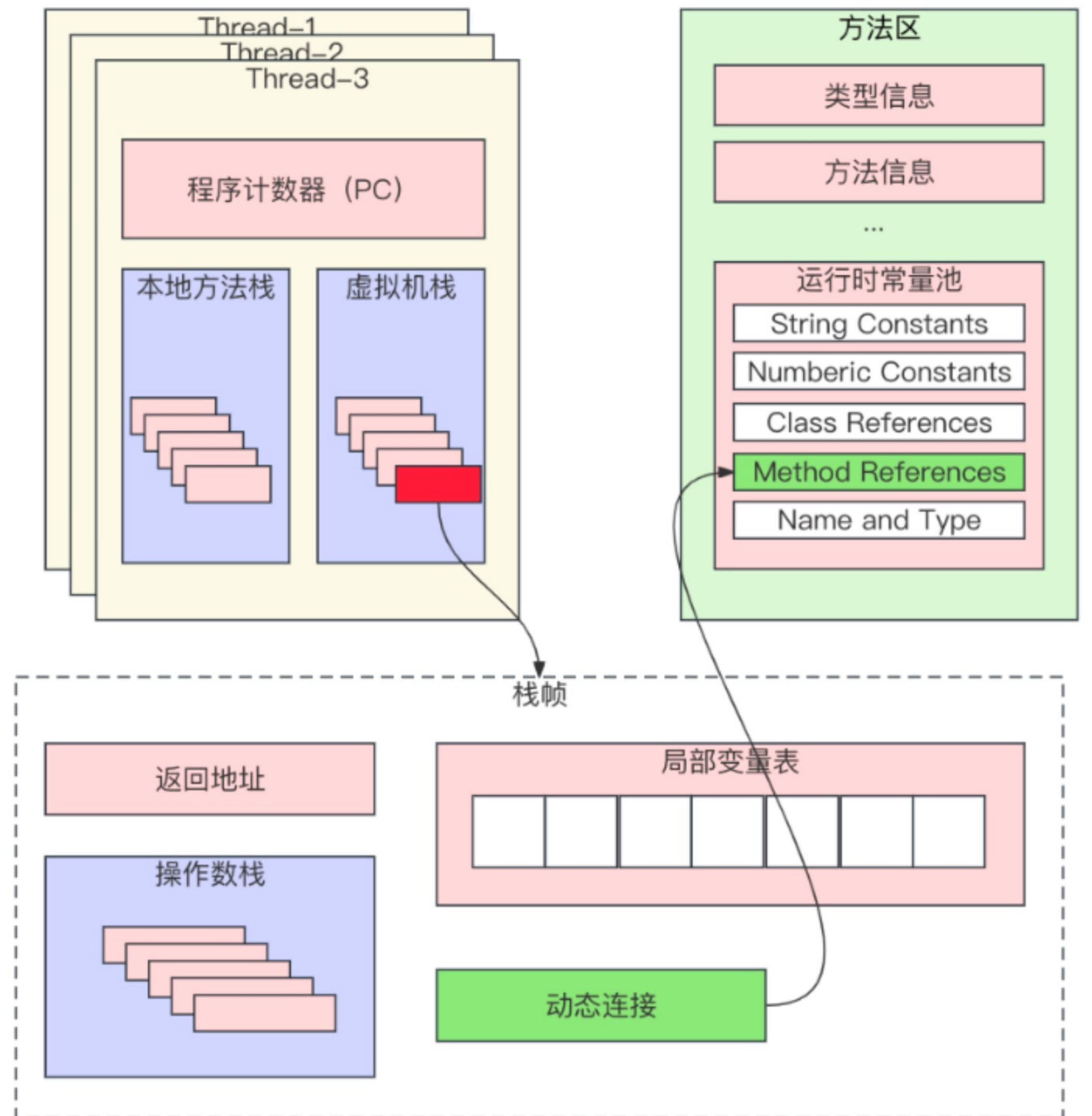


# 常量池、运行时常量池、字符串常量池

常量池：存放于字节码里，字节码文件的资源仓库。

运行时常量池：运行时期期间，JVM 会将字节码文件中的常量池加载到内存中，存放在运行时常量池中。

字符串常量池：字符串常量池，用于存放字符串常量的运行时的对象的引用。JDK 1.6 及之前字符串常量池位于方法区，JDK 1.7 之后字符串常量池被移至堆中。



# 运行时常量池

- 存放编译器生成的
  - 各种字面量
    - numeric literals
    - string literals
  - 符号引用
    - class references
    - field references
    - method references



The constant pool contains the following types:

|                                               |                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Integer                                       | A 4 byte int constant                                                                                                                                                                                                                                                                                                                                                                                              |
| Long                                          | An 8 byte long constant                                                                                                                                                                                                                                                                                                                                                                                            |
| Float                                         | A 4 byte float constant                                                                                                                                                                                                                                                                                                                                                                                            |
| Double                                        | A 8 byte double constant                                                                                                                                                                                                                                                                                                                                                                                           |
| String                                        | A String constant that points at another Utf8 entry in the constant pool which contains the actual bytes                                                                                                                                                                                                                                                                                                           |
| Utf8                                          | A stream of bytes representing a Utf8 encoded sequence of characters                                                                                                                                                                                                                                                                                                                                               |
| Class                                         | A Class constant that points at another Utf8 entry in the constant pool which contains the fully qualified class name in the internal JVM format (this is used by the <a href="#">dynamic linking process</a> )                                                                                                                                                                                                    |
| NameAndType                                   | A colon separated pair of values each pointing at other entries in the constant pool. The first value (before the colon) points at a Utf8 string entry that is the method or field name. The second value points at a Utf8 entry that represents the type, in the case of a field this is the fully qualified class name, in the case of a method this is a list of fully qualified class names one per parameter. |
| Fieldref,<br>Methodref,<br>InterfaceMethodref | A dot separated pair of values each pointing at other entries in the constant pool. The first value (before the dot) points at a Class entry. The second value points at a NameAndType entry.                                                                                                                                                                                                                      |

---

# 深入理解字符串常量池

<https://zhuanlan.zhihu.com/p/149507972>

<https://blog.csdn.net/HD243608836/article/details/126635377>

- String中==和equals()的区别
- 直接赋值和new String的区别

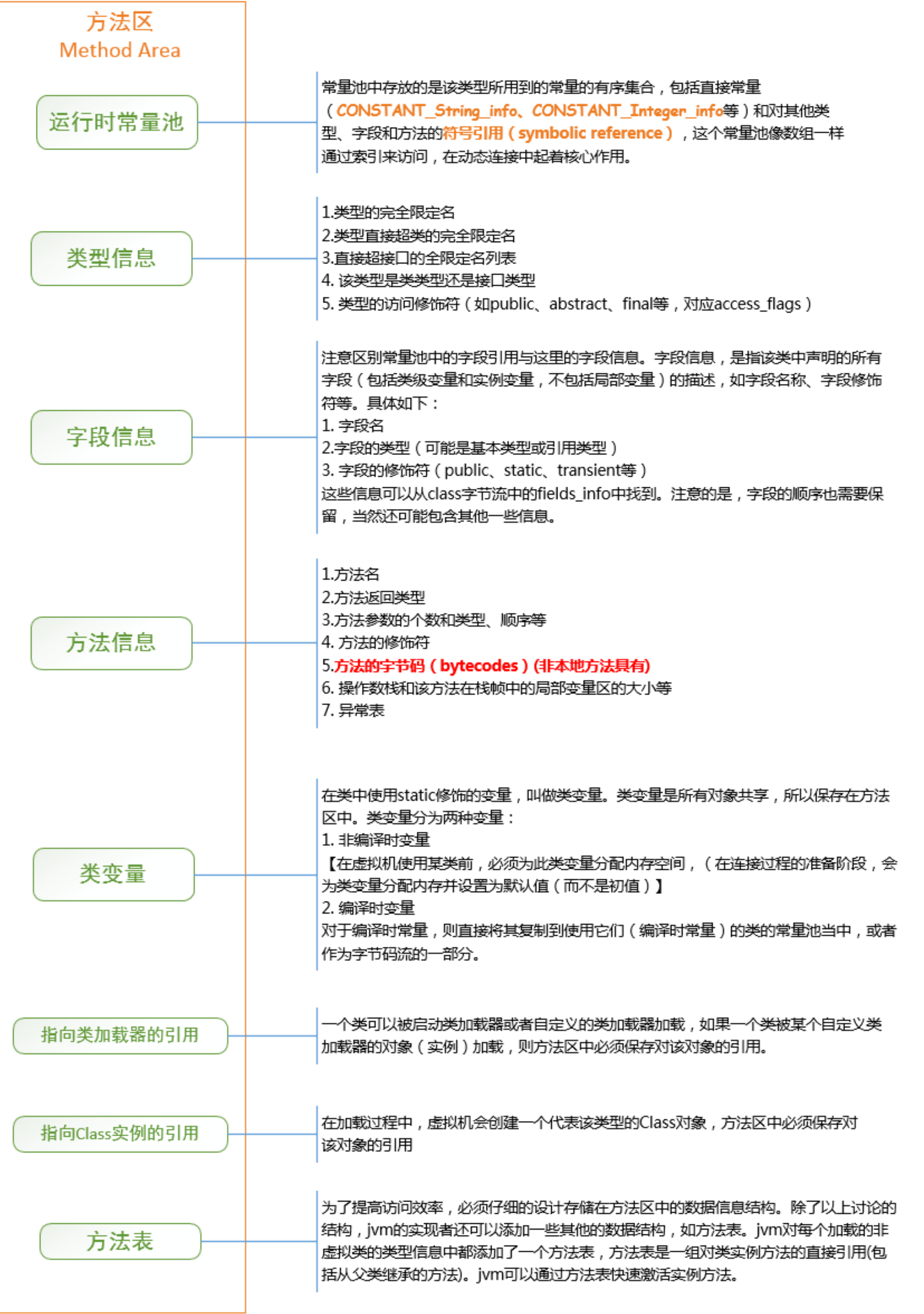
```
String s1 = new String("hello"); // 在堆上创建新对象
String s2 = s1.intern();           // 返回字符串常量池中的引用
String s3 = "hello" ;             // 直接使用字符串常量池中的引用
```

```
System.out.println(s1 == s2); // false
System.out.println(s2 == s3); // true
```

String.intern()

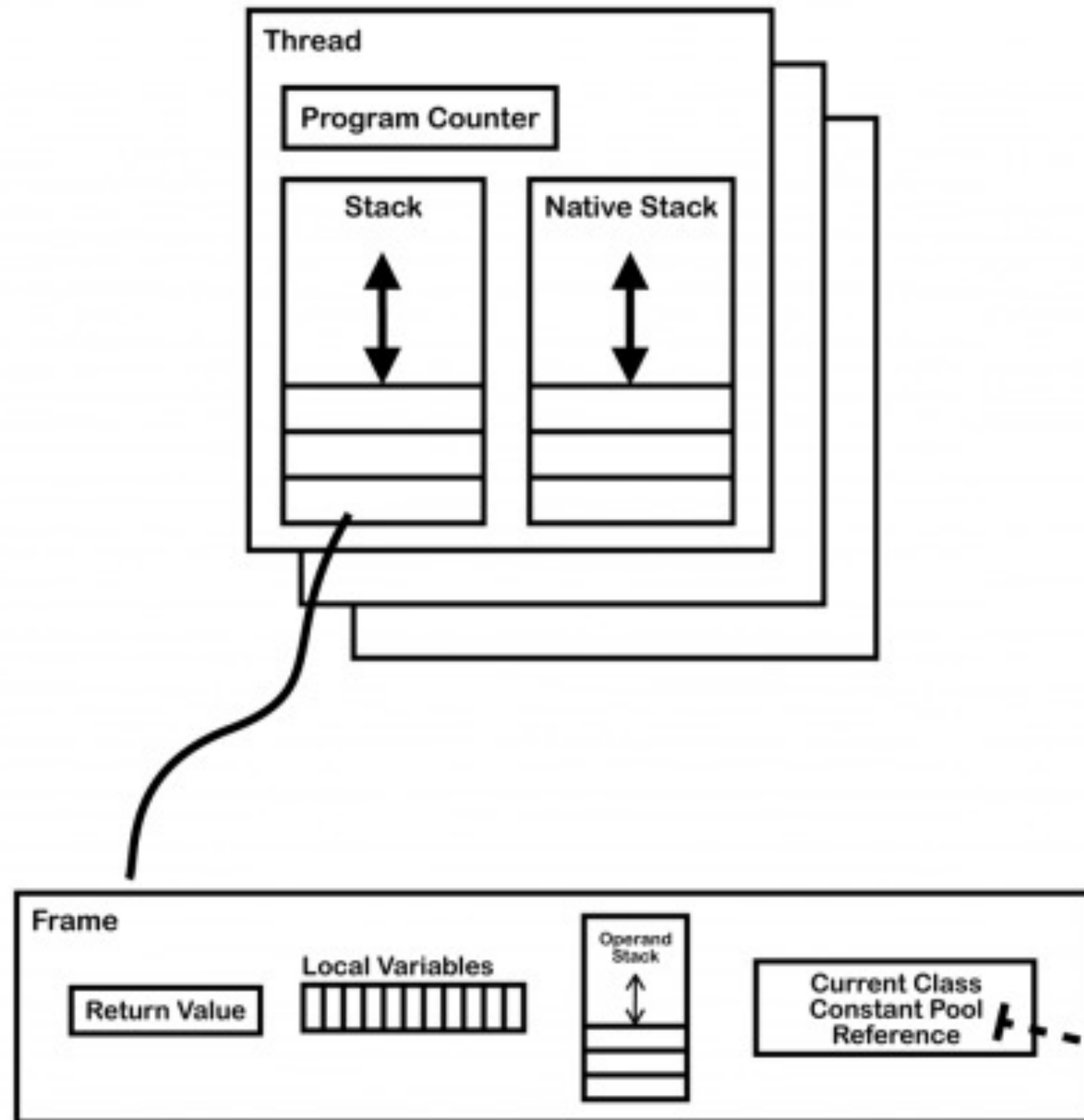
- 当调用 intern() 方法时，JVM 会检查字符串常量池中是否已经存在内容相同的字符串
- 如果存在，则直接返回常量池中的引用
- 如果不存在，则将该字符串对象添加到常量池，并返回其引用

Topic 4. 方法区是什么？方法区里有什么？

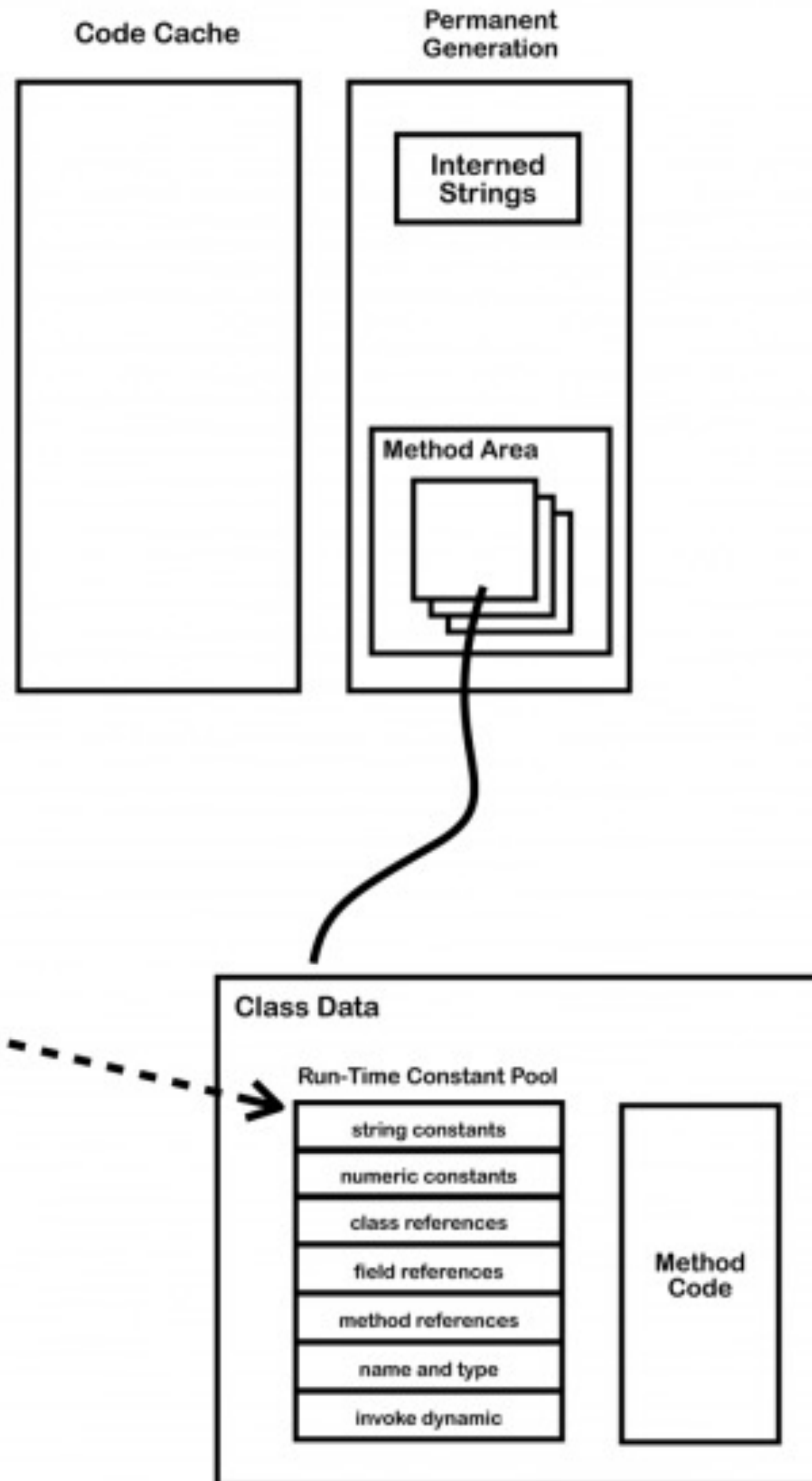




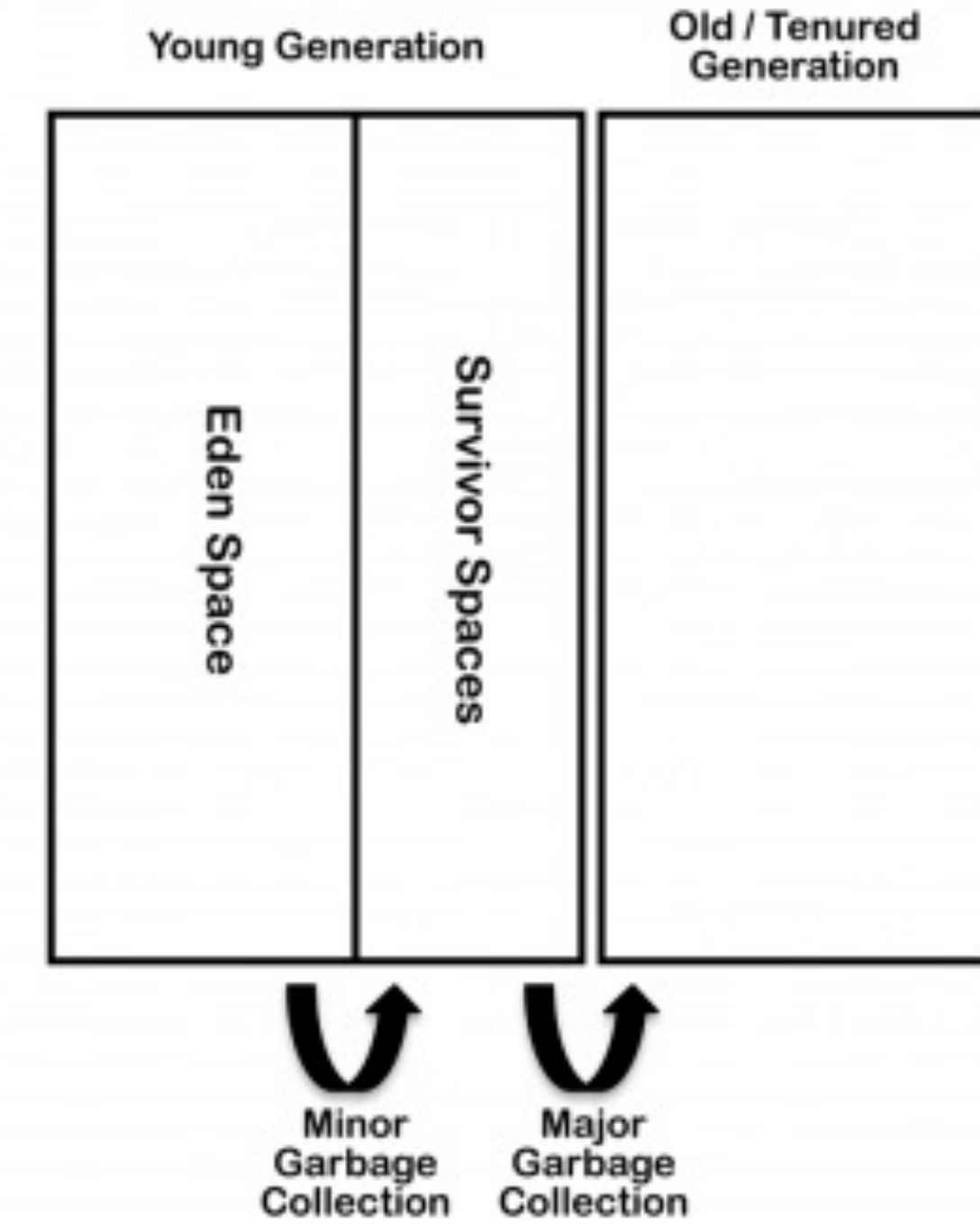
## Stack



## Non Heap



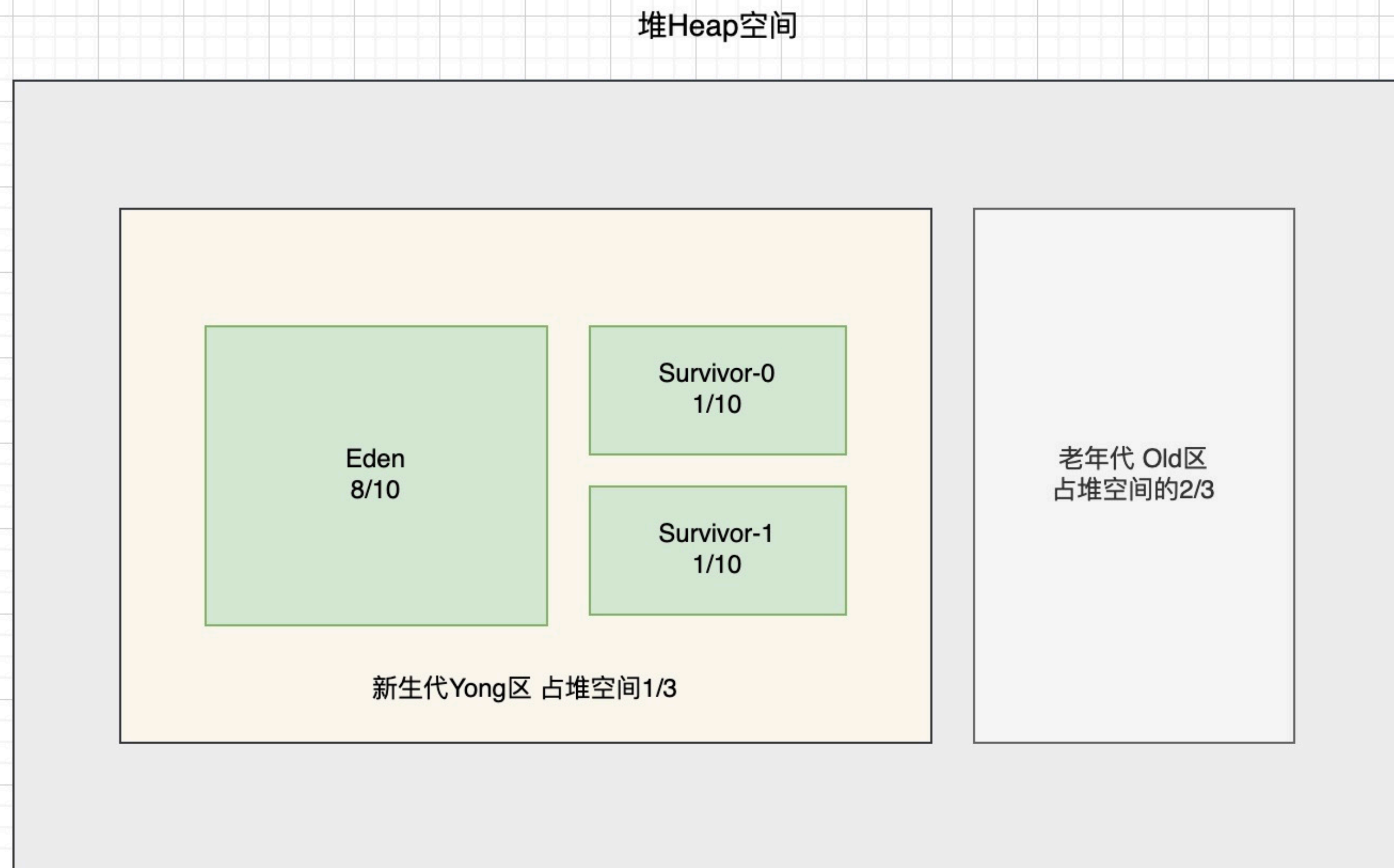
## Heap



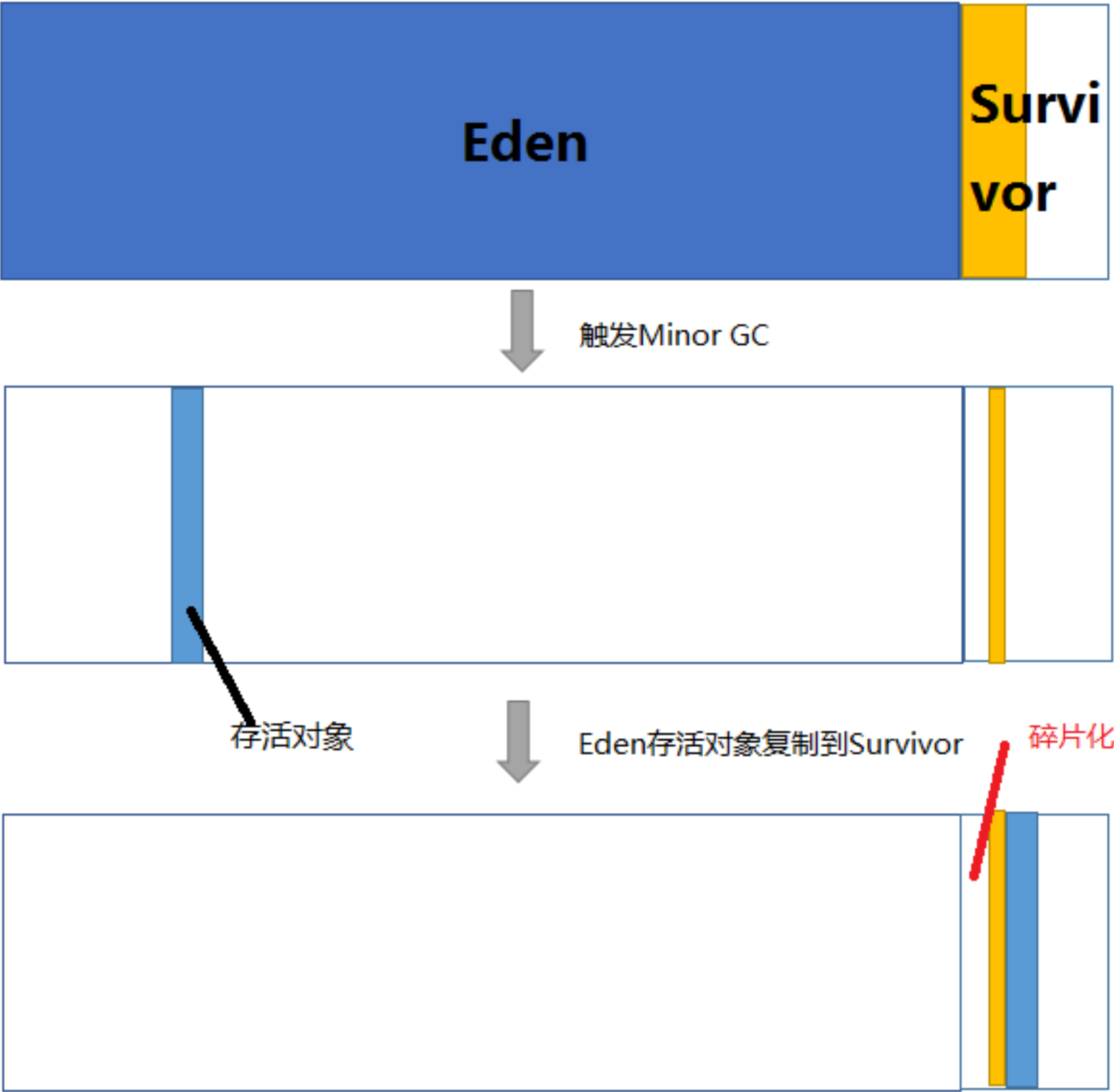


JVM将内存分为年轻代和老年代，其中年轻代又分为Eden区和两个Survivor区

- 年轻代
  - 用于存放新生的对象，其中Eden区是新对象的分配区域
  - 当Eden区满时，会触发Minor GC，将存活的对象移动到Survivor区，同时清空Eden区
  - Survivor区则用于存放经过一次Minor GC后存活的对象
- 老年代
  - 用于存放长时间存活的对象，当年轻代中的对象经过多次Minor GC后仍然存活，就会被移动到老年代
  - 当老年代满时，会触发Major GC



刚刚新建的对象在Eden中，经历一次Minor GC，Eden中的存活对象就会被移动到第一块From survivor区，Eden被清空；等Eden区再满了，就再触发一次Minor GC，Eden和From区中的存活对象又会被复制送入第二块To survivor区中（这个复制算法保证了To区中来自From和Eden两部分的存活对象占用连续的内存空间，避免了碎片化的发生）。From和Eden被清空，然后下一轮From survivor与To survivor交换角色，如此循环往复。



# 字节码指令集



# 基于栈的指令集架构 vs. 基于寄存器的指令集架构

- 寄存器 (Register) 是中央处理器 (CPU) 内用来暂存指令、数据和地址的存储器，也是 CPU 中读写最快的存储器。
- 从硬件层面来说，栈位于内存当中，而寄存器位于 CPU 当中。通常寄存器比栈更快。
- 基于栈的指令集的优势在于它可以在不同平台之间移植，而基于寄存器的指令集是与硬件架构紧密关联的，无法做到可移植。

访问时间

1 ns

2 ns

10 ns

10 ms

寄存器

高速缓存

主存

磁盘

容量

< 1 KB

4 MB

1 - 8GB

1 - 4TB

存储层次结构



06\_栈虚拟机与寄存器虚拟机

栈与寄存器

**栈 (Stack)** 是一种后进先出 (LIFO) 的数据结构，广泛应用于计算机科学中。栈的操作主要包括入栈 (Push) 和出栈 (Pop)，适用于需要临时存储和快速访问的场景

**寄存器 (Register)** 是中央处理器 (CPU) 内部的存储单元，用于暂存指令、数据和地址。寄存器是CPU中读写速度最快的存储器，通常用于执行算术和逻辑操作

从硬件层面来看，栈位于内存中，而寄存器位于CPU内部。这也是为什么基于寄存器的虚拟机通常比基于栈的虚拟机更快的原因

基于栈的虚拟机

**HotSpot VM** 是Java虚拟机 (JVM) 的代表，它采用了基于栈的架构。当Java程序运行时，HotSpot VM会加载编译后的字节码文件 ( `.class` 文件)，并通过解释器或JIT编译器将字节码指令解释或编译为机器码

在基于栈的虚拟机中，方法调用和执行过程中的数据（如局部变量和中间结果）存储在栈中。字节码指令操作这些数据，执行程序逻辑。每个方法调用都会创建一个栈帧 (Stack Frame)，栈帧包含局部变量表、操作数栈、动态链接和方法出口等信息

**操作数栈 (Operand Stack)** 是栈帧的一部分，用于保存方法执行时的中间结果、参数和返回值。操作数栈的内容是临时的，生命周期与方法的生命周期相同

基于寄存器的虚拟机

除了基于栈的虚拟机，还有基于寄存器的虚拟机实现，例如LuaVM。LuaVM负责执行Lua语言，Lua 5.0之前的版本使用基于栈的指令集，而5.0之后改为基于寄存器的指令集

基于寄存器的虚拟机通过减少数据移动次数和虚拟指令条数来提高执行效率

优缺点对比

基于栈的虚拟机

**优点：**可移植性更好，指令更短，实现简单

**缺点：**不能随机访问栈中的元素，完成相同功能所需的指令数较多，需要频繁的入栈和出栈操作

基于寄存器的虚拟机

**优点：**速度快，有利于程序运行速度的优化

**缺点：**操作数需要显式指定，指令较长

总结

基于栈的虚拟机和基于寄存器的虚拟机各有优缺点。基于栈的虚拟机在可移植性和实现简单性方面表现出色，但执行效率相对较低。而基于寄存器的虚拟机在执行效率上具有明显优势，但指令较长且需要显式指定操作数

# 指令分类

- 指令可以基本分为以下几类：
- 存储指令（例如：aload\_0, istore）
- 算术与逻辑指令（例如：ladd, fcmpl）
- 类型转换指令（例如：i2b, d2i）
- 对象创建与操作指令（例如：new, putfield）
- 堆栈操作指令（例如：swap, dup2）
- 控制转移指令（例如：ifeq, goto）
- 方法调用与返回指令（例如：invokespecial, areturn）

# 前后缀

- 前/后缀 操作数类型
- i 整数
- l 长整数
- s 短整数
- b 字节
- c 字符
- f 单精度浮点数
- d 双精度浮点数
- z 布尔值
- a 引用



# 加载和存储指令

- 加载和存储指令用于将数据从栈帧的局部变量表和操作数栈之间来回传输。
- 将一个局部变量加载到操作数栈的指令包括：iload、iload\_<n>、lload、lload\_<n>、fload、fload\_<n>、dload、dload\_<n>，aload、aload\_<n>。
- 将一个数值从操作数栈存储到局部变量表的指令：istore, istore\_<n>、lstore、lstore\_<n>、fstore、fstore\_<n>、dstore、dstore\_<n>、astore,astore\_<n>
- 将常量加载到操作数栈的指令：bipush、sipush、ldc、ldc\_w、ldc2\_w、aconst\_null、iconst\_ml、iconst\_<i>、lconst\_<l>、fconst\_<f>、dconst\_<d>
- 局部变量表的访问索引指令:wide
- 一部分以尖括号结尾的指令代表了一组指令，如iload\_<i>，代表了iload\_0,iload\_1等，这几组指令都是带有一个操作数的通用指令。



# 运算指令

- 算术指令用于对两个操作数栈上的值进行某种特定运算，并把结果重新存入到操作栈顶。
  - 1)加法指令:iadd,ladd,fadd,dadd
  - 2)减法指令:isub,lsub,fsub,dsub
  - 3)乘法指令:imul,lmul,fmul,dmul
  - 4)除法指令:idiv,ldiv,fdiv,ddiv
  - 5)求余指令:irem,lrem,frem,drem
  - 6)取反指令:ineg,lneg,fneg,dneg
  - 7)位移指令  
:ishl,ishr,iushr,lshl,lshr,lushr
  - 8)按位或指令:ior,lor
  - 9)按位与指令:iand,land
  - 10)按位异或指令:ixor,lxor
  - 11)局部变量自增指令:iinc
  - 12)比较指令  
:dcmplt,dcmlt,fcmlt,fcmlt,lcmplt
- Java虚拟机没有明确规定整型数据溢出的情况，但规定了处理整型数据时，只有除法和求余指令出现除数为0时会导致虚拟机抛出异常。
- Java虚拟机要求在浮点数运算的时候，所有结果否必须舍入到适当的精度，如果有两种可表示的形式与该值一样，会优先选择最低有效位为零的。称之为最接近数舍入模式。
- 浮点数向整数转换的时候，Java虚拟机使用IEEE 754标准中的向零舍入模式，这种模式舍入的结果会导致数字被截断，所有小数部分的有效字节会被丢掉。

# 类型转换指令

- 类型转换指令将两种Java虚拟机数值类型相互转换，这些操作一般用于实现用户代码的显式类型转换操作。
- JVM直接就支持宽化类型转换(小范围类型向大范围类型转换):
  - 1)int类型到long,float,double类型
  - 2)long类型到float,double类型
  - 3)float到double类型
- 但在处理窄化类型转换时，必须显式使用转换指令来完成，这些指令包括：i2b、i2c、i2s、l2i、f2i、f2l、d2i、d2l和 d2f。
- 将int 或 long 窄化为整型T的时候，仅仅简单的把除了低位的N个字节以外的内容丢弃，N是T的长度。这有可能导致转换结果与输入值有不同的正负号。
- 在将一个浮点值窄化为整数类型T（仅限于 int 和 long 类型），将遵循以下转换规则：
  - 1) 如果浮点值是NaN，转换结果就是int 或 long 类型的0
  - 2) 如果浮点值不是无穷大，浮点值使用IEEE 754 的向零舍入模式取整，获得整数v，如果v在T表示范围之内，那就过就是v
  - 3) 否则，根据v的符号，转换为T 所能表示的最大或者最小正数

# 对象创建于访问指令

- 虽然类实例和数组都是对象，Java虚拟机对类实例和数组的创建与操作使用了不同的字节码指令。
- 1)创建实例的指令:new
- 2)创建数组的指令:newarray,anewarray,multianewarray
- 3)访问字段指令:getfield,putfield,getstatic,putstatic
- 4)把数组元素加载到操作数栈指令:baload,caload,saload,iaload,laload,faload,daload,aaload
- 5)将操作数栈的数值存储到数组元素中执行  
:bastore,castore,castore,sastore,iastore,fastore,dastore,aastore
- 6)取数组长度指令:arraylength JVM支持方法级同步和方法内部一段指令序列同步，这两种都是通过moniter实现的。
- 7)检查实例类型指令:instanceof,checkcast



# 操作数栈管理指令

- 如同操作一个普通数据结构中的堆栈那样，Java 虚拟机提供了一些用于直接操作操作数栈的指令，包括：
  - 1) 将操作数栈的栈顶一个或两个元素出栈：pop、pop2
  - 2) 复制栈顶一个或两个数值并将复制值或双份的复制值重新压入栈顶：dup、dup2、dup\_x1、dup2\_x1、dup\_x2、dup2\_x2。
  - 3) 将栈最顶端的两个数值互换：swap



# 控制转移指令

- 让JVM有条件或无条件从指定指令而不是控制转移指令的下一条指令继续执行程序。控制转移指令包括：
- 条件分支
- 复合条件分支
- 无条件分支
- JVM中有专门的指令集处理int和reference类型的条件分支比较操作，为了可以无明显标示一个实体值是否是null,有专门的指令检测null 值。boolean类型和byte类型,char类型和short类型的条件分支比较操作，都使用int类型的比较指令完成，而long,float,double条件分支比较操作，由相应类型的比较运算指令，运算指令会返回一个整型值到操作数栈中，随后再执行int类型的条件比较操作完成整个分支跳转。各种类型的比较都最终会转化为int类型的比较操作

| 指令名          | 字节码  | 描述                       |
|--------------|------|--------------------------|
| ifeq         | 0x99 | 如果栈顶int型变量等于0，则跳转        |
| ifne         | 0x9A | 如果栈顶int型变量不等于0，则跳转       |
| iflt         | 0x9B | 如果栈顶int型变量小于0，则跳转        |
| ifge         | 0x9C | 如果栈顶int型变量大于等于0，则跳转      |
| ifgt         | 0x9D | 如果栈顶int型变量大于0，则跳转        |
| ifle         | 0x9E | 如果栈顶int型变量小于等于0，则跳转      |
| if_icmpeq    | 0x9F | 比较栈顶两个int型变量，如果相等则跳转     |
| if_icmpne    | 0xA0 | 比较栈顶两个int型变量，如果不相等则跳转    |
| if_icmplt    | 0xA1 | 比较栈顶两个int型变量，如果小于则跳转     |
| if_icmpge    | 0xA2 | 比较栈顶两个int型变量，如果大于等于则跳转   |
| if_icmpgt    | 0xA3 | 比较栈顶两个int型变量，如果大于则跳转     |
| if_icmple    | 0xA4 | 比较栈顶两个int型变量，如果小于等于则跳转   |
| if_acmpeq    | 0xA5 | 比较栈顶两个引用类型变量，如果相等则跳转     |
| if_acmpne    | 0xA6 | 比较栈顶两个引用类型变量，如果不相等则跳转    |
| goto         | 0xA7 | 无条件跳转                    |
| tableswitch  | 0xAA | switch条件跳转，case值紧凑的情况下使用 |
| lookupswitch | 0xAB | switch条件跳转，case值稀疏的情况下使用 |

# 方法调用和返回指令

- `invokevirtual`指令: 调用对象的实例方法, 根据对象的实际类型进行分派(虚拟机分派)。
- `invokeinterface`指令: 调用接口方法, 在运行时搜索一个实现这个接口方法的对象, 找出合适的方法进行调用。
- `invokespecial`: 调用需要特殊处理的实例方法, 包括实例初始化方法, 私有方法和父类方法
- `invokestatic`: 调用类方法(static)
- 方法返回指令是根据返回值的类型区分的, 包括 `ireturn`(返回值是 `boolean, byte, char, short` 和 `int`), `lreturn`, `freturn`, `dreturn` 和 `areturn`, 另外一个 `return` 供 `void` 方法, 实例初始化方法, 类和接口的类初始化方法使用。

# 异常处理指令

- 在Java程序中显式抛出异常的操作（throw语句）都由athrow 指令来实现，除了用throw 语句显示抛出异常情况外，Java虚拟机规范还规定了许多运行时异常会在其他Java虚拟机指令检测到异常状况时自动抛出。
- 在Java虚拟机中，处理异常不是由字节码指令来实现的，而是采用异常表来完成的。



# 同步指令

- 方法级的同步是隐式的，无需通过字节码指令来控制，它实现在方法调用和返回操作中。虚拟机从方法常量池中的方法标结构中的 ACC SYNCHRONIZED 标志区分是否是同步方法。方法调用时，调用指令会检查该标志是否被设置，若设置，执行线程持有moniter，然后执行方法，最后完成方法时释放moniter。
- 同步一段指令集序列，通常由synchronized块标示，JVM指令集中有monitorenter和monitorexit来支持synchronized语义。
- 结构化锁定是指方法调用期间每一个monitor退出都与前面monitor进入相匹配的情形。JVM通过以下两条规则来保证结构化锁成立(T代表一线程，M代表一个monitor)：
  - 1) T在方法执行时持有M的次数必须与T在方法完成时释放的M次数相等
  - 2) 任何时刻都不会出现T释放M的次数比T持有M的次数多的情况



虚拟机字节码执行

# 类的生命周期

- 加载
- 连接
  - 验证
  - 准备
  - 解析
- 初始化
- 使用
- 卸载

# 类加载

加载、验证、准备、解析、初始化、使用和卸载。

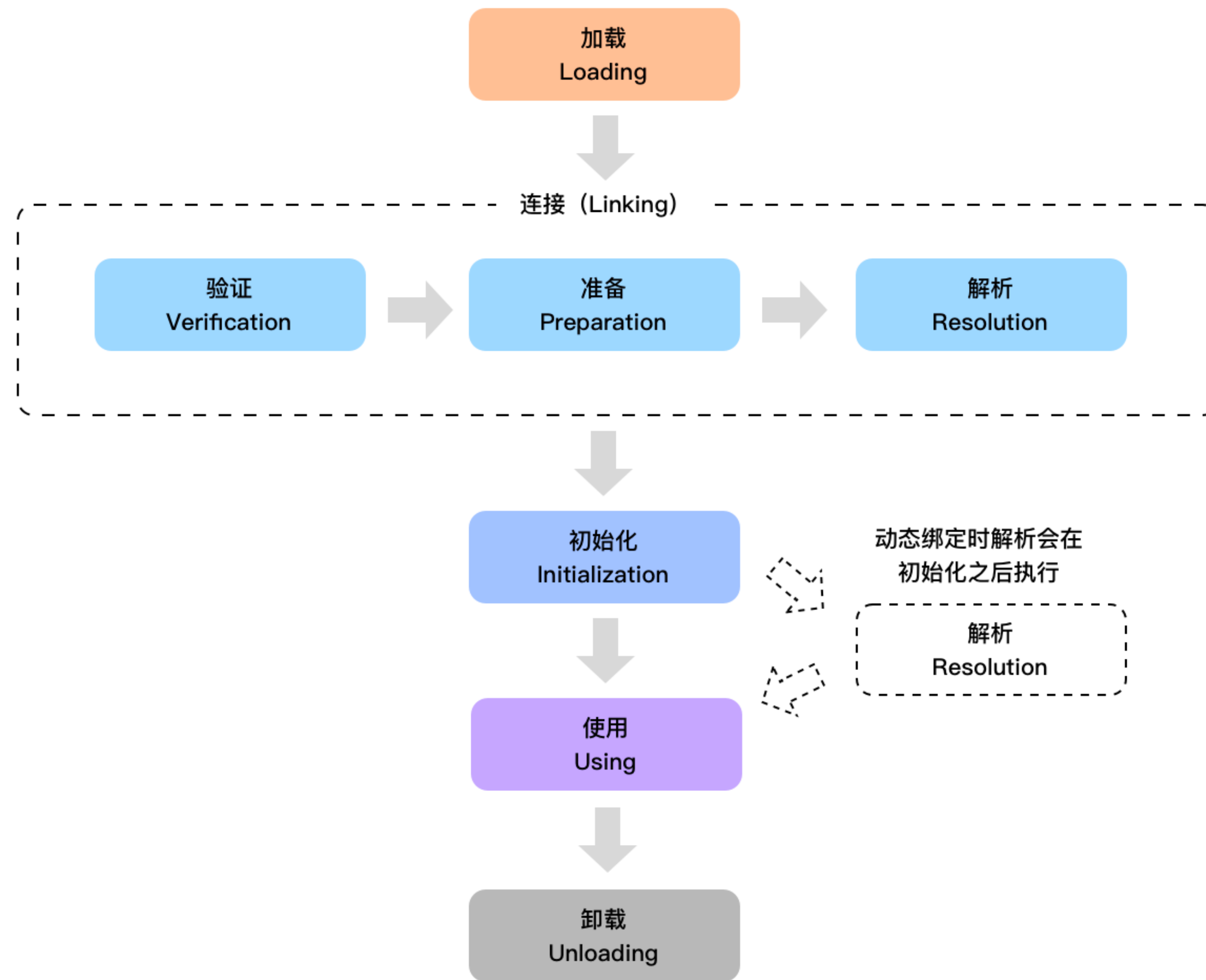
1) Loading（载入）：将字节码从不同的数据源转化为二进制字节流加载到内存中，并生成一个代表该类的 `java.lang.Class` 对象。

2) Verification（验证）：对二进制字节流进行校验

3) Preparation（准备）：对类变量（也称为静态变量，`static` 关键字修饰的）分配内存并初始化。

4) Resolution（解析）：该阶段将常量池中的符号引用转化为直接引用。

5) Initialization（初始化）：类变量将被赋值为代码期望赋的值。换句话说，执行类构造器方法的过程。



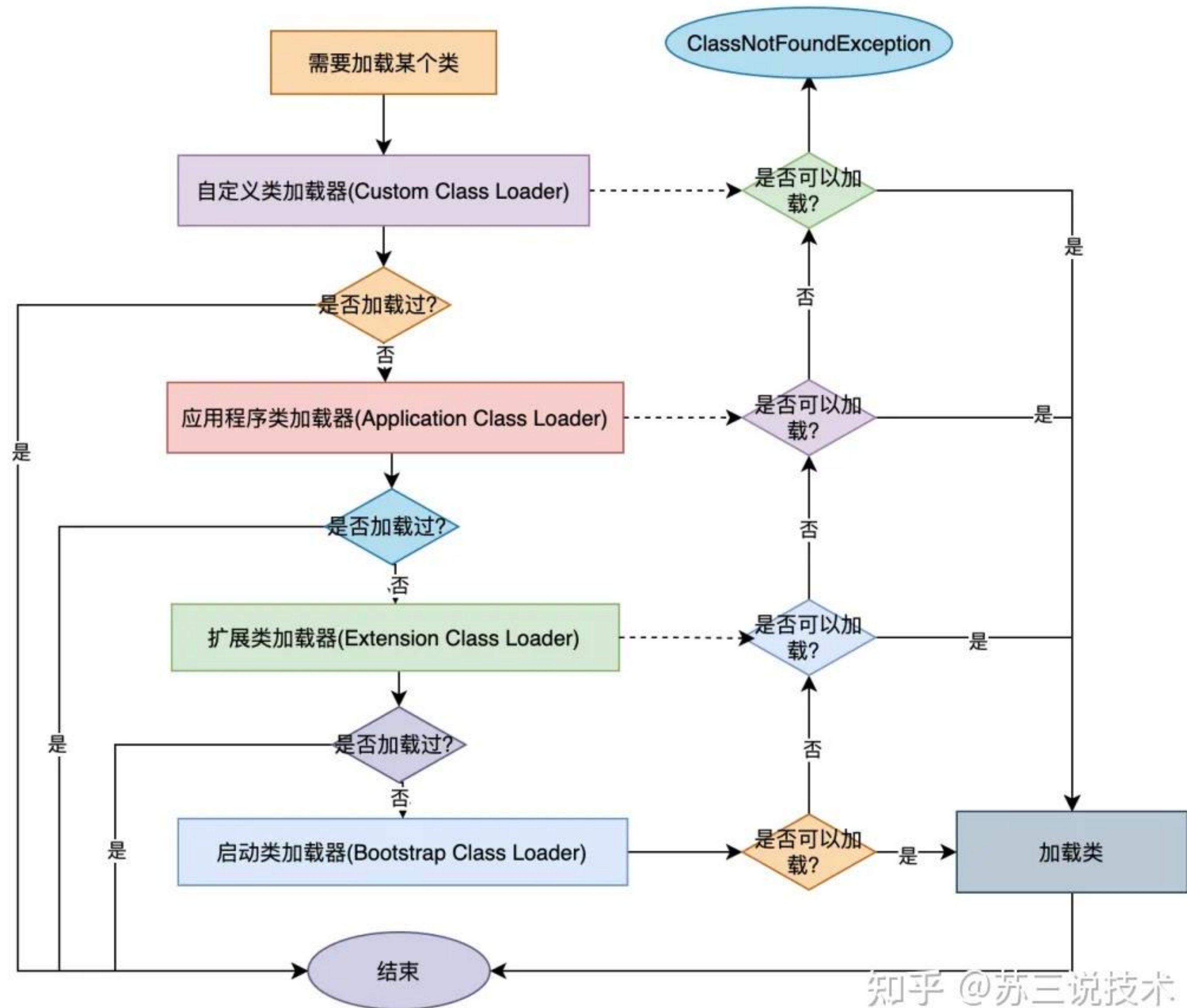
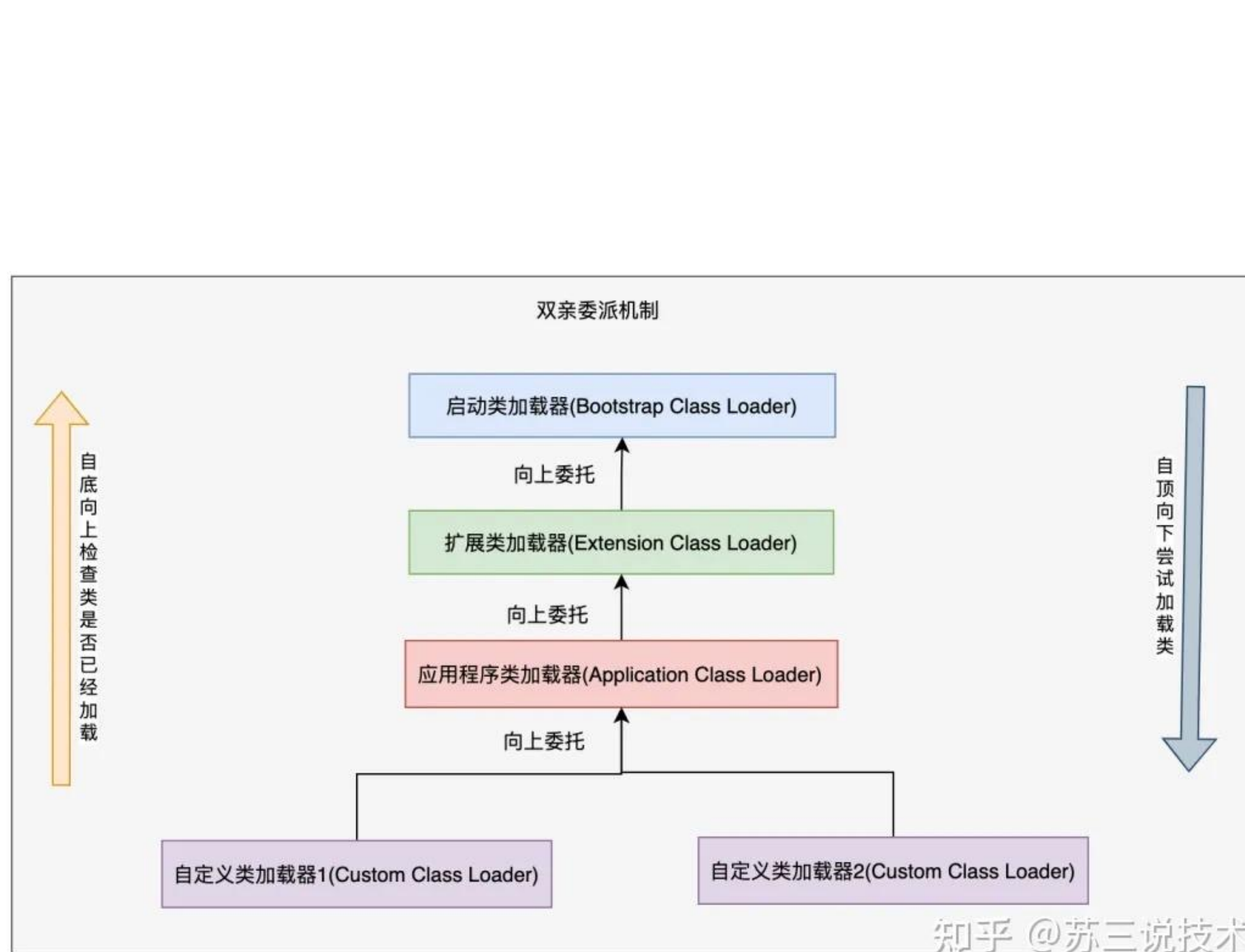


# 双亲委派机制

- 双亲委派机制的基本思想是：当一个类加载器试图加载某个类时，它会先委托给其父类加载器，如果父类加载器无法加载，再由当前类加载器自己进行加载。
  - 这种层层委派的方式有助于保障类的唯一性，避免类的重复加载，并提高系统的安全性和稳定性。
1. 启动类加载器（Bootstrap Class Loader）：负责加载 %JAVA\_HOME%/jre/lib 目录下的核心Java类库，比如：rt.jar、charsets.jar等。它是最顶层的类加载器，通常由C++编写。
  2. 扩展类加载器（Extension Class Loader）：负责加载Java的扩展库，一般位于 <JAVA\_HOME>/lib/ext目录下。
  3. 应用程序类加载器（Application Class Loader）：也称为系统类加载器，负责加载用户类路径（ClassPath）下的应用程序类。
  4. 自定义类加载器：用户没法控制，加载自己想要的一些类。



# 双亲委派机制



```
int foo(int a, int b, int c) {  
    return a + b * c;  
}  
//...  
foo(2, 3, 4);
```

输入：语言A的源代码

(由编译器编译)

```
mov    eax, dword ptr [esp + 8]  
imul   eax, dword ptr [esp + 0C]  
add    eax, dword ptr [esp + 4]  
ret  
;...  
push   4  
push   3  
push   2  
call   foo  
add    esp, 0C
```

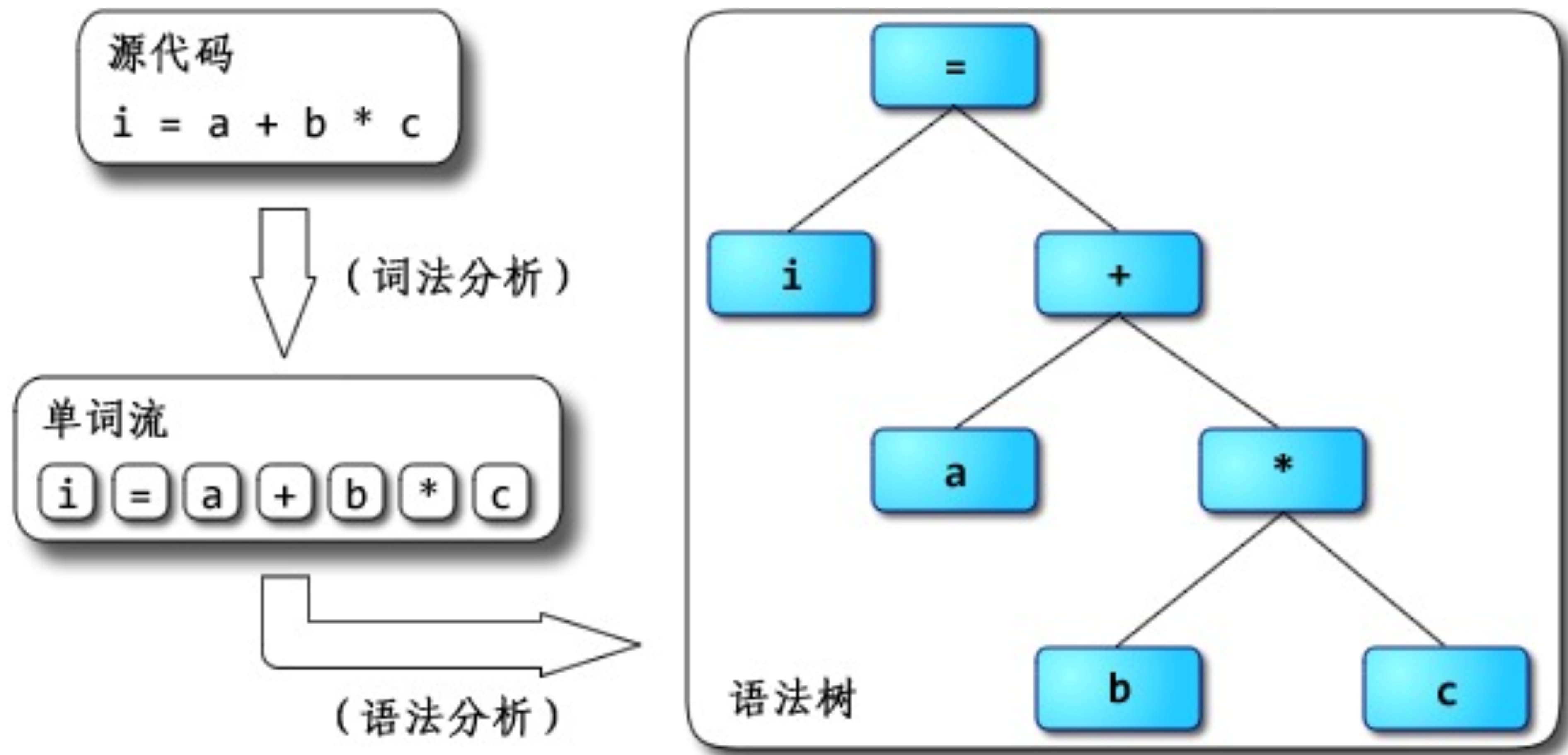
编译输出：与源程序等价的语言B源码

(由解释器解释执行)

14

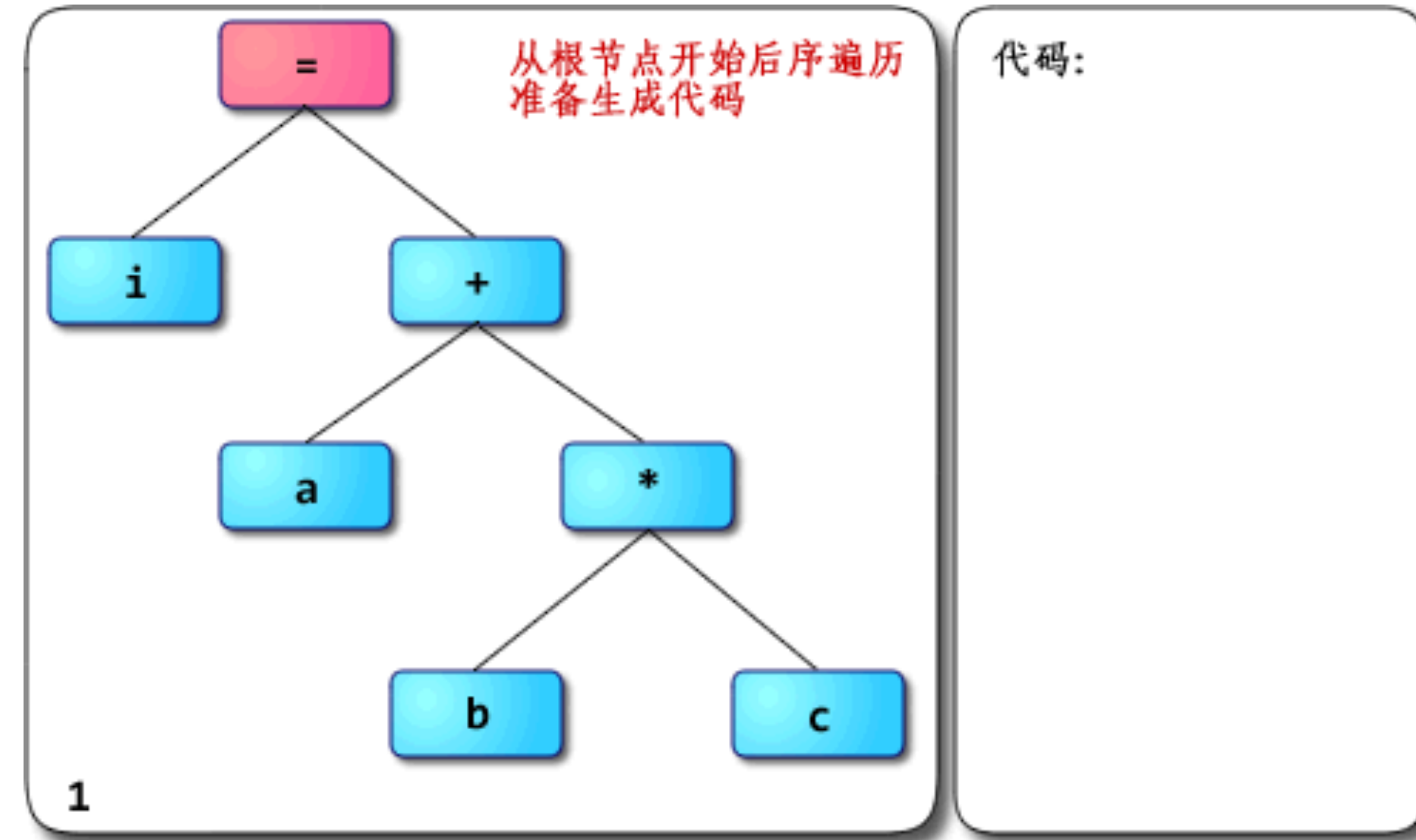
解释输出：程序运行结果

# 编译与解释



# 广义解析器





# 生成字节码



# 示例代码

- `public class Demo {`
- `public static void foo() {`
- `int a = 1;`
- `int b = 2;`
- `int c = (a + b) * 5;`
- `}`
- `}`

# Bytecode

- 0: iconst\_1
- 1: istore\_0
- 2: iconst\_2
- 3: istore\_1
- 4: iload\_0
- 5: iload\_1
- 6: iadd
- 7: iconst\_5
- 8: imul
- 9: istore\_2
- 10: return



# 栈架构

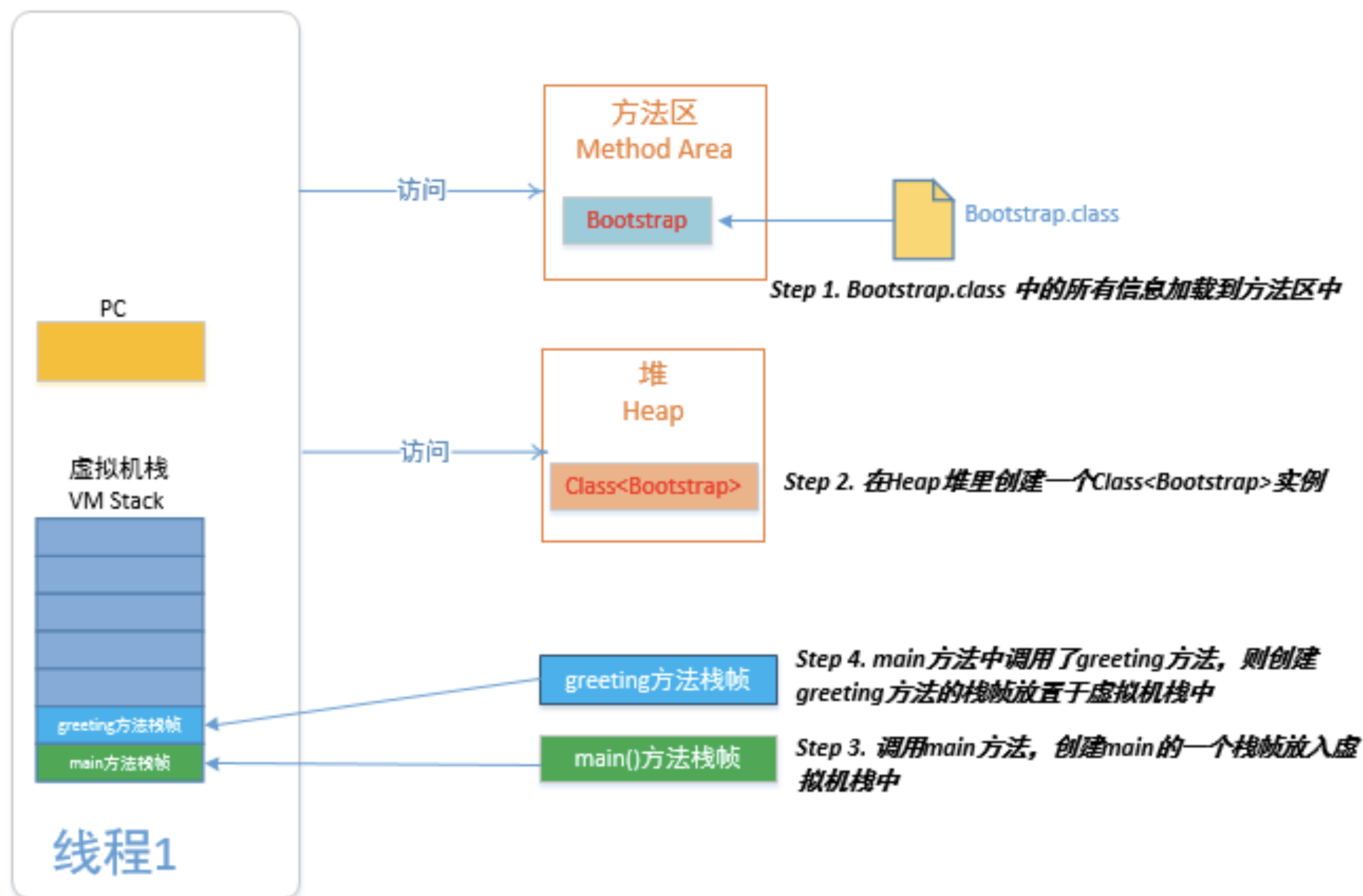


# Sample Code

```
1. package org.louis.jvm.codeset;
2. /**
3.  * JVM 原理简单用例
4.  * @author louis
5.  *
6.  */
7. public class Bootstrap {
8.
9.     public static void main(String[] args) {
10.         String name = "Louis";
11.         greeting(name);
12.     }
13.
14.     public static void greeting(String
        name)
15.     {
16.         System.out.println("Hello," + name);
17.     }
18.
19. }
```



## 运行时数据区



```
public static void main(String[] args) {  
    String name = "Louis";  
    greeting(name);  
}
```

编译后

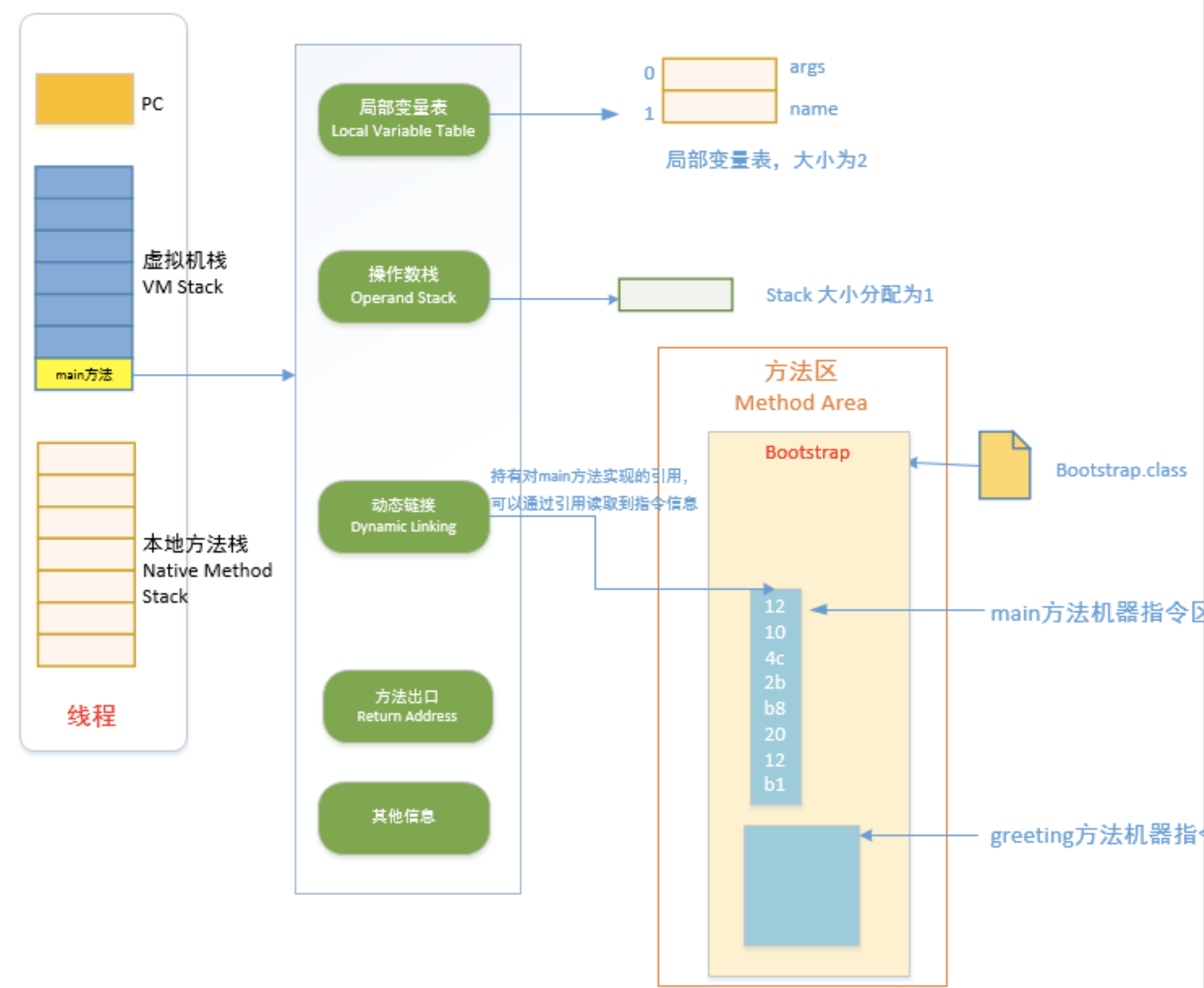
入参数量: 1  
局部变量数量: 2  
操作数栈大小: 1  
机器指令: 12 10 4c 2b b8 20 12 b1

```
public static void greeting(String name)  
{  
    System.out.println("Hello,"+name);  
}
```

编译后

入参数量: 1  
局部变量数量: 1  
操作数栈大小: 4  
机器指令: b2 20 1a bb 20 20 59 12 22 b7 20 24 2a b6 20 26 b6

main方法的栈帧信息 Stack Frame



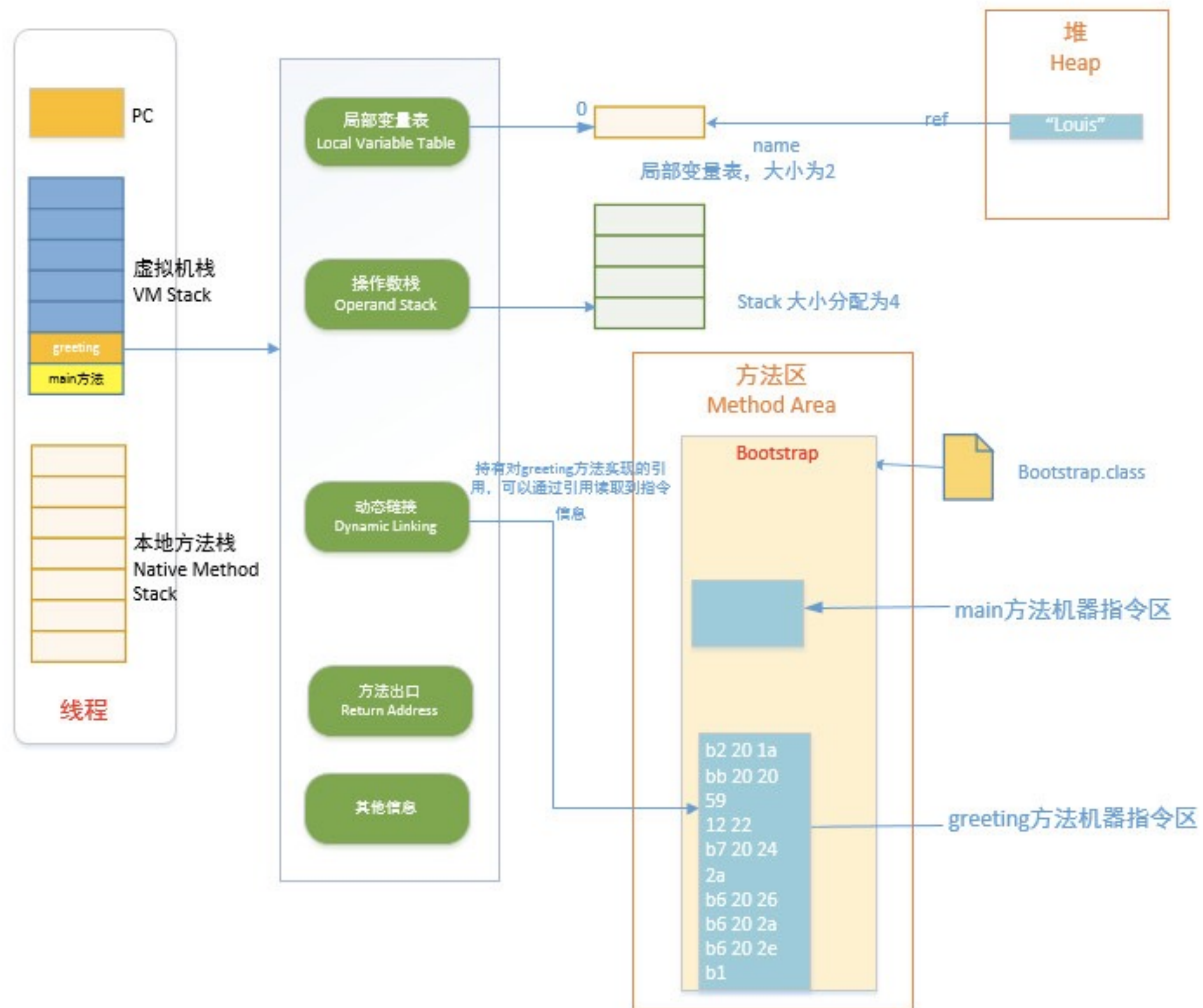


# main()

```
public static void main(java.lang.String[]);
  descriptor: ([Ljava/lang/String;)V
  flags: ACC_PUBLIC, ACC_STATIC
  Code:
    stack=1, locals=2, args_size=1
       0: ldc          #2             // String Louis
       2: astore_1
       3: aload_1
       4: invokestatic #3             // Method greeting:(Ljava/lang/String;)V
       7: return
 LineNumberTable:
    line 10: 0
    line 11: 3
    line 12: 7
```



## greeting方法的栈帧信息 Stack Frame



# greeting()

```
public static void greeting(java.lang.String);
  descriptor: (Ljava/lang/String;)V
  flags: ACC_PUBLIC, ACC_STATIC
  Code:
    stack=3, locals=1, args_size=1
       0: getstatic      #4          // Field java/lang/System.out:Ljava/io/PrintStream;
       3: new            #5          // class java/lang/StringBuilder
       6: dup
       7: invokespecial  #6          // Method java/lang/StringBuilder."<init>":()V
      10: ldc            #7          // String Hello,
      12: invokevirtual #8          // Method
java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
      15: aload_0
      16: invokevirtual #8          // Method
java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
      19: invokevirtual #9          // Method
java/lang/StringBuilder.toString:()Ljava/lang/String;
      22: invokevirtual #10         // Method java/io/PrintStream.println:(Ljava/lang/String;)V
      25: return
 LineNumberTable:
    line 16: 0
    line 17: 25
```

机器指令是用来告诉机器做什么的指令。操作码是鉴别是什么指令，而操作数相当于执行相应的指令所需要的参数。



greeting方法机器指令案例

| 机器指令     | 操作码 | 操作数1 | 操作数2 | 操作数3 | 助记符及解释            |
|----------|-----|------|------|------|-------------------|
| b2 20 1a | b2  | 20   | 1a   |      | getstatic #26     |
| bb 20 20 | bb  | 20   | 20   |      | new #32           |
| 59       | 59  |      |      |      | dup               |
| 12 22    | 12  | 22   |      |      | ldc #34           |
| b7 20 24 | b7  | 20   | 24   |      | invokespecial #36 |
| 2a       | 2a  |      |      |      | aload_0           |
| b6 20 26 | b6  | 20   | 26   |      | invokevirtual #38 |
| b6 20 2a | b6  | 20   | 2a   |      | invokevirtual #42 |
| b6 20 2e | b6  | 20   | 2e   |      | invokevirtual #46 |
| b1       | b1  |      |      |      | return            |
|          |     |      |      |      |                   |



# Java指令与字节码

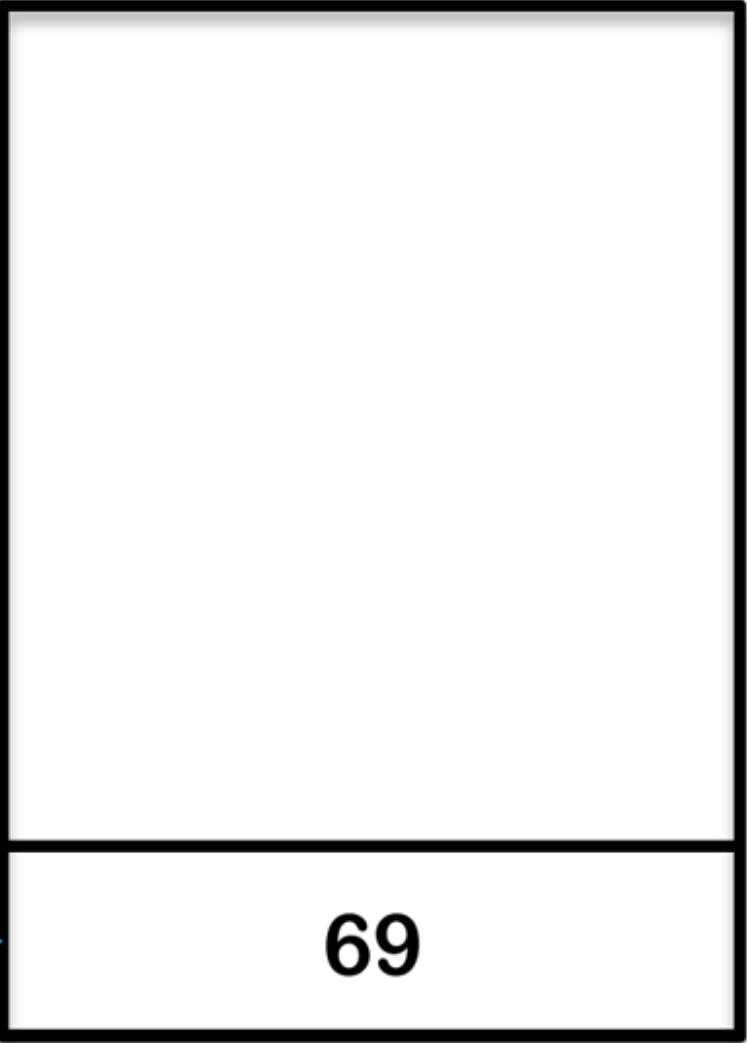


变量

```
int i = 69;
```

```
0: bipush 69
```

Operand Stack

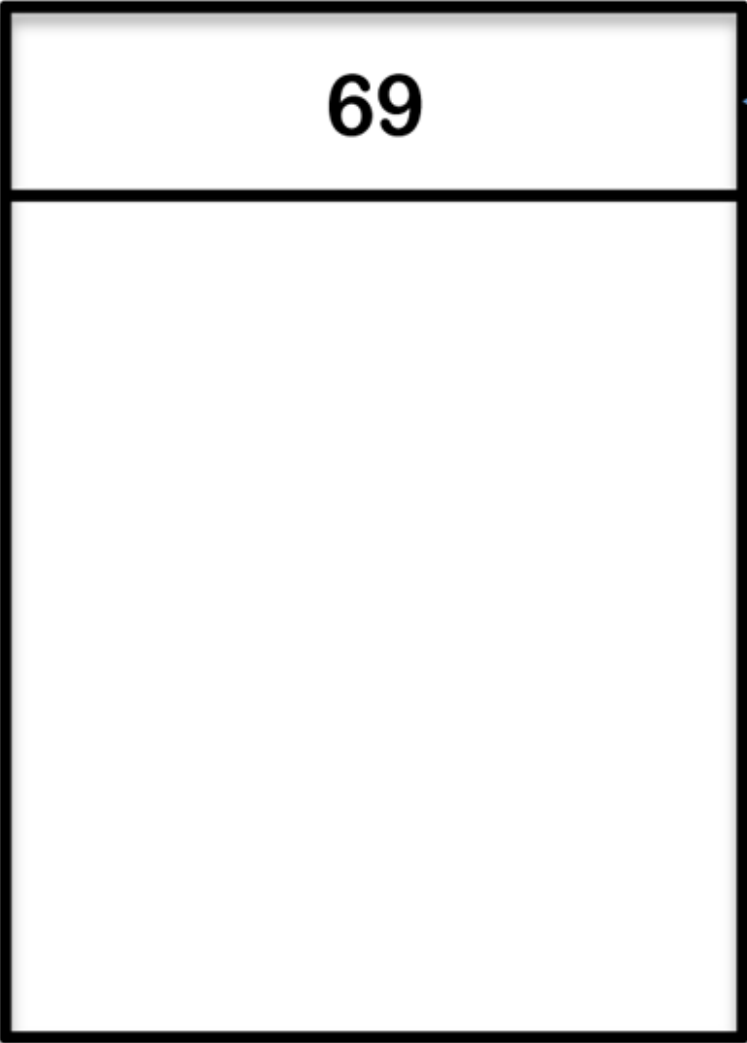


push

69

```
2: istore_0
```

Local Variables



Operand Stack



pop

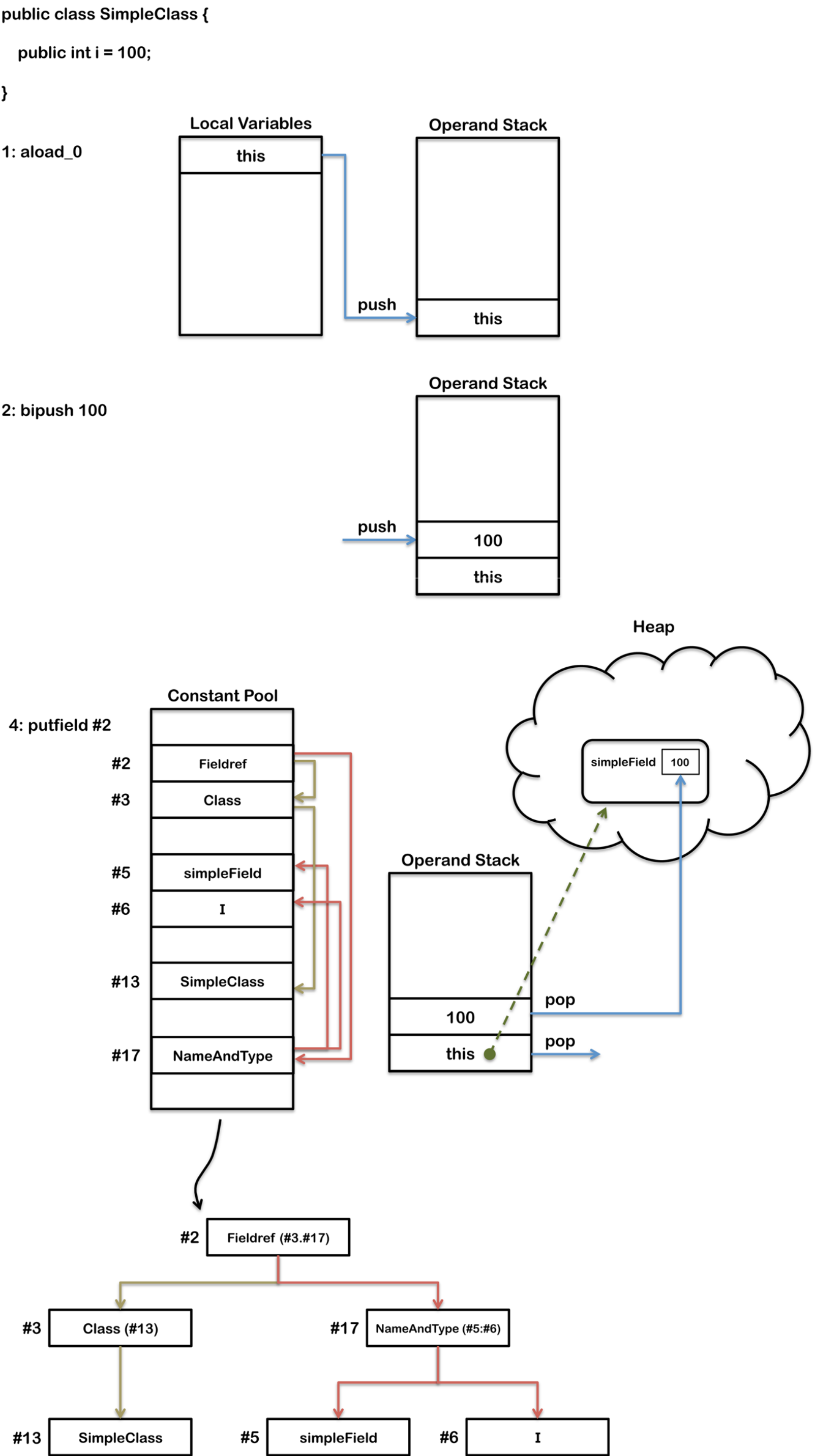
69

局部变量

```
public class SimpleClass{
    public int i = 100;
}
```

```
1: aload_0
2: bipush 100
4: putfield #2
```

成员变量



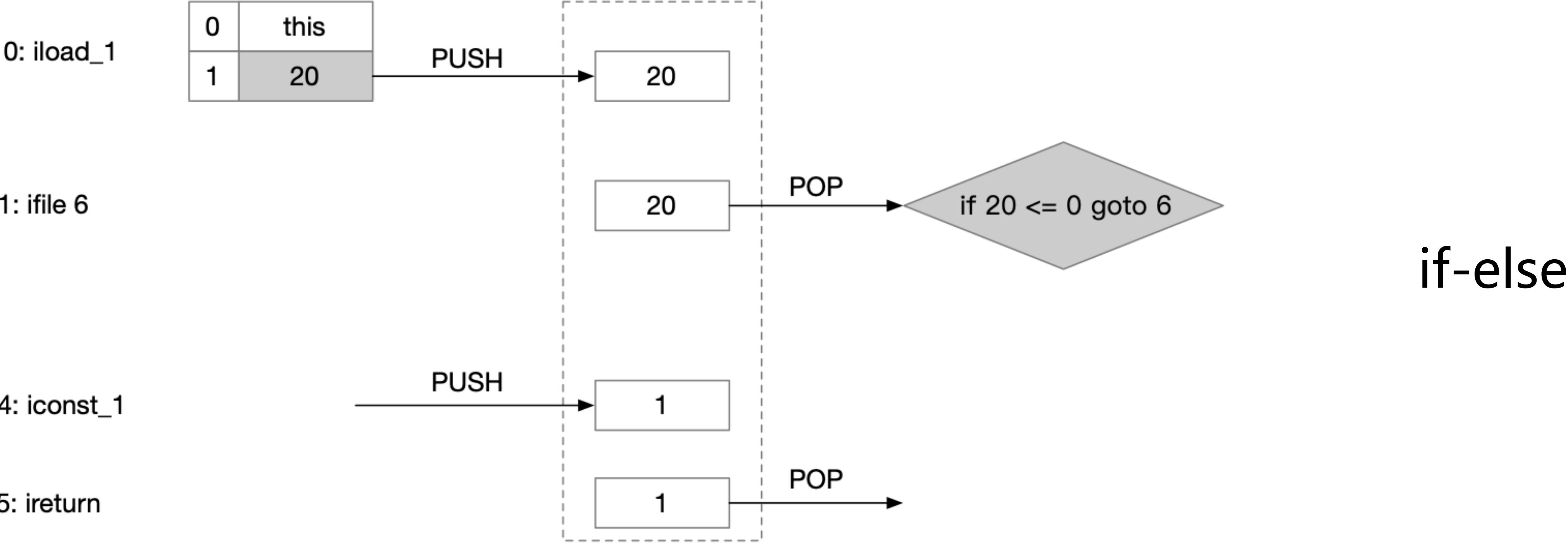


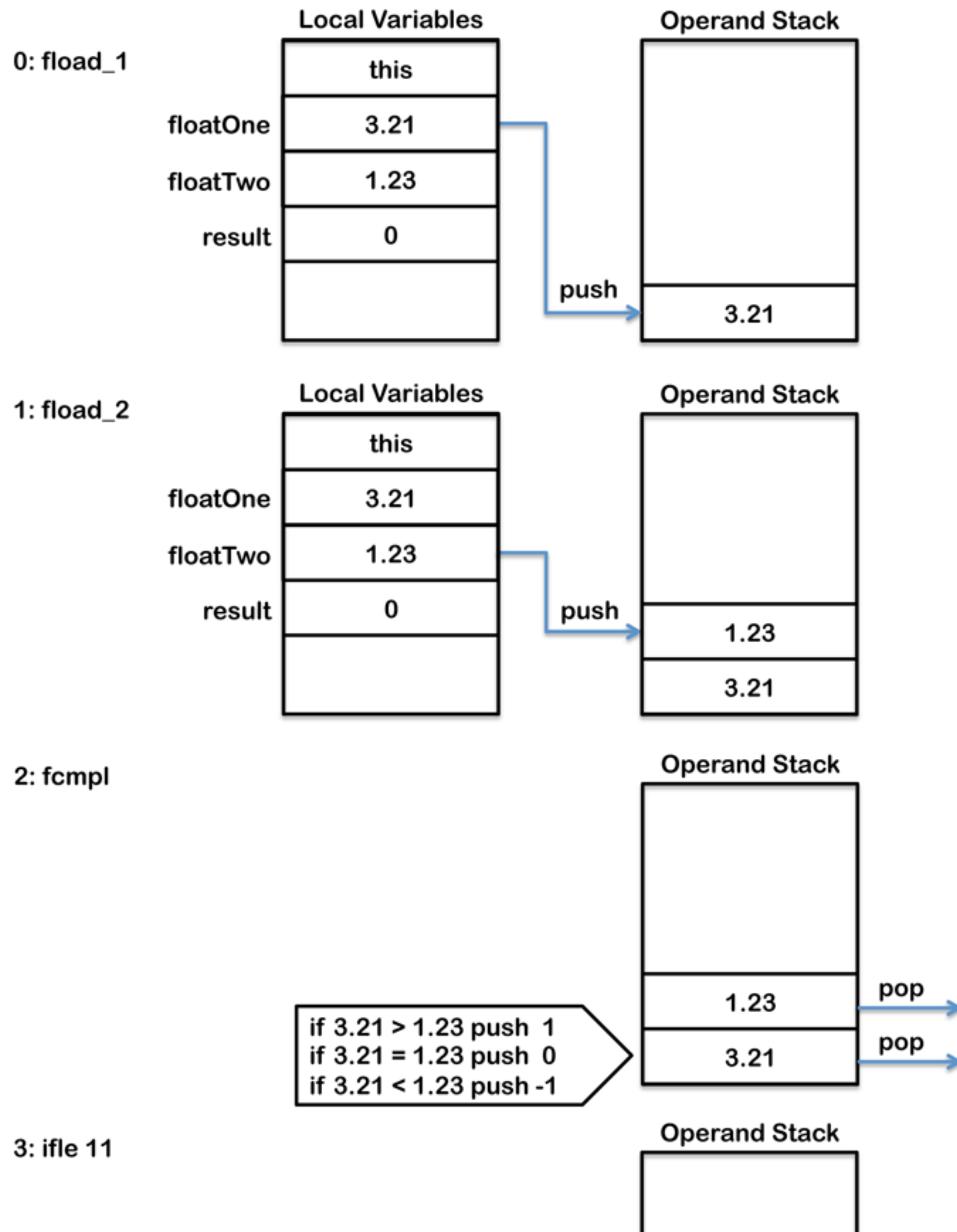
# 条件语句



```
public int isPositive(int n){
    if (n > 0){
        return 1;
    }else {
        return 0;
    }
}
```

0: iload\_1 // 将局部变量表中下标为1的int变量加载到操作数栈上  
1: ifle 6 // 将操作数栈顶元素出栈与0比较, 如果小于等于0则跳转到字节码第六行  
4: iconst\_1 // 4~5行对应代码" return 1 ", 这里先将常量1压到栈顶  
5: ireturn // 将栈顶整数1出栈并返回  
6: iconst\_0 // 将常量0压入栈顶  
7: ireturn // 将栈顶整数0出栈返回





```
public int greaterThen(float floatOne, float floatTwo){
    int result;
    if (floatOne > floatTwo) {
        result = 1;
    } else {
        result = 2;
    }
    return result;
}
```

```
0 fload_1
1 fload_2
2 fcmpl
3 ifle 11 (+8)
6 iconst_1
7 istore_3
8 goto 13 (+5)
11 iconst_2
12 istore_3
13 iload_3
14 ireturn
```

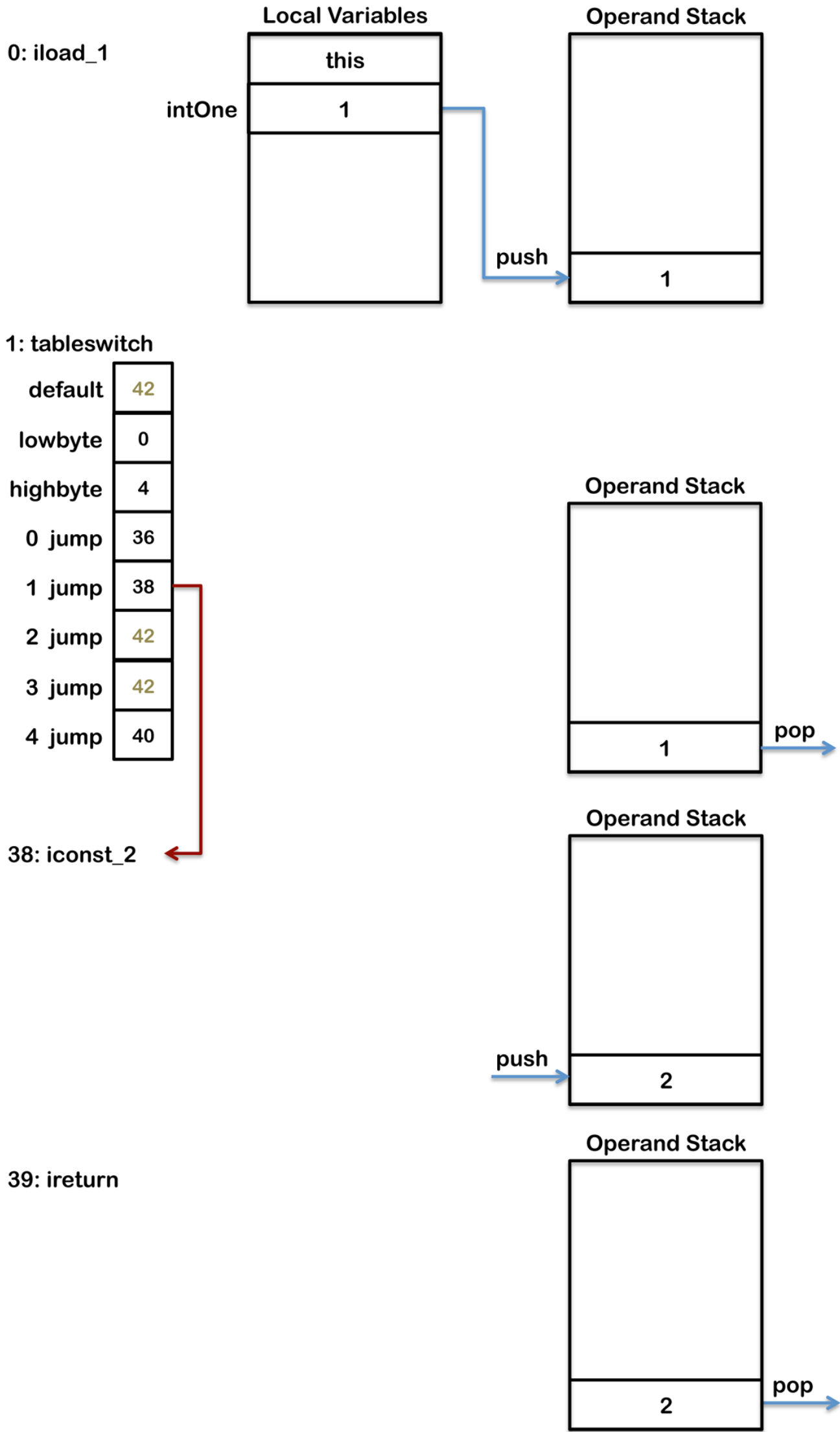
if else

```
public int simpleSwitch (int intOne) {
    switch (intOne) {
        case 0:
            return 3;
        case 1:
            return 2;
        case 4:
            return 1;
        default:
            return -1;
    }
}
```

```
0 iload_1
1 tableswitch 0 to 4
0: 36 (+35)
1: 38 (+37)
2: 42 (+41)
3: 42 (+41)
4: 40 (+39)
default: 42 (+41)
36 iconst_3
37 ireturn
38 iconst_2
39 ireturn
40 iconst_1
41 ireturn
42 iconst_m1
43 ireturn
```

```
public int simpleSwitch(int intOne) {
    switch (intOne) {
        case 0:
            return 3;
        case 1:
            return 2;
        case 4:
            return 1;
        default:
            return -1;
    }
}

simpleSwitch(1);
```



# Switch



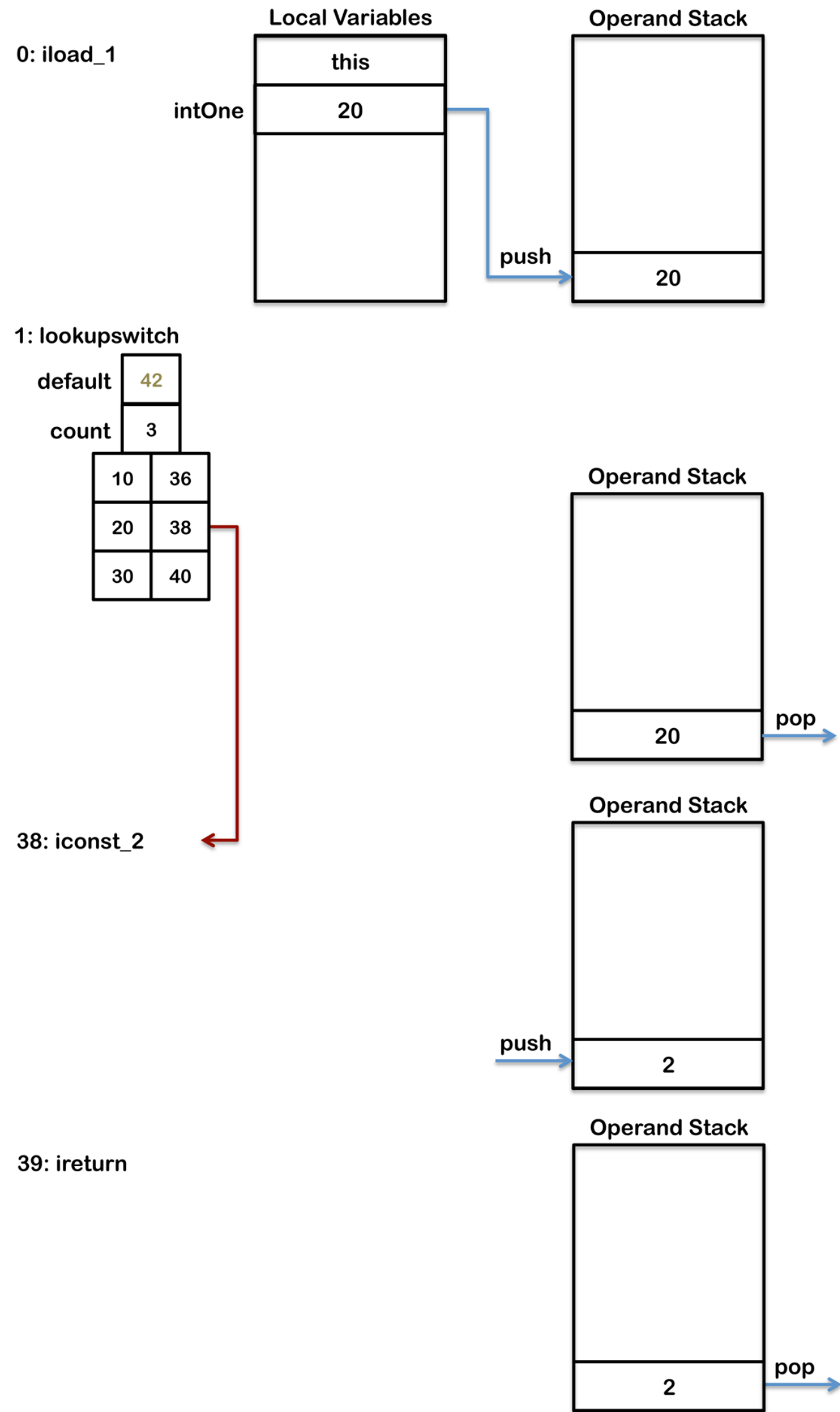
```
public int simpleSwitch (int intOne) {
    switch (intOne) {
        case 10:
            return 1;
        case 20:
            return 2;
        case 30:
            return 3;
        default:
            return -1;
    }
}
```

```
0 iload_1
1 lookupswitch 3
10: 36 (+35)
20: 38 (+37)
30: 40 (+39)
default: 42 (+41)
36 iconst_1
37 ireturn
38 iconst_2
39 ireturn
40 iconst_3
41 ireturn
42 iconst_m1
43 ireturn
```

# Switch

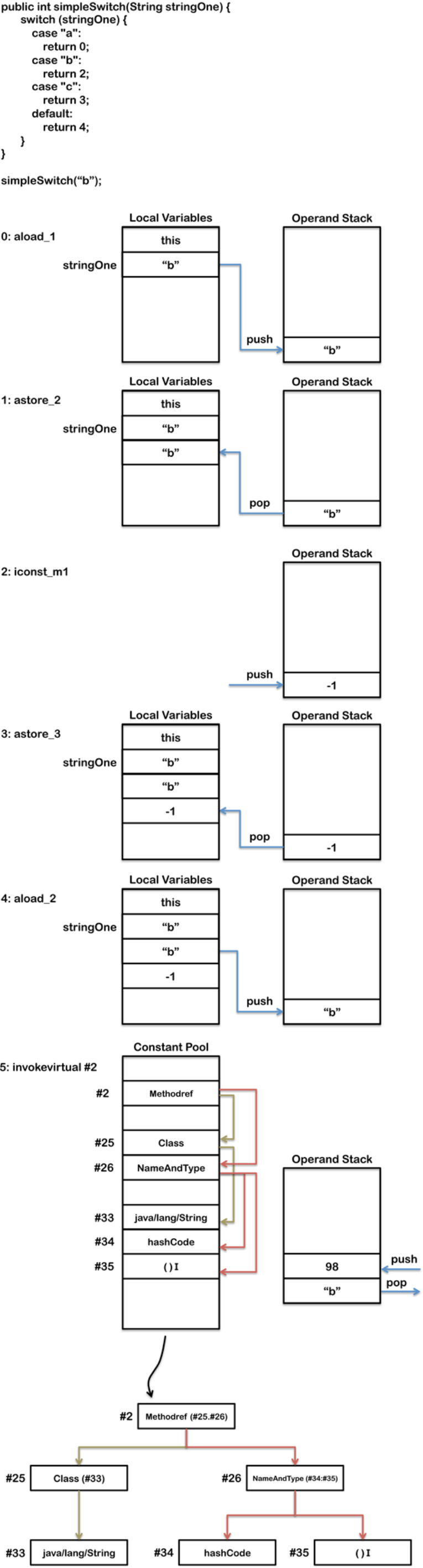
```
public int simpleSwitch(int intOne) {
    switch (intOne) {
        case 10:
            return 1;
        case 20:
            return 2;
        case 30:
            return 3;
        default:
            return -1;
    }
}
```

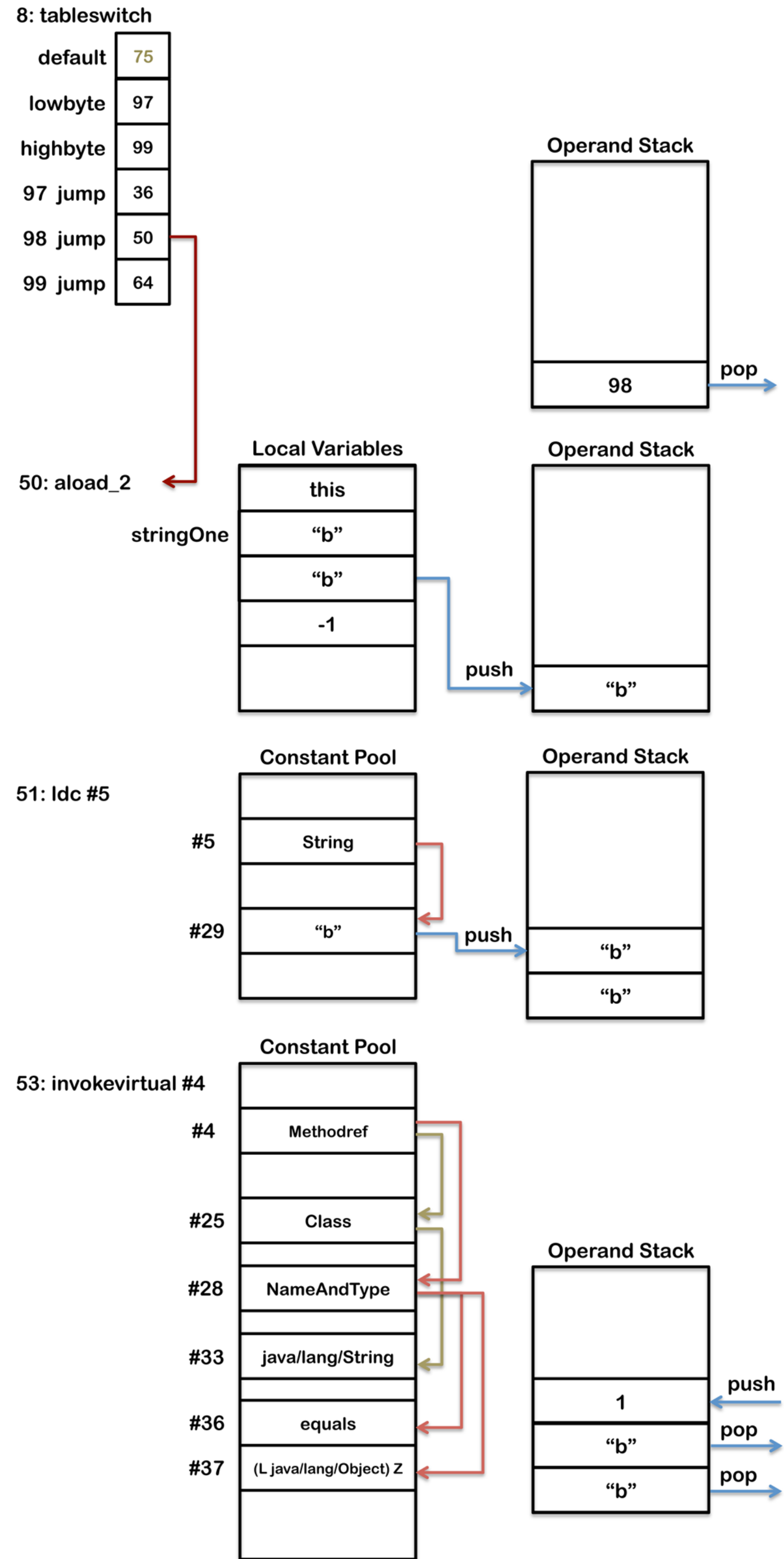
simpleSwitch(20);

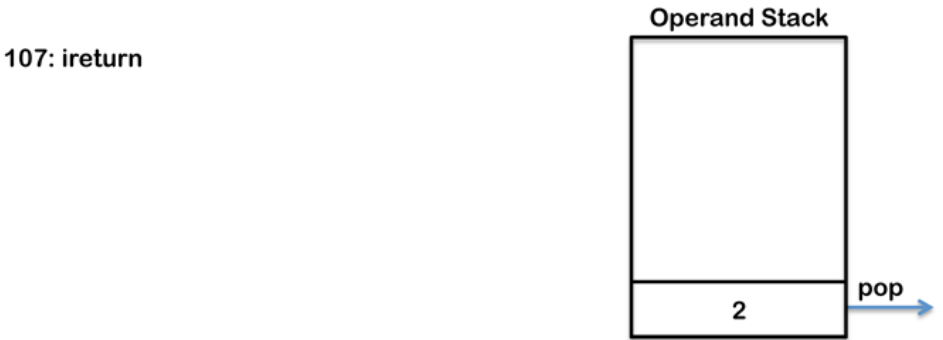
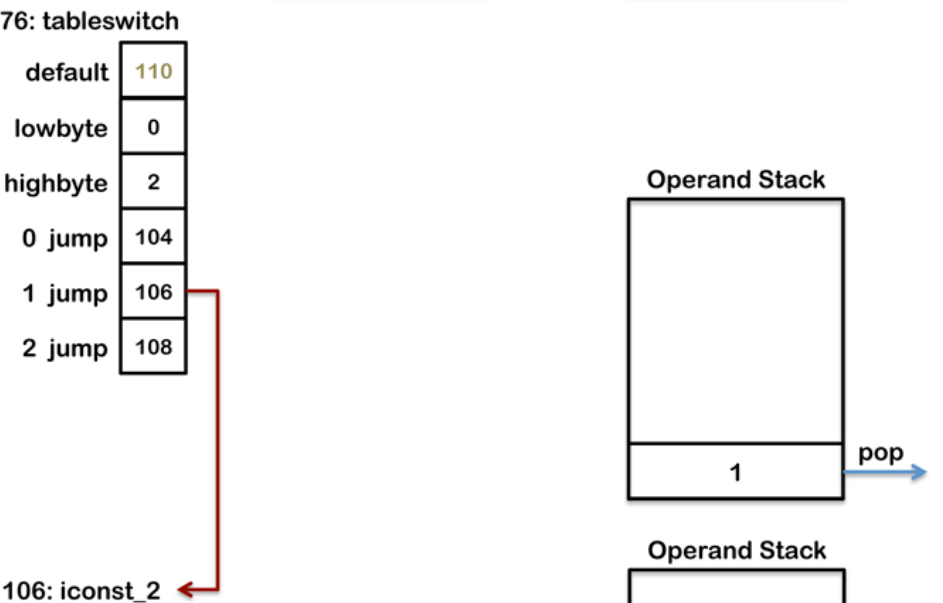
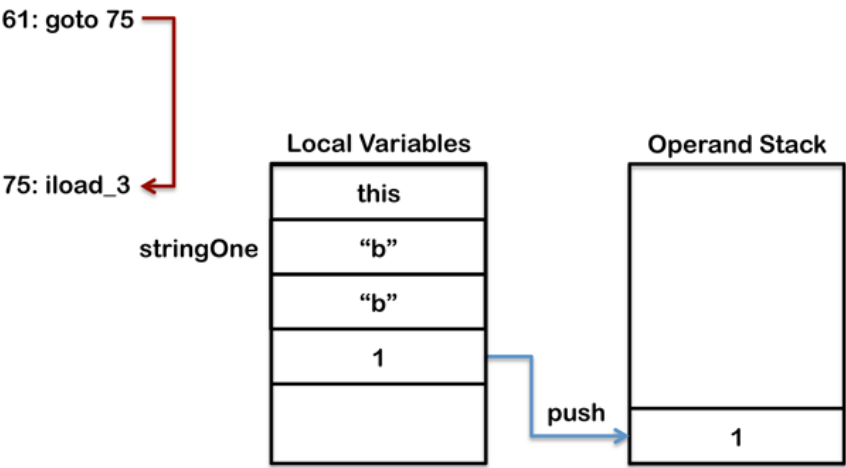
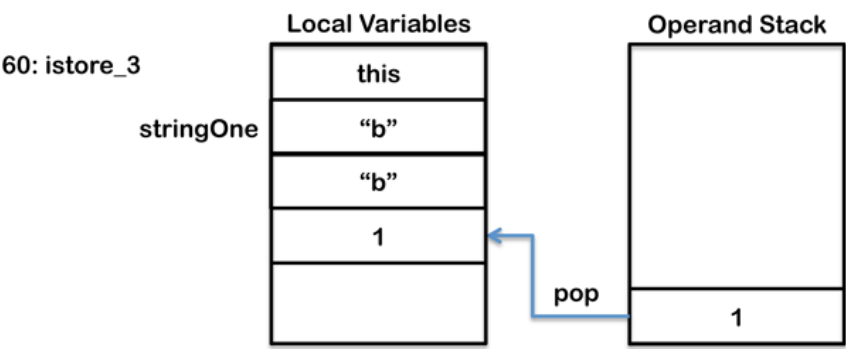
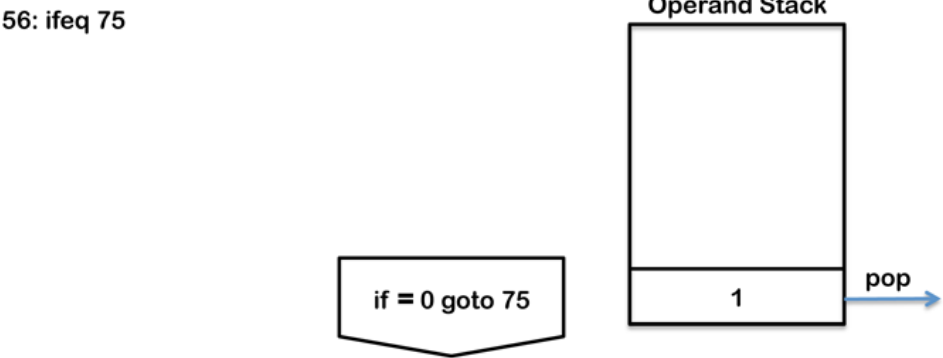




# String switch









# 循环语句

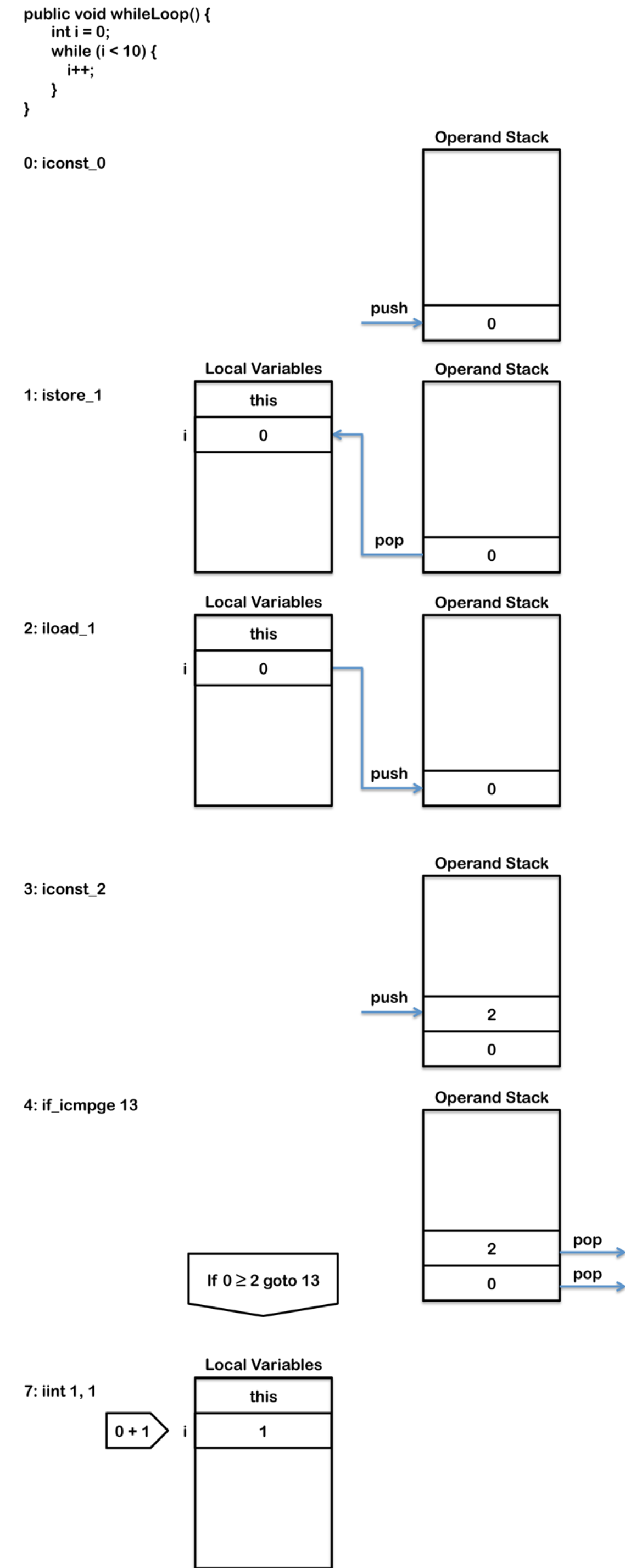
```
for (int i = 0; i < 10; i++) {  
    // 循环体  
}
```

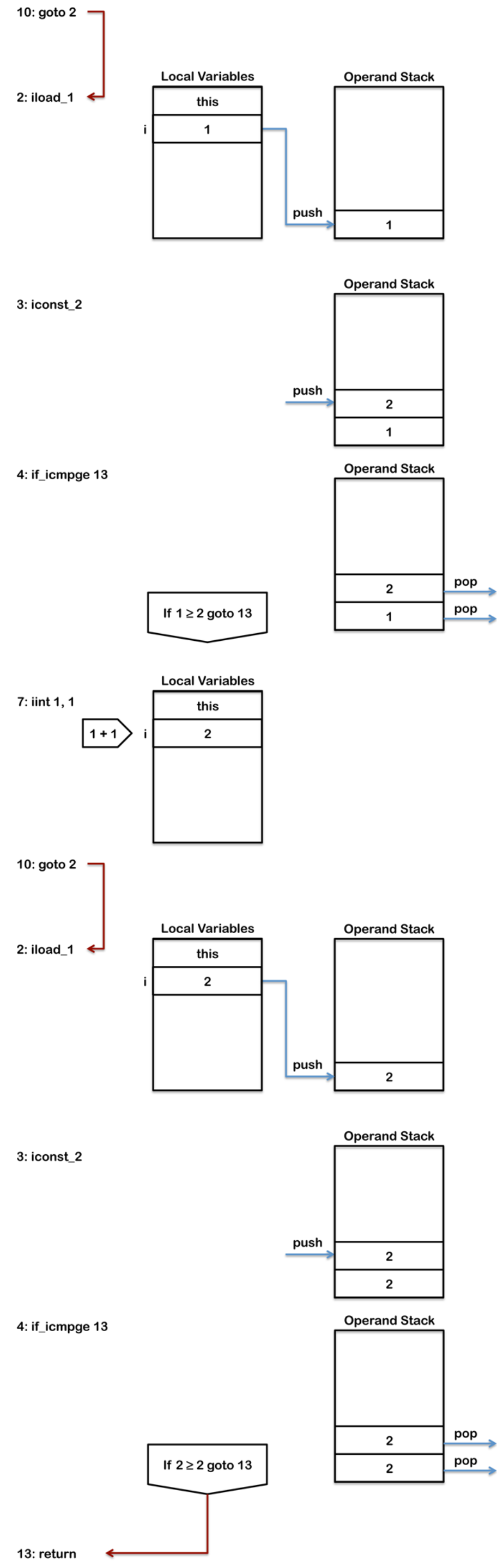
```
0: iconst_0    // 将0压入操作数栈  
1: istore_1    // 将栈顶的0弹出并存储到局部变量表位置1(i)  
2: iload_1     // 将局部变量i的值压入栈  
3: bipush 10   // 将10压入栈  
5: if_icmpge 18 // 弹出栈顶两个值比较, 如果i>=10跳转到18  
8: iinc 1, 1    // 局部变量i自增1 (不涉及操作数栈)  
11: iload_1     // 将i的值压入栈 (循环体后的i++部分)  
12: bipush 10   // 将10压入栈  
14: if_icmplt 8  // 比较i和10, 如果i<10跳转到8  
17: return     // 方法返回
```

# for

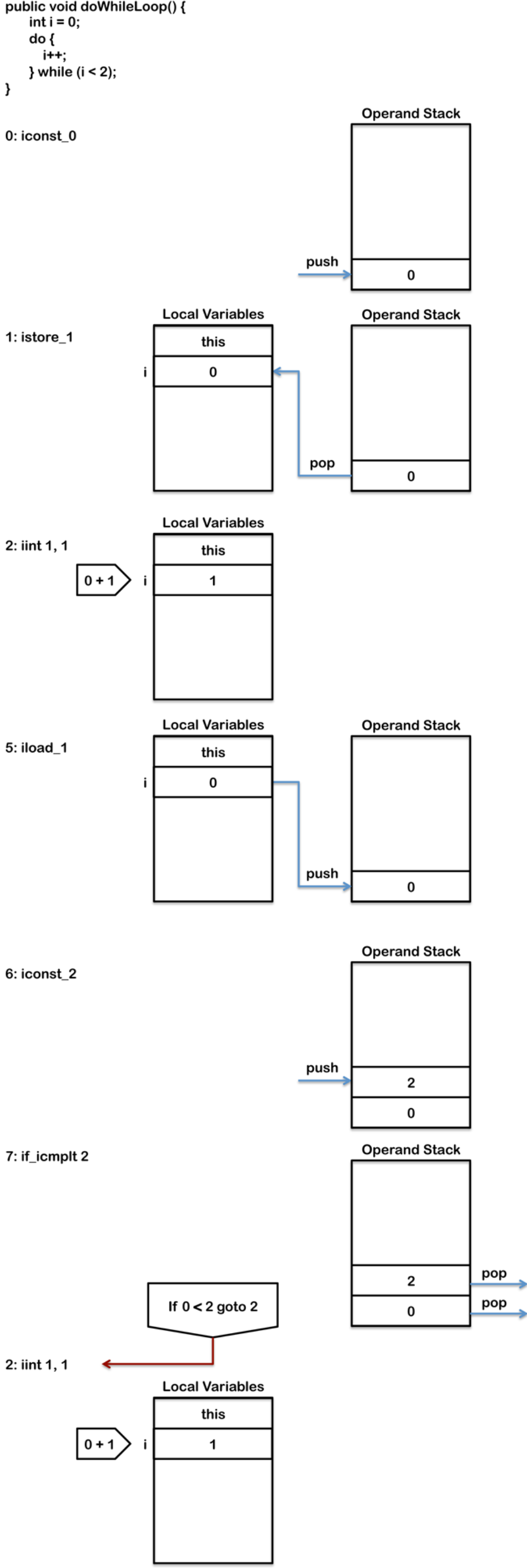


while

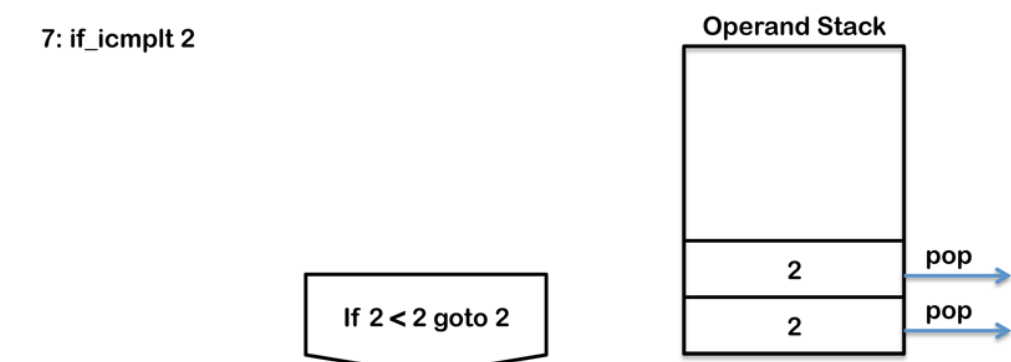
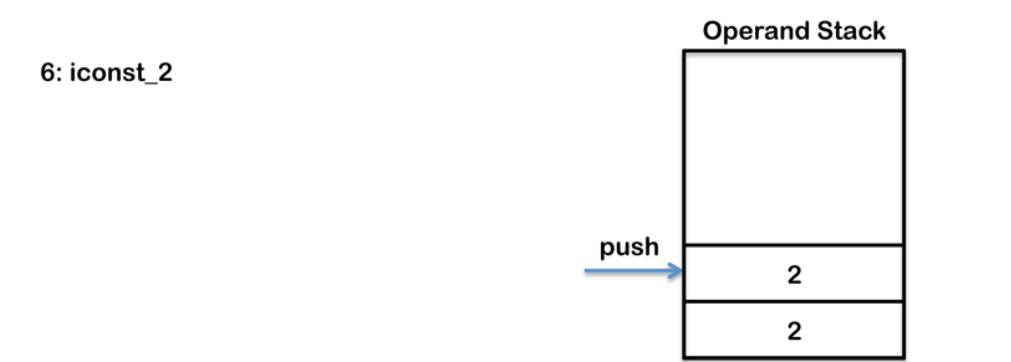
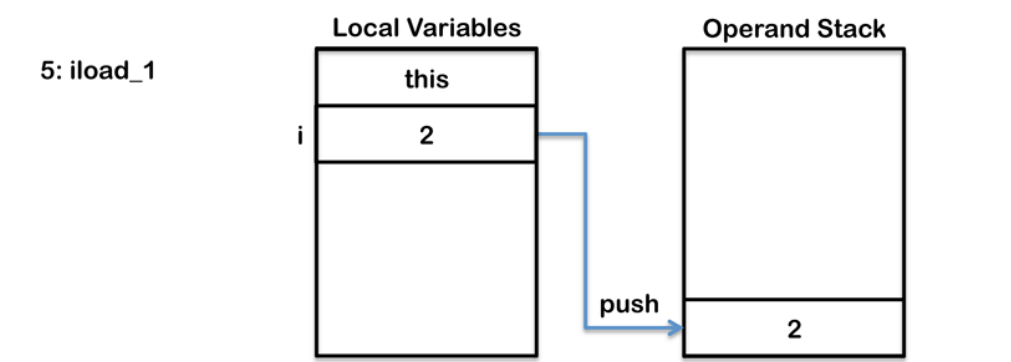
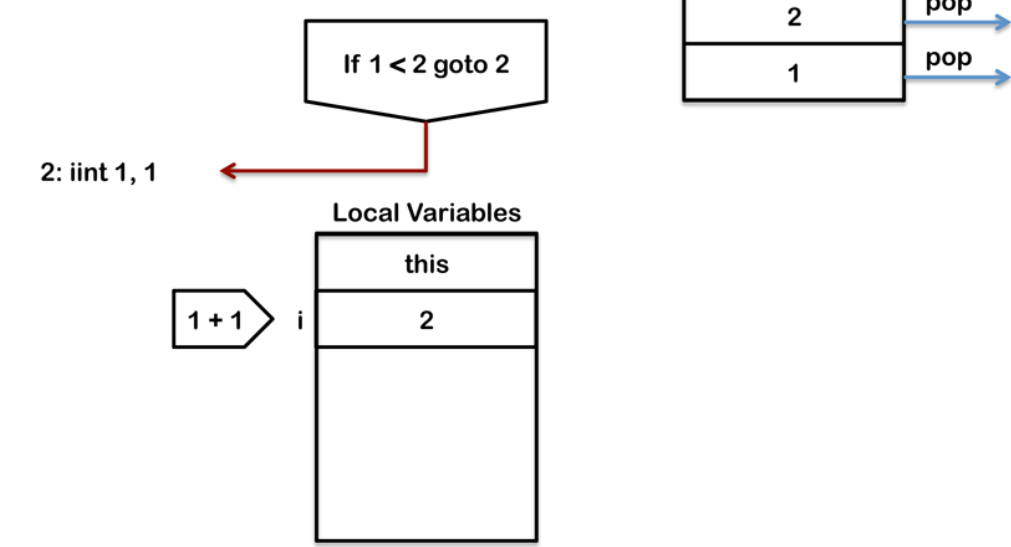
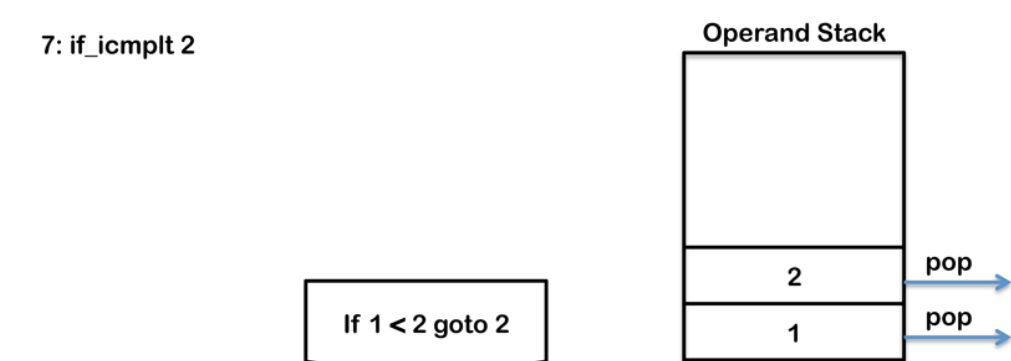
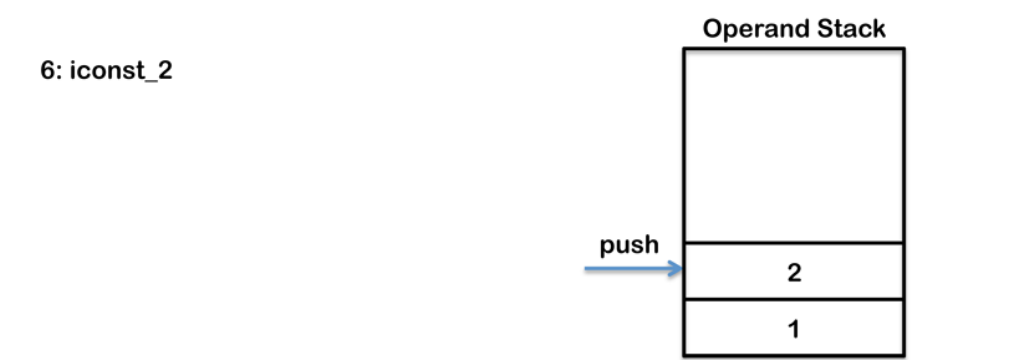
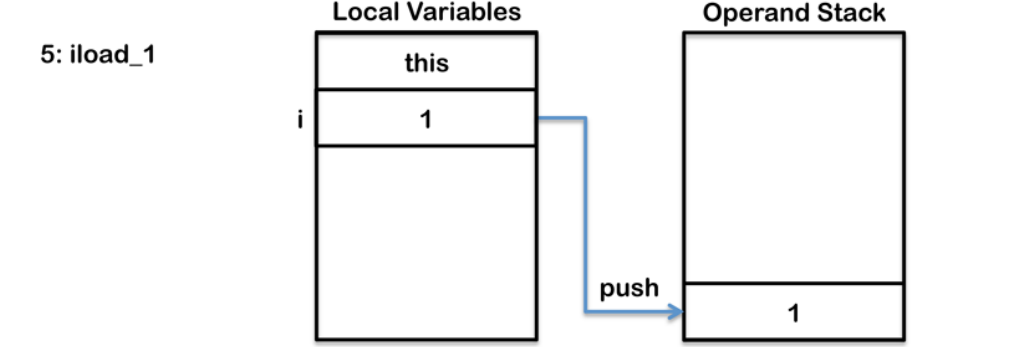




# do while







10: return